Modern C++ for Python Programmers

From Absolute Beginner to Systems, Graphics, and Game Development

Target Standard: C++23 (notes on C++11/14/17/20 throughout) **Prerequisites:** Comfortable Python; zero C++ assumed

Every concept is taught from zero and contrasted with how Python does the same thing.

Table of Contents

C++ Meets C, eBPF, and Go

Appendices - A. Setting Up Your Toolchain - B. Compiler Flags Reference - C. Python → C++ Idiom Cheat Sheet - D. Common Mistakes and How to Debug Them

---

Part I — Foundations

Chapter 1: The Compilation Model & Your First Program

How Python Runs Your Code

When you run `python hello.py`, the CPython interpreter reads your source file, parses it into an AST, compiles it to bytecode, and then executes that bytecode in a virtual machine — all in one step, invisibly. You never think about it.

```
# hello.py
print("Hello, world!")
```

```
$ python hello.py
Hello, world!
```

Python trades speed for convenience. The interpreter does a huge amount of work at runtime: looking up variable names in dictionaries, figuring out types on the fly, managing memory with a garbage collector.

How C++ Works

C++ is a compiled language. Your source code is transformed into native machine code *before* it ever runs. There is no virtual machine. The CPU executes your program directly. This is why C++ programs are fast — and why they demand more from you up front.

The transformation happens in three stages:

```
Source (.cpp) → Preprocessor → Compiler → Object file (.o)
Object files + Libraries → Linker → Executable
```

Preprocessor: Handles `#include`, `#define`, and other `#` directives. Purely textual substitution — it doesn't understand C++ at all.

Compiler: Reads the preprocessed source, checks types, and emits machine code into an *object file*. One `.cpp` → one `.o`.

Linker: Combines all the object files (yours and the standard library's) into a single executable. Resolves references between files — when `main.cpp` calls a function defined in `math.cpp`, the linker is what wires them together.

Your First C++ Program

```cpp
#include <iostream>

int main() {
    std::cout << "Hello, world!" << std::endl;
    return 0;
}
```

Save this as `hello.cpp`. Then compile and run:

```
$ g++ -std=c++23 -o hello hello.cpp
$ ./hello
Hello, world!
```

Let's break down every token:

- `#include <iostream>` — tells the preprocessor to paste the contents of the `iostream` header here. This gives you access to `std::cout`. The angle brackets mean "look in the system include path."
- `int main()` — the entry point of every C++ program. It must return `int`. The OS reads that return value; `0` means success.
- `std::cout` — the standard output stream. `std` is a *namespace* (a named grouping of identifiers). `cout` lives inside it.
- `<<` — the *stream insertion operator*. It sends the string to `cout`.
- `std::endl` — flushes the stream and writes a newline. In tight loops, prefer `'\n'` (just a newline, no flush) for performance.
- `return 0;` — exit code. You can actually omit this in `main` and the compiler inserts it, but being explicit is clearer.

The Compiler is Your Friend

Unlike Python, which tells you about most errors at runtime, C++ tells you at compile time. Type mismatches, calling functions that don't exist, using variables before declaring them — all caught before the program runs. This feels annoying at first and becomes invaluable.

```
# Ask the compiler to tell you everything it knows
$ g++ -std=c++23 -Wall -Wextra -o hello hello.cpp
```

`-Wall` and `-Wextra` enable warnings. A warning is the compiler saying "this compiles, but I suspect you made a mistake." Always treat warnings as errors when learning.

Compilation Unit Model

In Python you can split code across files and `import` them freely. In C++, the model is stricter.

Each `.cpp` file is an independent *translation unit*. It is compiled in isolation. If `main.cpp` wants to call a function defined in `util.cpp`, it needs a *declaration* of that function — typically provided via a header file `util.h`. The linker then resolves the actual address of the function.

```
main.cpp   includes util.h (declaration)  →  compiled to main.o
util.cpp   implements the function        →  compiled to util.o
```

`linker: main.o + util.o → executable`

We cover this fully in Chapter 11. For now, keep everything in one file.

Key Takeaways

- Python: interpreted at runtime. C++: compiled to machine code before running.
- The pipeline is: preprocess → compile → link.
- `main()` is the entry point and must return `int`.
- `std::cout <<` is how you print to the terminal.
- The compiler catches type errors before your program ever runs.

Chapter 2: Variables, Types, and the Static Type System

Python's Dynamic Types

In Python, variables are labels that point to objects. The object carries its type. You can rebind a label to anything:

```
x = 42        # x points to an int object
x = "hello"   # now x points to a str object — perfectly legal
x = [1, 2, 3] # now a list — still fine
```

The type lives with the data, not the variable name. Python figures out what operations are valid at runtime.

C++'s Static Types

In C++, a variable *is* a named region of memory with a fixed type determined at compile time. The type tells the compiler how many bytes to reserve and how to interpret those bytes.

```
int x = 42;      // x is always an int. Forever.
// x = "hello";  // COMPILE ERROR — cannot assign string to int
```

This is not a limitation; it is a design choice that enables the compiler to catch entire classes of bugs before your code runs and to generate maximally efficient machine code.

Fundamental Types

C++ Type	Python Equivalent	Size (typical)	Notes
`bool`	`bool`	1 byte	`true` / `false`
`char`	`str` (1 char)	1 byte	Holds one ASCII character
`int`	`int`	4 bytes	−2,147,483,648 to 2,147,483,647
`long`	`int`	4 or 8 bytes	Platform-dependent
`long long`	`int`	8 bytes	Guaranteed 64-bit
`float`	`float`	4 bytes	Single precision
`double`	`float`	8 bytes	Double precision (prefer this)
`std::string`	`str`	varies	In `<string>` header

Python's `int` is arbitrary precision. C++'s `int` wraps around (with undefined behavior for signed overflow — more on that later). If you need big integers in C++, use a library.

Declaring and Initializing Variables

```
int age;          // declared but UNINITIALIZED — value is garbage, reading it is UB
int age = 25;     // copy initialization
int age(25);      // direct initialization
int age{25};      // uniform/brace initialization (prefer this — prevents narrowing)
```

Always initialize your variables. Uninitialized variables are one of C++'s most notorious pitfalls. The compiler may not warn you; the program may silently use whatever bytes happened to be in memory.

Brace initialization `{}` is safest because it prevents *narrowing conversions*:

```
int x{3.7};  // COMPILE ERROR — 3.7 can't fit in int without losing data
int x = 3.7; // silently truncates to 3. No error. Bug.
```

Type Inference with `auto`

You don't always have to write the type — you can let the compiler deduce it:

```
auto x = 42;         // int
auto y = 3.14;       // double
auto z = true;       // bool
auto s = std::string{"hello"};  // std::string
```

`auto` does not make C++ dynamically typed. The type is still fixed at compile time — `auto` just saves you from writing it. Use it when the type is obvious from the right-hand side.

`sizeof` and Memory

Every type occupies a specific number of bytes. You can query it:

```
#include <iostream>

int main() {
    std::cout << sizeof(int)    << "\n";  // 4
    std::cout << sizeof(double) << "\n";  // 8
    std::cout << sizeof(bool)   << "\n";  // 1
    std::cout << sizeof(char)   << "\n";  // 1
}
```

This matters because C++ gives you control over how much memory your data structures consume. A game storing 10 million particles cares whether each one uses 4 bytes or 8.

Fixed-Width Integer Types

Avoid `int` when you need a specific size. Use the types from `<cstdint>`:

```cpp
#include <cstdint>

int32_t  a = 42;   // exactly 32 bits, signed
uint64_t b = 0;    // exactly 64 bits, unsigned
int8_t   c = 127;  // exactly 8 bits, signed
```

These are aliases that map to the right platform type automatically.

Constants

```cpp
const double PI = 3.14159265358979;  // runtime constant — cannot be reassigned
constexpr int MAX = 100;             // compile-time constant — value known at compile time
```

`constexpr` is stronger: the value must be computable at compile time. The compiler can use it in places a `const` cannot (array sizes, template arguments). Prefer `constexpr` for values that are truly constant.

Type Casting

Python does implicit coercion freely. C++ requires explicit casts when converting between types:

```cpp
int a = 10;
int b = 3;
double result = static_cast<double>(a) / b;  // 3.333...
double wrong  = a / b;                        // 3.0 — integer division first!
```

`static_cast<T>(x)` is the C++ way to convert. It is checked at compile time. Avoid C-style casts like `(double)a` — they are less safe and harder to grep for.

Key Takeaways

- Variables have a fixed type set at compile time. The type cannot change.
- Always initialize variables, preferably with `{}`.
- `auto` lets the compiler infer the type — the variable is still statically typed.
- Use `const` and `constexpr` for values that don't change.
- `static_cast<T>` is the safe way to convert between types.

Chapter 3: Operators and Expressions

Most Operators Are Familiar

C++ inherited most of its operators from C, and Python borrowed many of them. Addition, subtraction, multiplication, comparison — they work as you expect:

```cpp
# Python
x = 10 + 3   # 13
y = 10 - 3   # 7
z = 10 * 3   # 30
w = 10 / 3   # 3.333... (float division)
```

```python
v = 10 // 3  # 3        (integer division)
m = 10 % 3   # 1        (modulo)
p = 2 ** 8   # 256      (exponentiation)
```

```cpp
// C++
int x = 10 + 3;   // 13
int y = 10 - 3;   // 7
int z = 10 * 3;   // 30
double w = 10.0 / 3.0;  // 3.333...
int v = 10 / 3;   // 3  — integer division when BOTH operands are int
int m = 10 % 3;   // 1
// No ** operator — use std::pow(2.0, 8.0) from <cmath>
```

The critical difference: **division in C++ depends on operand types**. When both operands are integers, `/` does integer division. This bites beginners constantly.

```cpp
double bad  = 1 / 2;       // 0.0! — integer division, then converted
double good = 1.0 / 2;     // 0.5
double also = static_cast<double>(1) / 2;  // 0.5
```

Compound Assignment

These work identically to Python:

```cpp
int x = 10;
x += 3;   // x = 13
x -= 2;   // x = 11
x *= 4;   // x = 44
x /= 2;   // x = 22
x %= 7;   // x = 1
```

Increment and Decrement

C++ has `++` and `--` operators that Python lacks:

```cpp
int x = 5;
x++;   // post-increment: returns x (5), then adds 1. x is now 6.
++x;   // pre-increment:  adds 1 first, then returns x. x is now 7.
x--;   // post-decrement: returns x (7), then subtracts 1. x is now 6.
--x;   // pre-decrement.  x is now 5.
```

In a standalone statement `x++` and `++x` do the same thing. The difference matters in expressions:

```cpp
int a = 5;
int b = a++;  // b = 5, a = 6 (post: use then increment)
int c = ++a;  // c = 7, a = 7 (pre: increment then use)
```

Prefer `++x` (pre-increment) as a habit — it's never slower and sometimes faster (matters for iterators).

Comparison Operators

```
int a = 5, b = 10;
bool eq  = (a == b);  // false
bool neq = (a != b);  // true
bool lt  = (a < b);   // true
bool gt  = (a > b);   // false
bool lte = (a <= b);  // true
bool gte = (a >= b);  // false
```

C++20 adds the three-way comparison operator `<=>` (the "spaceship operator") that returns an ordering value. We cover it when discussing operator overloading in Chapter 22.

Logical Operators

```
bool x = true, y = false;
bool a = x && y;   // AND: false  (Python: x and y)
bool b = x || y;   // OR:  true   (Python: x or y)
bool c = !x;       // NOT: false  (Python: not x)
```

Short-circuit evaluation works the same as Python: `&&` stops at the first `false`, `||` stops at the first `true`.

Bitwise Operators

These operate on individual bits of integers. Python has them too, but C++ programmers use them far more often (hardware, flags, graphics).

```
int a = 0b1010;  // 10
int b = 0b1100;  // 12

int and_  = a & b;   // 0b1000 = 8   (AND each bit)
int or_   = a | b;   // 0b1110 = 14  (OR each bit)
int xor_  = a ^ b;   // 0b0110 = 6   (XOR each bit)
int not_  = ~a;      // flips all bits (result is -11 for signed int)
int lsh   = a << 1;  // 0b10100 = 20 (shift left = multiply by 2)
int rsh   = a >> 1;  // 0b0101  = 5  (shift right = divide by 2)
```

A common pattern: use bits as boolean flags.

```
const int FLAG_VISIBLE  = 1 << 0;  // 0001
const int FLAG_COLLIDABLE = 1 << 1;  // 0010
const int FLAG_ACTIVE   = 1 << 2;  // 0100

int entity_flags = FLAG_VISIBLE | FLAG_ACTIVE;  // 0101

bool is_visible = entity_flags & FLAG_VISIBLE;  // true
entity_flags |= FLAG_COLLIDABLE;  // set a flag
entity_flags &= ~FLAG_ACTIVE;     // clear a flag
```

Operator Precedence

Like Python, multiplication binds tighter than addition, etc. When in doubt, use parentheses — they cost nothing and prevent bugs.

```cpp
int x = 2 + 3 * 4;    // 14, not 20
int y = (2 + 3) * 4;  // 20
```

Key Takeaways

- `/` is integer division when both operands are integers. Cast one to `double` if you want a decimal result.
- `++x` (pre-increment) and `x++` (post-increment) differ in expressions. Prefer `++x`.
- Bitwise operators (`&`, `|`, `^`, `~`, `<<`, `>>`) are common in C++ for flags and low-level code.
- When precedence is unclear, parenthesize.

Chapter 4: Control Flow: Branching and Loops

`if` / `else if` / `else`

Syntax is similar to Python but uses braces instead of indentation, and conditions must be in parentheses:

```python
# Python
score = 85
if score >= 90:
    print("A")
elif score >= 80:
    print("B")
else:
    print("C")
```

```cpp
// C++
int score = 85;
if (score >= 90) {
    std::cout << "A\n";
} else if (score >= 80) {
    std::cout << "B\n";
} else {
    std::cout << "C\n";
}
```

You can omit braces for single-statement bodies, but don't — it's a famous source of bugs (the Apple "goto fail" SSL vulnerability was caused by this exact mistake).

`if` with Initializer (C++17)

C++17 lets you declare a variable inside the `if` condition, scoped to just that block:

```cpp
if (int result = compute(); result > 0) {
    std::cout << "Positive: " << result << "\n";
} else {
    std::cout << "Non-positive: " << result << "\n";
}
// result is not accessible here
```

This is useful for functions that return status codes or optional values.

`switch`

For branching on a single integer (or enum) value, `switch` is more efficient than a chain of `if/else`:

```cpp
int day = 3;
switch (day) {
    case 1:
        std::cout << "Monday\n";
        break;
    case 2:
        std::cout << "Tuesday\n";
        break;
    case 3:
    case 4:
        std::cout << "Wednesday or Thursday\n";
        break;
    default:
        std::cout << "Other\n";
        break;
}
```

Always include `break` unless you intentionally want fallthrough. Accidental fallthrough is one of C's most notorious bugs. In C++17 you can annotate intentional fallthrough with `[[fallthrough]]` to silence compiler warnings.

`switch` works only on integral types and enums — not on strings or arbitrary objects (unlike Python's structural pattern matching).

`while` Loop

```python
# Python
x = 0
while x < 5:
    print(x)
    x += 1
```

```cpp
// C++
int x = 0;
while (x < 5) {
    std::cout << x << "\n";
```

```
    ++x;
}
```

`do-while` Loop

Executes the body at least once before checking the condition. Python has no equivalent.

```
int x = 0;
do {
    std::cout << x << "\n";
    ++x;
} while (x < 5);
```

Useful for input validation: ask for input, then check if it's valid.

`for` Loop — C-Style

The classic C-style `for` loop has three parts: initializer, condition, increment:

```
for (int i = 0; i < 10; ++i) {
    std::cout << i << "\n";
}
```

This is the equivalent of Python's `for i in range(10)`. The variable `i` exists only inside the loop.

You can have multiple initializers and increments:

```
for (int i = 0, j = 10; i < j; ++i, --j) {
    std::cout << i << " " << j << "\n";
}
```

Range-Based `for` Loop (C++11)

Equivalent to Python's `for x in iterable`:

```
# Python
numbers = [1, 2, 3, 4, 5]
for n in numbers:
    print(n)
```

```
// C++
#include <vector>
std::vector<int> numbers = {1, 2, 3, 4, 5};
for (int n : numbers) {
    std::cout << n << "\n";
}
```

If you want to modify the elements, use a reference (Chapter 6):

```
for (int& n : numbers) {
    n *= 2;  // doubles each element in place
}
```

If you don't want to copy large objects, use `const` reference:

```
for (const int& n : numbers) {
    std::cout << n << "\n";  // no copy, no modification
}
```

Or just `auto&`:

```
for (auto& n : numbers) {
    n *= 2;
}
```

`break` and `continue`

Work exactly like Python:

```
for (int i = 0; i < 10; ++i) {
    if (i == 5) break;     // exit the loop entirely
    if (i % 2 == 0) continue;  // skip to next iteration
    std::cout << i << "\n";  // prints 1, 3
}
```

Ternary Operator

C++ has a ternary operator Python lacks (Python has a different form):

```
# Python
result = "yes" if condition else "no"
```

```
// C++
std::string result = condition ? "yes" : "no";
```

They're equivalent. In C++, the ternary is an *expression* — it produces a value, so you can use it inline.

Key Takeaways

- Braces around loop/if bodies are technically optional but always required by good practice.
- `switch` is fast for integral values; don't forget `break`.
- Range-based `for` is the idiomatic way to iterate containers. Use `auto&` to avoid copies.
- `do-while` runs the body at least once — useful for "try once, then check" patterns.

Chapter 5: Functions, Overloading, and Declarations vs Definitions

Functions in Python vs C++

```
# Python
def add(a, b):
    return a + b
```

```cpp
// C++
int add(int a, int b) {
    return a + b;
}
```

The differences: 1. You must declare the **return type** (`int`). 2. Each parameter must have a **type** (`int a`). 3. The compiler verifies at every call site that you're passing the right types.

If a function returns nothing, its return type is `void`:

```cpp
void greet(std::string name) {
    std::cout << "Hello, " << name << "\n";
    // no return statement needed
}
```

Declarations vs Definitions

This is a concept Python doesn't have, and it trips up every beginner.

A **definition** is the actual function body — the code. There can only be one definition per function in a program.

A **declaration** (also called a *prototype*) is just a statement telling the compiler "this function exists, here is its signature." No body. It ends in `;`.

```cpp
// Declaration (prototype) — just a promise to the compiler
int add(int a, int b);

int main() {
    int result = add(3, 4);  // compiler knows the signature — OK
    std::cout << result << "\n";
    return 0;
}

// Definition — the actual code
int add(int a, int b) {
    return a + b;
}
```

Why does this matter? Because C++ compiles files from top to bottom. Without a declaration, the compiler doesn't know `add` exists when it encounters the call in `main`. Declarations go in header files (`.h`), definitions go in source files (`.cpp`). Chapter 11 covers this in detail.

Parameters and Arguments

Pass by value (the default) copies the argument. The function gets its own copy; changes don't affect the caller.

```cpp
void double_it(int x) {
    x *= 2;          // modifies the local copy
}
```

```cpp
int main() {
    int n = 5;
    double_it(n);
    std::cout << n;   // still 5
}
```

This is the same as Python's behavior for immutable objects (ints, strings). To modify the caller's variable, use references (Chapter 6).

Default Parameters

Like Python, C++ supports default parameter values:

```python
# Python
def connect(host, port=8080):
    ...
```

```cpp
// C++
void connect(std::string host, int port = 8080) {
    // ...
}

connect("localhost");        // port = 8080
connect("localhost", 9000);  // port = 9000
```

Default parameters must come at the *end* of the parameter list. You can't have `void f(int a = 0, int b)` — that would make calling ambiguous.

Function Overloading

C++ lets you define multiple functions with the **same name** but **different parameter types**. The compiler picks the right one based on the arguments at the call site.

```cpp
int    add(int a, int b)       { return a + b; }
double add(double a, double b) { return a + b; }
std::string add(std::string a, std::string b) { return a + b; }

int main() {
    add(1, 2);           // calls int version
    add(1.0, 2.0);       // calls double version
    add("hi", "there");  // calls string version
}
```

Python doesn't have overloading — you'd use default parameters or `*args`. C++'s overloading is resolved entirely at compile time, producing no runtime cost.

The compiler matches calls to overloads using a set of rules. Exact matches are preferred; then conversions. If two overloads are equally good matches, it's a compile error (ambiguous call).

`inline` Functions

For very short functions, the overhead of a function call (push arguments, jump, return) can be significant. Declaring a function `inline` hints to the compiler that it should paste the function body directly at the call site:

```cpp
inline int square(int x) { return x * x; }
```

Modern compilers are good at deciding this themselves (even without `inline`), but `inline` is required when you define a function in a header file to avoid "multiple definition" linker errors. More on this in Chapter 11.

Returning Multiple Values

Python can return a tuple. C++ can too, using `std::tuple` or a struct, but the cleaner C++17 way is structured bindings:

```python
# Python
def minmax(lst):
    return min(lst), max(lst)

lo, hi = minmax([3, 1, 4, 1, 5])
```

```cpp
// C++
#include <algorithm>
#include <vector>
#include <tuple>

std::pair<int, int> minmax(std::vector<int>& v) {
    auto [lo, hi] = std::minmax_element(v.begin(), v.end());
    return {*lo, *hi};
}

int main() {
    std::vector<int> v = {3, 1, 4, 1, 5};
    auto [lo, hi] = minmax(v);
    std::cout << lo << " " << hi << "\n";
}
```

We cover structured bindings fully in Chapter 33.

Key Takeaways

- Every parameter and the return type must be explicitly typed.
- A declaration is a signature with `;`. A definition has a body. You need a declaration before any call site.
- Pass by value copies. The caller's variable is unaffected. Use references (Chapter 6) to modify the caller's data.
- Overloading lets multiple functions share a name — the compiler picks based on argument types.
- Default parameters must be at the end.

Part II — Core C++ (the part that isn't like Python)

Chapter 6: References — Aliases for Variables

The Problem References Solve

In Python, when you pass an object to a function, you're passing a reference automatically. The function can mutate mutable objects:

```python
def append_zero(lst):
    lst.append(0)  # modifies the original list

nums = [1, 2, 3]
append_zero(nums)
print(nums)  # [1, 2, 3, 0]
```

In C++, by default everything is copied. If you want a function to modify the caller's variable, you need an explicit reference.

What Is a Reference?

A reference is an alias — another name for an existing variable. It doesn't create a copy; it refers to the same memory location.

```cpp
int x = 42;
int& r = x;   // r is a reference to x — same memory, different name

r = 100;      // modifying r modifies x
std::cout << x;  // 100
```

The `&` in the type declaration means "reference to." This is different from the `&` address-of operator (Chapter 7).

References must be initialized. Unlike pointers, you can't have a reference that refers to nothing. And once bound, a reference cannot be rebound to a different variable.

```cpp
int x = 1, y = 2;
int& r = x;   // r refers to x
r = y;        // this ASSIGNS y's value to x — r still refers to x!
              // you cannot make r refer to y after initialization
```

Pass by Reference

This is the main use of references: letting functions modify the caller's variables.

```cpp
void double_it(int& x) {  // x is a reference to the caller's variable
    x *= 2;
}

int main() {
    int n = 5;
    double_it(n);          // passes n by reference
    std::cout << n;        // 10
}
```

Compare with pass by value from Chapter 5 — the parameter type is `int&` instead of `int`.

Const References

Passing large objects by value is expensive (it copies every byte). Passing by reference lets the function modify the caller's data, which may not be desired. The solution: `const` reference. It's efficient (no copy) and safe (no modification).

```cpp
// Bad: copies the entire string on every call
void print_name(std::string name) {
    std::cout << name << "\n";
}

// Good: no copy, no modification
void print_name(const std::string& name) {
    std::cout << name << "\n";
}
```

Rule of thumb: **pass by `const&` for anything larger than a word-sized type** (anything bigger than a pointer/`int`). Pass by value for small types like `int`, `double`, `bool`.

References as Return Values

Functions can return references, giving the caller direct access to a variable:

```cpp
int& get_element(std::vector<int>& v, int i) {
    return v[i];  // returns a reference to the element
}

int main() {
    std::vector<int> v = {10, 20, 30};
    get_element(v, 1) = 99;   // assigns directly to v[1]
    std::cout << v[1];         // 99
}
```

Never return a reference to a local variable. The local variable dies when the function returns, leaving a dangling reference — undefined behavior.

```cpp
int& bad_function() {
    int local = 42;
```

```
    return local;  // UNDEFINED BEHAVIOR — local dies here
}
```

References vs Pointers

You'll learn about pointers in Chapter 7. The quick comparison:

Feature	Reference	Pointer
Can be null	No	Yes
Can be rebound	No	Yes
Syntax	Feels like a value	Requires `*` and `&`
Use for	Aliases, parameters	Optional/nullable things, arrays

When you have a choice, prefer references. They're safer because they can't be null or dangling (as long as you don't return them from functions with local variables).

Key Takeaways

- A reference is an alias to an existing variable — same memory, different name.
- Use `int& param` to pass by reference so a function can modify the caller's variable.
- Use `const int& param` to pass large objects cheaply without allowing modification.
- References cannot be null and cannot be rebound. These constraints make them safer than pointers.
- Never return a reference to a local variable.

Chapter 7: Pointers and Memory Addresses

Every Variable Has an Address

Every byte of memory has a numeric address. When you declare `int x = 42`, the value 42 is stored somewhere in RAM at some address. In Python this is hidden from you. In C++ you can see and use it.

The **address-of operator** `&` gives you the address of a variable:

```
int x = 42;
std::cout << &x << "\n";  // prints something like 0x7ffee4bc3a4c
```

That hexadecimal number is the memory address — the location in RAM where `x` lives.

What Is a Pointer?

A pointer is a variable that stores a memory address. If `x` is an `int`, then `int*` is "pointer to int" — a variable that holds the address of an `int`.

```
int  x = 42;
int* p = &x;   // p stores the address of x
```

```
Memory:
Address:  0x1000   0x1008
Value:    42       0x1000
Variable: x        p
```

`p` contains the number `0x1000` (the address of `x`). When you look at `p`, you see an address. When you *follow* the pointer, you see the value at that address.

Dereferencing

The **dereference operator** `*` follows a pointer to get the value it points to:

```cpp
int  x = 42;
int* p = &x;

std::cout << p;   // prints the address (e.g. 0x1000)
std::cout << *p;  // prints 42 — the value at that address
```

You can also write through a pointer:

```cpp
*p = 100;         // sets x to 100 via the pointer
std::cout << x;   // 100
```

This is what Python does automatically for mutable objects. C++ makes it explicit.

`nullptr`

A pointer that doesn't point to anything should be set to `nullptr` (C++11). Never leave a pointer uninitialized.

```cpp
int* p = nullptr;   // safe: p points to nothing

if (p != nullptr) { // always check before dereferencing
    std::cout << *p;
}
// or equivalently:
if (p) {
    std::cout << *p;
}
```

Dereferencing a null pointer is undefined behavior — usually a crash (segmentation fault). Always check.

Pointer Arithmetic

Pointers can be incremented, and they advance by the size of the pointed-to type:

```cpp
int arr[3] = {10, 20, 30};
int* p = arr;    // points to arr[0]

std::cout << *p;      // 10
++p;
```

```
std::cout << *p;      // 20
++p;
std::cout << *p;      // 30
```

When `p` is an `int*` and you do `p + 1`, the address advances by 4 bytes (the size of `int`), not by 1 byte. This is how C arrays work — the array name decays to a pointer to its first element.

Pointers to Pointers

A pointer can point to another pointer:

```
int   x = 42;
int*  p  = &x;   // pointer to int
int** pp = &p;   // pointer to pointer to int

std::cout << **pp;  // 42
```

This appears in C-style APIs, especially for passing pointers by reference to functions that need to update them.

`const` and Pointers

There are two separate things you can `const`: the pointer itself, or the data it points to.

```
int x = 10, y = 20;

const int* p1 = &x;   // pointer to const int — data is read-only, pointer can change
p1 = &y;              // OK — pointer can be rebound
// *p1 = 99;          // ERROR — data is read-only

int* const p2 = &x;   // const pointer to int — pointer is fixed, data can change
*p2 = 99;             // OK — can modify data
// p2 = &y;           // ERROR — pointer cannot be rebound

const int* const p3 = &x;  // both const
```

Read the declaration right-to-left: "p3 is a const pointer to a const int."

When to Use Pointers vs References

This confuses most beginners. The short rule:

- **Use references** when you always have a valid object and don't need to rebind.
- **Use pointers** when the thing might be absent (`nullptr`), when you need to rebind, or when you're working with arrays.

Modern C++ has `std::optional` (Chapter 32) for the "might not exist" case. Smart pointers (Chapter 14) replace raw pointer ownership. After Part III, you'll rarely use raw pointers for ownership.

Key Takeaways

- A pointer is a variable that stores a memory address.
- `&x` gives the address of `x`. `*p` dereferences pointer `p` to get the value.
- Initialize pointers to `nullptr` if they don't point to anything.
- Always check a pointer before dereferencing it.
- Pointer arithmetic advances by the size of the pointed-to type.
- `const int*` = data is const. `int* const` = pointer is const.

Chapter 8: The Stack and the Heap

Python Hides Memory from You

In Python, you never decide where an object lives in memory. The interpreter allocates objects on the heap, and the garbage collector frees them when they're no longer referenced. This is simple, but it comes with costs: allocation overhead, garbage collection pauses, and no control over memory layout.

Two Kinds of Memory

C++ programs use two main regions of memory:

The Stack: Fast, automatic, limited in size (typically 1–8 MB). Variables declared inside functions live here. The stack grows and shrinks automatically as functions are called and return.

The Heap: Slow(er), manually managed, limited only by available RAM. Objects allocated with `new` live here. You control when they're created and destroyed.

Stack Memory

Every local variable you've used so far has lived on the stack:

```
void foo() {
    int x = 42;       // allocated on the stack
    double y = 3.14;  // also on the stack
    // x and y are destroyed automatically when foo() returns
}
```

The stack works like a stack of plates: when you call a function, a *stack frame* is pushed with space for all its local variables. When the function returns, that frame is popped and the memory is instantly reclaimed.

Stack allocation is extremely fast — it's just moving a pointer. But the size is fixed at program start, and you can't allocate more than a few megabytes without a stack overflow.

```
void explode() {
    int huge_array[10'000'000];  // ~40 MB on the stack — stack overflow!
}
```

Heap Memory

The heap is a large pool of memory managed by the allocator (`malloc`/`free` at the C level, `new`/`delete` in C++). You request a block, use it, and return it when done.

```
int* p = new int(42);      // allocate an int on the heap, initialized to 42
std::cout << *p << "\n";   // 42
delete p;                  // return the memory to the heap
p = nullptr;               // good practice: null the pointer after deleting
```

`new` returns a pointer to the allocated memory. `delete` frees it. If you forget to `delete`, you have a **memory leak** — the memory is never returned until the program exits. In long-running programs this causes ever-growing memory usage.

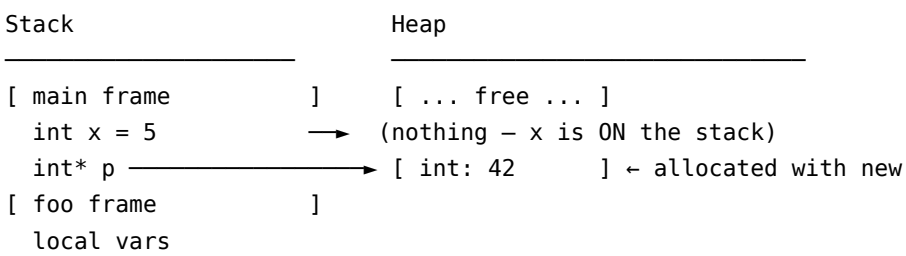
Heap Arrays

To allocate an array on the heap:

```
int n = 1000;
int* arr = new int[n];   // heap array of 1000 ints
arr[0] = 1;
arr[1] = 2;
// ...
delete[] arr;            // MUST use delete[], not delete
arr = nullptr;
```

Notice `delete[]` — it calls the destructor on each element and frees the whole array. Using `delete` (without `[]`) on an array is undefined behavior.

Visualizing the Difference



Why Does This Matter?

Python puts almost everything on the heap automatically. C++ makes you choose:

- **Stack**: Use for small, short-lived values. Variables, local structs, small arrays. Fast, automatic cleanup.
- **Heap**: Use for large data, data that outlives a function, data whose size isn't known at compile time.

But here's the secret: **in modern C++, you almost never call `new` and `delete` directly.** `std::vector`, `std::string`, and smart pointers manage heap memory for you, giving heap flexibility with stack-like safety. Chapters 10 and 14 cover this.

Stack Overflow

When functions call functions recursively too deep, the stack runs out of space:

```
void infinite() {
    infinite();  // calls itself forever — stack overflow → crash
}
```

Each call pushes a frame; you exhaust the stack in seconds.

Key Takeaways

- Stack: automatic, fast, limited (~1-8MB). Local variables live here. Freed when the function returns.
- Heap: manual, large, flexible. Allocated with `new`, freed with `delete`.
- Memory leaks occur when heap memory is never freed. They grow silently.
- `delete[]` for arrays, `delete` for single objects.
- Modern C++ wraps heap allocation in containers and smart pointers — you rarely call `new` raw.

Chapter 9: `const` Correctness

The Concept

`const` means "this value will not change." In Python you signal immutability by convention (all-caps names) or by using immutable types. C++ enforces immutability at the compiler level.

```
const int MAX_PLAYERS = 8;
MAX_PLAYERS = 10;  // COMPILE ERROR
```

This is more than a convenience — it's a design tool. When you mark something `const`, you're communicating intent to readers of the code and enabling the compiler to catch accidental modifications.

`const` Variables

```
const double PI = 3.14159265358979;
const std::string GREETING = "Hello";
```

Always initialize `const` variables at declaration — you can't assign them later.

`const` References (recap)

From Chapter 6: passing by `const` reference is the idiomatic way to pass large objects cheaply without allowing modification:

```
void print(const std::string& s) {
    std::cout << s << "\n";
    // s = "modified";  // COMPILE ERROR
}
```

`const` Member Functions

In a class (Chapter 17), a member function can be marked `const`, promising it won't modify the object:

```
class Circle {
    double radius;
public:
    Circle(double r) : radius(r) {}

    double area() const {           // this function won't modify the Circle
        return 3.14159 * radius * radius;
    }

    void set_radius(double r) {     // non-const: modifies the object
        radius = r;
    }
};
```

If you have a `const Circle`, you can only call `const` member functions on it:

```
const Circle c(5.0);
c.area();       // OK — const function
// c.set_radius(3.0);  // ERROR — would modify a const object
```

This is *const correctness*: the compiler propagates `const` through your code, and you must explicitly opt in to modifications. It sounds tedious but prevents entire classes of bugs — especially in large codebases where functions are called from many places.

`constexpr` vs `const`

`const` means "doesn't change after initialization." The value might not be known until runtime.

`constexpr` means "computed at compile time." The value is embedded directly into the machine code.

```
int n = get_user_input();  // runtime value
const int x = n;           // const, but not constexpr — value known at runtime

constexpr int ARRAY_SIZE = 100;  // known at compile time
int arr[ARRAY_SIZE];             // valid — array size must be a compile-time constant
```

```
constexpr int square(int x) { return x * x; }
constexpr int s = square(5);  // computed at compile time: s = 25
```

When a function is `constexpr`, it can be evaluated at compile time when given compile-time arguments, and at runtime otherwise. This is a core C++ performance tool — Chapter 34 covers it deeply.

Mutable Members

Sometimes you need one member of a `const` object to be modifiable — for example, a cache or a mutex. The `mutable` keyword overrides `const` for that member:

```
class ExpensiveComputation {
    mutable int cache = -1;  // OK to modify even in const functions
public:
    int result() const {
        if (cache == -1) {
            cache = compute();  // populates cache on first call
        }
        return cache;
    }
};
```

Use `mutable` sparingly — it's a hole in const correctness.

`const` Propagation

When you have a `const` reference or pointer, calling a non-const method on the object through it is an error. This forces you to be explicit about what can change:

```
void display(const std::vector<int>& v) {
    for (const int& x : v) {  // also const — good habit
        std::cout << x << " ";
    }
    // v.push_back(99);  // ERROR — v is const
}
```

Key Takeaways

- `const` is enforced by the compiler, not just a convention.
- Pass large objects as `const T&` to avoid copies while preventing modification.
- Mark member functions `const` when they don't modify the object. This enables calling them on `const` objects.
- `constexpr` goes further: the value must be computable at compile time, enabling use in array sizes and template arguments.
- Build `const` correctness from the start. Retrofitting it into an existing codebase is painful.

Chapter 10: Arrays, `std::vector`, and `std::string`

C-Style Arrays (and Why to Avoid Them)

C++ inherits raw arrays from C. They're fast but treacherous:

```
int arr[5] = {1, 2, 3, 4, 5};
arr[0] = 10;
std::cout << arr[2];  // 3
```

```cpp
// Danger: no bounds checking
arr[10] = 99;  // undefined behavior — writes past the array
```

The biggest problem: raw arrays decay to pointers and lose their size information. `sizeof(arr)` gives the total bytes, not the element count. You have to track the size yourself.

```cpp
void print_array(int* arr, int size) {  // must pass size separately
    for (int i = 0; i < size; ++i) std::cout << arr[i] << " ";
}
```

Use raw arrays only when you need compile-time fixed-size arrays with no overhead (SIMD, hardware buffers, embedded systems). For everything else, use `std::array` or `std::vector`.

`std::array` — Fixed-Size with Safety

`std::array<T, N>` is a zero-overhead wrapper around a C array that knows its size and works with standard algorithms:

```cpp
#include <array>

std::array<int, 5> arr = {1, 2, 3, 4, 5};
std::cout << arr.size();  // 5
arr.at(10);               // throws std::out_of_range (unlike raw arr[10])
```

Use `std::array` when the size is known at compile time. It lives on the stack, has no heap allocation, and is just as fast as a C array.

`std::vector` — Python's `list`

`std::vector<T>` is C++'s closest equivalent to Python's list. It's a dynamic array that lives on the heap and resizes automatically.

```python
# Python
nums = [1, 2, 3]
nums.append(4)
nums.pop()
print(len(nums))
```

```cpp
// C++
#include <vector>

std::vector<int> nums = {1, 2, 3};
nums.push_back(4);   // append
nums.pop_back();     // remove last
std::cout << nums.size();  // 3
```

Common operations:

```cpp
std::vector<int> v = {10, 20, 30, 40, 50};

v[2]          // 30 — no bounds check (fast)
v.at(2)       // 30 — bounds-checked (throws on out-of-range)
```

```cpp
v.front()     // 10 — first element
v.back()      // 50 — last element
v.size()      // 5  — number of elements
v.empty()     // false
v.clear()     // remove all elements
v.push_back(60)  // append
v.pop_back()     // remove last

// insert at position 2
v.insert(v.begin() + 2, 99);

// erase element at position 2
v.erase(v.begin() + 2);

// resize to 10 elements (new ones value-initialized)
v.resize(10);
```

How `std::vector` Manages Memory

A vector has three quantities: **size** (elements in use), **capacity** (elements allocated), and the data pointer.

When `push_back` fills the capacity, the vector allocates a new, larger block (typically 2× capacity), copies all elements, and frees the old block. This is O(1) amortized but occasionally causes a big copy.

```cpp
std::vector<int> v;
v.reserve(1000);   // pre-allocate for 1000 elements — prevents reallocations
```

Use `reserve()` when you know in advance how many elements you'll have. It's one of the most impactful performance micro-optimizations for vectors.

`std::string`

`std::string` is `std::vector<char>` with string-specific operations.

```python
# Python
s = "Hello"
s += ", world"
print(len(s))
print(s.upper())
print(s[0])
```

```cpp
// C++
#include <string>

std::string s = "Hello";
s += ", world";
std::cout << s.size() << "\n";     // 12
std::cout << s[0] << "\n";          // 'H'
```

```cpp
std::cout << s.substr(0, 5) << "\n"; // "Hello"

// Find and replace
size_t pos = s.find("world");       // returns position or std::string::npos
if (pos != std::string::npos) {
    s.replace(pos, 5, "C++");
}
```

Converting between `std::string` and numbers:

```cpp
int n = std::stoi("42");           // string to int
double d = std::stod("3.14");      // string to double
std::string s = std::to_string(42); // int to string
```

String Literals and `std::string_view`

A string literal like `"hello"` is actually a `const char*` — a C-style string. Converting it to `std::string` copies the data.

C++17 introduced `std::string_view`: a non-owning view into any string-like data (a `std::string`, a string literal, a buffer). Use it for read-only string parameters to avoid copies:

```cpp
#include <string_view>

void print(std::string_view s) {   // works with std::string AND string literals
    std::cout << s << "\n";
}

print("hello");         // no copy — string_view points to the literal
print(my_std_string);   // no copy — string_view points into the std::string
```

Key Takeaways

- Raw C arrays lose size information and don't bounds-check. Avoid except in low-level code.
- `std::array<T, N>` is a safe, zero-overhead fixed-size array.
- `std::vector<T>` is the dynamic array. Use it as your default container.
- `reserve()` vectors when you know the final size to avoid reallocations.
- Use `std::string_view` for read-only string parameters — it avoids copies and works with all string types.

Chapter 11: Scope, Lifetime, and Organizing Code into Files

Scope

Scope is the region of code where a name is visible. In Python, scope is determined by function/class/module boundaries (LEGB rule). In C++, scope is determined by curly braces `{}`.

```cpp
int x = 10;          // file scope (global)
```

```cpp
void foo() {
    int x = 20;      // function scope — shadows global x
    {
        int x = 30;  // block scope — shadows function x
        std::cout << x;  // 30
    }
    std::cout << x;  // 20 — inner x is gone
}

std::cout << x;      // 10 — global x
```

A variable's scope ends at the closing `}` of the block it was declared in. This is fundamentally different from Python where variables declared in `if` blocks persist after the block ends.

Lifetime

Scope is about visibility. Lifetime is about when memory is allocated and freed.

- **Automatic** (local variables): lifetime = scope. Created when entering the block, destroyed when leaving. Stack-allocated.
- **Static** (global variables, `static` locals): lifetime = program duration. Created once, destroyed when the program exits.
- **Dynamic** (heap-allocated): lifetime is manual. Created with `new`, destroyed with `delete`. You control when.

```cpp
void counter() {
    static int count = 0;   // static local: initialized once, persists across calls
    ++count;
    std::cout << count << "\n";
}

counter();  // 1
counter();  // 2
counter();  // 3
```

Header Files and Source Files

In Python, each file is a module. You import it and all its functions become available. C++ is more explicit.

Header file (`.h` or `.hpp`): Contains *declarations* — function signatures, class definitions, type aliases. Tells the compiler what exists.

Source file (`.cpp`): Contains *definitions* — the actual implementations. Compiled to an object file.

Every `.cpp` that uses a function or class includes the header that declares it.

```cpp
// math.h  — declarations
#pragma once          // modern include guard: prevents double-inclusion
```

```
int add(int a, int b);
int multiply(int a, int b);
```

```
// math.cpp — definitions
#include "math.h"

int add(int a, int b) {
    return a + b;
}

int multiply(int a, int b) {
    return a * b;
}
```

```
// main.cpp — uses math functions
#include <iostream>
#include "math.h"    // quotes = look in local directory; <> = system path

int main() {
    std::cout << add(3, 4) << "\n";
}
```

Compile and link:

```
g++ -std=c++23 -c math.cpp    # produces math.o
g++ -std=c++23 -c main.cpp    # produces main.o
g++ -o program main.o math.o  # links both
# or all at once:
g++ -std=c++23 -o program main.cpp math.cpp
```

Include Guards

If a header is included from multiple files, the preprocessor pastes it multiple times. Without a guard, class definitions get duplicated → compile error.

```
// old style
#ifndef MATH_H
#define MATH_H
// ... declarations ...
#endif

// modern style (preferred)
#pragma once
// ... declarations ...
```

`#pragma once` is supported by all major compilers and is simpler.

What Goes in Headers vs Source Files

In headers: class definitions, function declarations, `inline` function bodies, templates (must be in headers — the compiler needs the full definition to instantiate), `constexpr` values, type aliases.

In source files: function definitions, global variable definitions, `static` variable definitions, `main`.

Never put `using namespace std;` in a header — it pollutes the namespace of every file that includes that header.

Namespaces

Namespaces group related names to prevent collisions:

```
namespace geometry {
    double area(double radius);
    double circumference(double radius);
}

namespace statistics {
    double area(/* ... */);  // different function, same name — no conflict
}

// Usage:
geometry::area(5.0);
statistics::area(/* ... */);
```

`std::` is the namespace for the standard library. You can avoid typing it with `using`:

```
using std::cout;      // only bring in cout
using std::string;    // only bring in string
// now you can write cout instead of std::cout

// OR (in .cpp files only — never in headers):
using namespace std;  // bring in everything from std
```

Key Takeaways

- Scope is defined by `{}`. Variables are destroyed when their scope ends.
- `static` local variables persist across function calls, initialized only once.
- Headers contain declarations; source files contain definitions. Headers are included by any file that needs the declarations.
- `#pragma once` prevents double-inclusion.
- Namespaces prevent name collisions. Never `using namespace std;` in headers.

Part III — Ownership and Memory Management

Chapter 12: RAII — The Core Idea That Replaces Garbage Collection

The Problem

Python has a garbage collector. When objects have no more references, the GC frees their memory. You don't think about it.

C++ has no garbage collector. If you allocate memory (or a file handle, or a network socket, or a mutex lock), *you* are responsible for releasing it. Failing to do so leaks resources.

```
void leak() {
    int* p = new int(42);
    if (some_condition) return;  // returns without deleting p — LEAK
    delete p;
}
```

Real programs have error paths, exceptions, and early returns. Manually tracking "did I release this?" in every code path is error-prone.

RAII: Resource Acquisition Is Initialization

The solution is a C++ idiom called **RAII** (Resource Acquisition Is Initialization). The idea:

- **Acquire** the resource in a constructor.
- **Release** the resource in a destructor.
- The compiler guarantees the destructor runs when the object goes out of scope — even if an exception occurs.

```
class FileHandle {
    FILE* file;
public:
    FileHandle(const char* path) {
        file = fopen(path, "r");  // acquire in constructor
    }
    ~FileHandle() {               // destructor: name is ~ClassName
        if (file) fclose(file);   // release in destructor
    }
    // ... methods to read, etc.
};

void process() {
    FileHandle f("data.txt");  // file opened here
    // ... do work ...
    // even if an exception is thrown, f's destructor runs
    // and closes the file
}  // f goes out of scope — ~FileHandle() called automatically
```

The destructor (`~FileHandle()`) is called automatically when `f` goes out of scope. It doesn't matter how the function exits — normal return, exception, early return — the destructor always runs.

RAII Is Everywhere in the Standard Library

`std::vector`, `std::string`, file streams, smart pointers — all use RAII. When they go out of scope, they release their resources.

```
{
    std::vector<int> v = {1, 2, 3, 4, 5};
    // vector allocates heap memory in constructor
}   // vector destroyed here — heap memory freed automatically
```

This is why modern C++ rarely calls `delete` explicitly. The containers and smart pointers do it for you via RAII.

Destructors

A destructor is a special member function: - Named `~ClassName()` - No parameters, no return type - Called automatically when the object is destroyed (goes out of scope, is deleted, etc.)

```
class Timer {
    std::chrono::time_point<std::chrono::high_resolution_clock> start;
public:
    Timer() : start(std::chrono::high_resolution_clock::now()) {}

    ~Timer() {
        auto end = std::chrono::high_resolution_clock::now();
        auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);
        std::cout << "Elapsed: " << ms.count() << "ms\n";
    }
};

void timed_operation() {
    Timer t;         // starts timing
    do_work();
}   // prints elapsed time automatically when t goes out of scope
```

RAII and Exceptions

One of RAII's biggest benefits: it makes exception safety automatic. Without RAII:

```
void bad() {
    int* p = new int(42);
    might_throw();       // if this throws, p is never deleted
    delete p;
}
```

With RAII (or smart pointers):

```
void good() {
    auto p = std::make_unique<int>(42);
    might_throw();  // if this throws, p's destructor still runs → no leak
}
```

The destructor is called during *stack unwinding* — the process of cleaning up stack frames as an exception propagates.

Key Takeaways

- RAII: tie resource lifetime to object lifetime. Acquire in constructor, release in destructor.
- The destructor is guaranteed to run when the object goes out of scope, regardless of how (exception, return, end of scope).
- The entire standard library is built on RAII. `vector`, `string`, streams, smart pointers — all manage resources this way.
- RAII makes exception safety free — you don't need `try/finally` blocks (though C++ has them too).
- Understanding RAII is the single most important mental shift when moving from Python to C++.

Chapter 13: Dynamic Allocation: `new`, `delete`, and Why You Avoid Them

What `new` and `delete` Do

`new` allocates memory on the heap and calls the constructor. `delete` calls the destructor and frees the memory.

```
// Allocate a single object
int* p = new int(42);
std::cout << *p;   // 42
delete p;
p = nullptr;

// Allocate an array
int* arr = new int[10];
arr[0] = 1;
delete[] arr;   // MUST use delete[] for arrays
arr = nullptr;

// Allocate a class object
MyClass* obj = new MyClass(args);
obj->method();
delete obj;
```

The Rules You Must Follow

1. Every `new` must have exactly one matching `delete`.
2. Every `new[]` must have exactly one matching `delete[]`.
3. Never `delete` something you didn't `new`.
4. Never `delete` a null pointer (safe, but meaningless).
5. Never use a pointer after deleting it (*use after free* — undefined behavior).
6. Never `delete` the same pointer twice (*double free* — undefined behavior).

Getting these right manually across thousands of lines of code, with exceptions and early returns, is nearly impossible. Hence:

Why You Avoid Raw `new/delete` in Modern C++

The C++ Core Guidelines (the canonical modern C++ style guide) say: **"Never use raw `new/delete`."** Instead:

- For a single heap object with one owner: `std::unique_ptr`
- For shared ownership: `std::shared_ptr`
- For large collections: `std::vector` (manages its own heap memory via RAII)
- For strings: `std::string`

```
// Old C++ (don't write this)
void old_way() {
    MyClass* obj = new MyClass();
    obj->do_work();
    delete obj;  // easy to forget, skip on exceptions, double-delete
}

// Modern C++ (do this instead)
void modern_way() {
    auto obj = std::make_unique<MyClass>();
    obj->do_work();
}   // unique_ptr's destructor calls delete automatically
```

When You Actually See `new/delete`

You'll encounter raw `new/delete` in: - Legacy C++ code (pre-C++11) - Low-level allocators and memory pool implementations - Interoperability with C APIs - Custom placement new (advanced)

You must be able to read and debug such code, but don't write it in new code.

Stack vs Heap Decision

```
// Stack — prefer this
void stack_example() {
    MyClass obj;           // on the stack — automatic lifetime
    obj.do_work();
}   // obj destroyed automatically

// Heap — only when necessary
void heap_example() {
    auto obj = std::make_unique<MyClass>();  // on the heap
    obj->do_work();
}   // unique_ptr destroys obj automatically
```

When do you actually need the heap? - Object must outlive the function that created it - Object size not known at compile time - You want to return the object from a factory function - Object is very large (beyond stack limit)

Key Takeaways

- `new` allocates on the heap and calls the constructor. `delete` destructs and frees.
- The rules (match `new`/`delete`, `new[]`/`delete[]`, no double-free, no use-after-free) are hard to follow manually.
- **Modern C++: avoid raw `new`/`delete`.** Use smart pointers and containers instead.
- You still need to understand raw allocation to read existing code and understand what smart pointers do under the hood.

Chapter 14: Smart Pointers: `unique_ptr`, `shared_ptr`, `weak_ptr`

What Is a Smart Pointer?

A smart pointer is a class that wraps a raw pointer and manages its lifetime via RAII. When the smart pointer goes out of scope, it automatically calls `delete`.

Three smart pointers in `<memory>`:

- `std::unique_ptr<T>` — sole ownership. One pointer owns the resource. When it dies, the resource is freed.
- `std::shared_ptr<T>` — shared ownership. Multiple pointers share a resource. Freed when the last one dies.
- `std::weak_ptr<T>` — non-owning observer of a `shared_ptr`. Used to break cycles.

`unique_ptr` — Exclusive Ownership

```cpp
#include <memory>

auto p = std::make_unique<int>(42);  // preferred creation method
std::cout << *p << "\n";              // 42 — dereference like a raw pointer
// no delete needed — p's destructor calls it
```

`unique_ptr` **cannot be copied** — only *moved*. This enforces the "one owner" invariant.

```cpp
auto p1 = std::make_unique<int>(10);
auto p2 = p1;            // COMPILE ERROR — can't copy unique_ptr
auto p2 = std::move(p1); // OK — transfers ownership. p1 is now null.
```

This is perfect for factory functions:

```cpp
std::unique_ptr<Shape> make_shape(std::string type) {
    if (type == "circle")    return std::make_unique<Circle>(5.0);
    if (type == "rectangle") return std::make_unique<Rectangle>(3.0, 4.0);
    return nullptr;
}

auto shape = make_shape("circle");  // caller owns the Shape
shape->draw();
// automatically deleted when shape goes out of scope
```

`shared_ptr` — Shared Ownership

`shared_ptr` uses reference counting. Each copy increments the count. When the count reaches zero, the resource is freed.

```
#include <memory>

auto p1 = std::make_shared<int>(99);  // count = 1
{
    auto p2 = p1;     // count = 2
    auto p3 = p1;     // count = 3
    std::cout << *p2; // 99
}   // p2, p3 destroyed — count back to 1
// resource still alive (p1 holds it)
// when p1 goes out of scope: count = 0 → deleted
```

Use `shared_ptr` when: - Multiple owners need to keep an object alive - Ownership is unclear or shared across subsystems

Avoid using `shared_ptr` by default "just in case." It has overhead (atomic reference count increment/decrement) and can cause memory leaks via cyclic references.

Cyclic References and `weak_ptr`

If two `shared_ptrs` point to objects that point back to each other, neither reference count reaches zero — memory leak:

```
struct Node {
    std::shared_ptr<Node> next;
    std::shared_ptr<Node> prev;  // back-pointer — CREATES A CYCLE
};
```

Solution: make back-pointers `weak_ptr`. A `weak_ptr` doesn't own the resource and doesn't increment the reference count:

```
struct Node {
    std::shared_ptr<Node> next;
    std::weak_ptr<Node> prev;    // non-owning back-pointer — no cycle
};

// To use a weak_ptr, lock it first (get a temporary shared_ptr):
if (auto locked = node->prev.lock()) {
    // locked is a valid shared_ptr — the Node still exists
    locked->do_something();
} else {
    // the Node has been destroyed
}
```

Accessing the Raw Pointer

Sometimes (e.g., passing to C APIs) you need the raw pointer:

```cpp
auto p = std::make_unique<int>(42);
int* raw = p.get();    // get the raw pointer — p still owns it
some_c_api(raw);       // pass to C function that doesn't take ownership
// DO NOT delete raw — p still owns the memory
```

`unique_ptr` with Arrays

```cpp
auto arr = std::make_unique<int[]>(10);  // unique_ptr for arrays
arr[0] = 1;
arr[1] = 2;
// no delete[] needed — handled automatically
```

For runtime-sized arrays you actually want `std::vector` 99% of the time. `unique_ptr<T[]>` is for interoperating with C APIs that return arrays.

Choosing the Right Smart Pointer

```
Is ownership unique (one owner)?
  → std::unique_ptr<T>

Is ownership shared (multiple owners)?
  → std::shared_ptr<T>

Do you need to observe a shared_ptr without owning it (e.g., back-pointers)?
  → std::weak_ptr<T>

Is the object small, short-lived, and clearly scoped?
  → Skip pointers entirely — just use a value (no pointer)
```

Key Takeaways

- `unique_ptr`: sole ownership, no copy, only move. Zero overhead over raw pointer.
- `shared_ptr`: shared ownership via reference count. Slight overhead. Use when needed, not by default.
- `weak_ptr`: non-owning observer, prevents cyclic reference leaks.
- Create with `make_unique` and `make_shared` — never `new`.
- `get()` gives the raw pointer without transferring ownership.

Chapter 15: Move Semantics, lvalues and rvalues

The Problem: Unnecessary Copies

Imagine returning a large vector from a function:

```cpp
std::vector<int> make_big_vector() {
    std::vector<int> v(1'000'000);
    // fill v...
    return v;
```

```
}

auto result = make_big_vector();  // does this copy 1 million ints?
```

In old C++ (pre-C++11), this could copy all million elements. Modern C++ makes it free via *move semantics*.

lvalues and rvalues

An **lvalue** is an expression that refers to a persistent, named memory location. You can take its address. Variables are lvalues.

An **rvalue** is a temporary value that doesn't persist. The result of `a + b`, a function return value, a literal — these are rvalues. They're about to be thrown away.

```
int x = 5;    // x is an lvalue; 5 is an rvalue
int y = x + 3;  // (x + 3) is an rvalue — computed, used, gone
```

Moving Instead of Copying

For an rvalue, you don't need to *copy* its data — you can *steal* it. If a `vector` temporary is about to be destroyed, why copy its heap buffer? Just take the buffer pointer.

A **move constructor** does exactly this: it transfers ownership of resources from the source (which is being destroyed) to the new object, leaving the source in a valid but empty state.

```
std::vector<int> a = {1, 2, 3, 4, 5};
std::vector<int> b = std::move(a);  // b TAKES a's buffer — no copy

// a is now in a valid but unspecified state (typically empty)
// b owns the data
std::cout << b.size();  // 5
std::cout << a.size();  // 0
```

`std::move(a)` is a cast that says "treat `a` as an rvalue — it's OK to steal from it." After the move, `a` is valid but empty. Don't use `a`'s value after moving from it.

Why Returning Vectors Is Free (NRVO)

When a function returns a local variable, the compiler often applies **Named Return Value Optimization (NRVO)** — it constructs the return value directly in the caller's memory, skipping the copy entirely. When NRVO isn't possible, the move constructor is used. Either way, returning a `vector<int>` with a million elements is essentially free.

```
std::vector<int> make_big_vector() {
    std::vector<int> v(1'000'000);
    return v;   // NRVO: constructed directly in caller — zero copies
}
```

Rvalue References (`&&`)

An rvalue reference binds to temporaries. It's what move constructors and move assignment operators accept:

```cpp
void process(std::vector<int>&& v) {  // takes an rvalue reference
    // we can steal from v since caller indicated it's disposable
    internal_data = std::move(v);
}

process(std::vector<int>{1, 2, 3});  // temporary — OK
// process(my_vector);               // ERROR — lvalue can't bind to &&
process(std::move(my_vector));       // OK — explicit move
```

Perfect Forwarding

In templates, you sometimes want to forward an argument preserving whether it was an lvalue or rvalue. This uses *forwarding references* (`T&&` in a template) and `std::forward`:

```cpp
template<typename T>
void wrapper(T&& arg) {
    actual_function(std::forward<T>(arg));  // forwards as lvalue or rvalue
}
```

This pattern appears everywhere in the standard library and in factory functions like `make_unique`. We'll revisit it in the templates chapters.

Move Semantics Summary

Operation	What happens
Copy (`b = a`)	Both `a` and `b` have their own copy of the data
Move (`b = std::move(a)`)	`b` takes `a`'s data; `a` is left empty
NRVO (returning local)	Object constructed directly in destination — no copy, no move

Key Takeaways

- lvalues are named, persistent variables. rvalues are temporaries.
- Move semantics let you *steal* resources from objects about to be destroyed — vastly cheaper than copying for containers.
- `std::move(x)` casts `x` to an rvalue, signaling "you can steal this."
- After moving from an object, it's in a valid but unspecified state. Don't read its value.
- NRVO means returning local variables by value is usually free — return containers by value without fear.

Chapter 16: The Rule of 0, 3, and 5

The Compiler-Generated Special Member Functions

C++ automatically generates six special member functions if you don't define them:

1. **Default constructor** — `MyClass()` — creates an object with no arguments
2. **Copy constructor** — `MyClass(const MyClass&)` — creates a copy
3. **Copy assignment operator** — `operator=(const MyClass&)` — assigns from a copy
4. **Move constructor** — `MyClass(MyClass&&)` — creates from a temporary (C++11)
5. **Move assignment operator** — `operator=(MyClass&&)` — assigns from a temporary (C++11)
6. **Destructor** — `~MyClass()` — cleans up

The generated versions do *memberwise* operations: copy/move/destroy each member in turn.

The Rule of Zero

If you don't manage any resources directly, don't define any of the six.

Let the compiler-generated defaults do memberwise operations. Rely on your members (vectors, strings, smart pointers) to manage their own resources via RAII.

```
class Person {
    std::string name;     // std::string manages its own memory
    int age;
    std::vector<std::string> hobbies;  // vector manages its own memory
    // No raw pointers — no manual resource management
public:
    Person(std::string n, int a) : name(std::move(n)), age(a) {}
    // No destructor, copy constructor, etc. needed
    // The compiler generates correct versions automatically
};
```

This is the rule you should follow for most classes. The standard library types manage their own resources correctly, so memberwise operations are correct.

The Rule of Three

If your class manages a resource directly (raw pointer, file handle, etc.), you need to define all three of: **destructor**, **copy constructor**, **copy assignment operator**.

Why? Because the compiler's default copy just copies the pointer — both objects then point to the same resource. When one is destroyed, the other has a dangling pointer (double-free bug).

```
class Buffer {
    int* data;
    int  size;
public:
    Buffer(int n) : data(new int[n]), size(n) {}

    // MUST define: default copy would give two Buffers pointing to same memory
    Buffer(const Buffer& other) : data(new int[other.size]), size(other.size) {
        std::copy(other.data, other.data + size, data);
```

```
    }

    Buffer& operator=(const Buffer& other) {
        if (this == &other) return *this;  // self-assignment guard
        delete[] data;
        data = new int[other.size];
        size = other.size;
        std::copy(other.data, other.data + size, data);
        return *this;
    }

    ~Buffer() { delete[] data; }
};
```

The Rule of Five

C++11 adds move semantics. If you define any of the three above, also define **move constructor** and **move assignment operator** for efficiency:

```
class Buffer {
    int* data;
    int  size;
public:
    Buffer(int n) : data(new int[n]), size(n) {}

    // Copy (deep)
    Buffer(const Buffer& other) : data(new int[other.size]), size(other.size) {
        std::copy(other.data, other.data + size, data);
    }
    Buffer& operator=(const Buffer& other) {
        if (this == &other) return *this;
        delete[] data;
        data = new int[other.size];
        size = other.size;
        std::copy(other.data, other.data + size, data);
        return *this;
    }

    // Move (steal the pointer — no allocation)
    Buffer(Buffer&& other) noexcept : data(other.data), size(other.size) {
        other.data = nullptr;  // leave source in valid state
        other.size = 0;
    }
    Buffer& operator=(Buffer&& other) noexcept {
        if (this == &other) return *this;
        delete[] data;
        data = other.data;
        size = other.size;
```

```
        other.data = nullptr;
        other.size = 0;
        return *this;
    }

    ~Buffer() { delete[] data; }
};
```

The Practical Advice

1. **Default to Rule of Zero.** Use `std::vector`, `std::unique_ptr`, `std::string` for resources. Never define any of the five.
2. If you **must** hold a raw resource (rare in modern C++), apply the Rule of Five.
3. Use `= default` and `= delete` to explicitly control generation:

```
class NonCopyable {
public:
    NonCopyable() = default;
    NonCopyable(const NonCopyable&) = delete;             // disable copy
    NonCopyable& operator=(const NonCopyable&) = delete;  // disable copy assign
    NonCopyable(NonCopyable&&) = default;                 // allow move
    NonCopyable& operator=(NonCopyable&&) = default;      // allow move assign
};
```

Key Takeaways

- The compiler generates 6 special member functions. By default they do memberwise operations.
- **Rule of Zero**: don't define any if you don't manage resources directly (the best case).
- **Rule of Three**: if you manage a resource, define destructor + copy constructor + copy assignment.
- **Rule of Five**: add move constructor + move assignment for efficiency.
- Use `= delete` to explicitly prohibit copying. Use `= default` to explicitly request the compiler-generated version.

Part IV — Object-Oriented Programming

Chapter 17: Classes, Objects, and Encapsulation

Classes in Python vs C++

Python classes are straightforward:

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def distance_to(self, other):
        return ((self.x - other.x)**2 + (self.y - other.y)**2) ** 0.5
```

C++ classes are similar but with explicit access control and a declaration/definition split:

```cpp
#include <cmath>

class Point {
public:                          // access specifier: anyone can access
    double x;
    double y;

    Point(double x, double y) : x(x), y(y) {}  // constructor

    double distance_to(const Point& other) const {
        double dx = x - other.x;
        double dy = y - other.y;
        return std::sqrt(dx*dx + dy*dy);
    }
};

int main() {
    Point p1{1.0, 2.0};
    Point p2{4.0, 6.0};
    std::cout << p1.distance_to(p2) << "\n";  // 5.0
}
```

Access Specifiers

C++ has three access levels. Python relies on convention (`_private`, `__mangled`); C++ enforces them at compile time.

```cpp
class BankAccount {
public:      // accessible from anywhere
    std::string owner_name;

    void deposit(double amount) { balance += amount; }
    double get_balance() const { return balance; }

protected:   // accessible from this class and derived classes
    double balance = 0.0;

private:     // accessible ONLY from within this class
    int account_number;
```

```
    std::string pin;
};

BankAccount acct;
acct.deposit(100.0);          // OK — public
acct.get_balance();           // OK — public
// acct.balance;              // depends — protected (not accessible here)
// acct.pin;                  // ERROR — private
```

The default access in a `class` is `private`. In a `struct`, it's `public`. This is the only difference between `class` and `struct` in C++ — use `struct` for plain data, `class` for things with invariants.

`this` Pointer

Inside a member function, `this` is a pointer to the current object (like Python's `self`):

```
class Counter {
    int value = 0;
public:
    Counter& increment() {
        ++value;
        return *this;  // return reference to self — enables chaining
    }
    int get() const { return value; }
};

Counter c;
c.increment().increment().increment();  // method chaining
std::cout << c.get();  // 3
```

Python makes `self` explicit. C++ makes `this` implicit (you don't have to name it in the parameter list) but accessible when needed.

Separating Declaration and Definition

For real projects, class declarations go in headers and method definitions go in source files:

```
// point.h
#pragma once

class Point {
public:
    double x, y;
    Point(double x, double y);
    double distance_to(const Point& other) const;
};

// point.cpp
#include "point.h"
#include <cmath>
```

```cpp
Point::Point(double x, double y) : x(x), y(y) {}

double Point::distance_to(const Point& other) const {
    double dx = x - other.x;
    double dy = y - other.y;
    return std::sqrt(dx*dx + dy*dy);
}
```

`Point::` is the *scope resolution operator* — it says "this function belongs to the `Point` class."

Encapsulation: Why It Matters

Encapsulation means hiding implementation details and exposing a controlled interface. The user of `BankAccount` shouldn't directly manipulate `balance` — they should go through `deposit`/`withdraw` which can enforce business rules (no negative balance, audit logging, etc.).

```cpp
class Temperature {
    double celsius;
public:
    Temperature(double c) : celsius(c) {}

    double get_celsius()    const { return celsius; }
    double get_fahrenheit() const { return celsius * 9.0/5.0 + 32.0; }
    double get_kelvin()     const { return celsius + 273.15; }

    void set_celsius(double c) {
        if (c < -273.15) throw std::invalid_argument("Below absolute zero");
        celsius = c;
    }
};
```

The internal representation is `celsius`. The class exposes conversions. If you later switch the internal representation to kelvin, none of the users' code changes — only the implementation.

Key Takeaways

- `public`, `protected`, `private` control access. `class` defaults to private; `struct` to public.
- `this` is an implicit pointer to the current object (like Python's `self`, but a pointer).
- Separate declaration (`.h`) from definition (`.cpp`) for non-trivial classes.
- Encapsulation: expose methods, hide data. Invariants are enforced in methods, not by trusting callers.

Chapter 18: Constructors, Destructors, and Initialization

Constructors

Constructors initialize objects. A class can have multiple constructors (overloaded). They have the same name as the class and no return type.

```cpp
class Rectangle {
    double width, height;
public:
    Rectangle() : width(0), height(0) {}              // default constructor
    Rectangle(double w, double h) : width(w), height(h) {}  // parameterized
    Rectangle(double side) : width(side), height(side) {}   // square

    double area() const { return width * height; }
};

Rectangle r1;           // default constructor
Rectangle r2(3.0, 4.0); // parameterized
Rectangle r3(5.0);      // square
Rectangle r4{3.0, 4.0}; // uniform initialization (preferred)
```

Member Initializer Lists

The `: width(w), height(h)` syntax is a *member initializer list*. It initializes members **before** the constructor body runs.

```cpp
class Foo {
    int x;
    int y;
public:
    // Good: uses initializer list
    Foo(int x, int y) : x(x), y(y) {}

    // Bad: assignment in body (default-constructs first, then assigns)
    Foo(int x, int y) { this->x = x; this->y = y; }
};
```

For `const` members and reference members, the initializer list is not optional — they *must* be initialized in the list (you can't assign to const or references after construction).

Members are initialized in **declaration order**, not the order in the initializer list. The compiler warns if the order disagrees.

`explicit` Constructors

A single-argument constructor can accidentally convert:

```cpp
class Kg {
    double value;
public:
    Kg(double v) : value(v) {}
};

void weigh(Kg mass) { /* ... */ }
weigh(75.0);  // silently converts double to Kg — OK? or bug?
```

Mark single-argument constructors `explicit` to prevent implicit conversion:

```
class Kg {
    double value;
public:
    explicit Kg(double v) : value(v) {}
};

weigh(75.0);          // COMPILE ERROR — no implicit conversion
weigh(Kg{75.0});      // OK — explicit construction
```

Delegating Constructors (C++11)

Constructors can call other constructors:

```
class Circle {
    double x, y, radius;
public:
    Circle(double x, double y, double r) : x(x), y(y), radius(r) {}
    Circle(double r) : Circle(0.0, 0.0, r) {}   // delegates to the 3-arg ctor
    Circle()         : Circle(1.0) {}             // delegates to the 1-arg ctor
};
```

Destructors

Covered in RAII (Chapter 12), but a few more details:

- There is exactly one destructor per class. No overloading, no parameters.
- If you don't define one, the compiler generates one that destructs members in reverse declaration order.
- In a class hierarchy with virtual functions, the base class destructor **must** be `virtual` (Chapter 20).

```
class FileWriter {
    std::ofstream file;  // std::ofstream is RAII — closes in destructor
public:
    FileWriter(const std::string& path) : file(path) {}
    void write(const std::string& s) { file << s; }
    // No explicit destructor needed — ofstream's destructor closes the file
};
```

Aggregate Initialization

Structs (or classes with all public members and no user-provided constructors) can be initialized with `{}` directly:

```
struct Color {
    float r, g, b, a;
};
```

```
Color red  = {1.0f, 0.0f, 0.0f, 1.0f};  // aggregate init
Color blue{0.0f, 0.0f, 1.0f, 1.0f};     // equivalent
```

`= default` and `= delete`

Request or suppress generated functions:

```
class Singleton {
public:
    static Singleton& instance() {
        static Singleton s;
        return s;
    }
    Singleton(const Singleton&) = delete;             // no copying
    Singleton& operator=(const Singleton&) = delete;  // no copy assign
private:
    Singleton() = default;  // private default constructor
};
```

Key Takeaways

- Use member initializer lists, not body assignments. They're more efficient and required for `const`/reference members.
- `explicit` prevents accidental implicit conversions from single-argument constructors.
- Delegating constructors let constructors call each other to reduce duplication.
- `= default` and `= delete` control compiler-generated functions explicitly.

Chapter 19: Inheritance and Composition

Two Ways to Reuse Code

Composition — "has-a": A `Car` has an `Engine`. The class contains an instance of another class as a member.

Inheritance — "is-a": A `Dog` is an `Animal`. The class derives from another class, inheriting its members and interface.

Python uses both. C++ uses both, but the details differ.

Inheritance Syntax

```
# Python
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "..."
```

```python
class Dog(Animal):
    def speak(self):
        return "Woof"
```

```cpp
// C++
class Animal {
protected:
    std::string name;
public:
    Animal(std::string n) : name(std::move(n)) {}
    virtual std::string speak() const { return "..."; }
};

class Dog : public Animal {        // Dog inherits from Animal
public:
    Dog(std::string n) : Animal(std::move(n)) {}  // call base constructor
    std::string speak() const override { return "Woof"; }
};
```

The `: public Animal` means `Dog` publicly inherits from `Animal`. `public` inheritance means the `public`/`protected` interface of `Animal` is preserved in `Dog`.

Constructor Chaining

Derived class constructors must call the base class constructor (explicitly or implicitly if there's a default constructor):

```cpp
class Vehicle {
    int year;
public:
    Vehicle(int y) : year(y) {}
    int get_year() const { return year; }
};

class Car : public Vehicle {
    std::string model;
public:
    Car(std::string m, int y) : Vehicle(y), model(std::move(m)) {}
    //                           ^^^^^^^^^^
    //                           call base constructor first
};
```

`protected` Members

`protected` members are accessible from the class itself *and* derived classes, but not from outside:

```cpp
class Animal {
protected:
    int energy = 100;   // derived classes can access this
private:
```

```
    int dna_sequence;   // derived classes cannot access this
};

class Dog : public Animal {
public:
    void eat() { energy += 20; }  // OK — energy is protected
};
```

Composition vs Inheritance

Prefer composition over inheritance for code reuse. Use inheritance only when you need polymorphism (the ability to treat derived objects as the base type).

```
// Composition (prefer this for reuse):
class Logger {
public:
    void log(const std::string& msg) { std::cout << msg << "\n"; }
};

class Server {
    Logger logger;   // Server HAS a Logger
public:
    void handle_request() {
        logger.log("Request received");
        // ...
    }
};

// Inheritance (use when polymorphism is needed):
class Shape {
public:
    virtual double area() const = 0;  // pure virtual — must override
};

class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius(r) {}
    double area() const override { return 3.14159 * radius * radius; }
};
```

Multiple Inheritance

C++ allows a class to inherit from multiple bases. Python does too, but C++ adds complexity:

```
class Flyable {
public:
    virtual void fly() = 0;
};
```

```cpp
class Swimmable {
public:
    virtual void swim() = 0;
};

class Duck : public Animal, public Flyable, public Swimmable {
public:
    Duck(std::string n) : Animal(std::move(n)) {}
    std::string speak() const override { return "Quack"; }
    void fly() override { std::cout << "Duck flying\n"; }
    void swim() override { std::cout << "Duck swimming\n"; }
};
```

Diamond inheritance (where a class inherits from two classes that both inherit from the same base) requires `virtual` inheritance to avoid duplicating the base. It's complex; avoid if possible.

Key Takeaways

- `: public Base` is the inheritance syntax. Access public base members with derived-class code.
- Derived constructors must call the base constructor via the initializer list.
- `protected` is accessible in derived classes but not from outside the hierarchy.
- Composition ("has-a") is usually better than inheritance for code reuse. Use inheritance for "is-a" polymorphism.
- C++ supports multiple inheritance but it adds complexity.

Chapter 20: Virtual Functions and Polymorphism

The Problem Without `virtual`

Without virtual functions, function calls resolve at compile time based on the static type of the variable:

```cpp
class Animal {
public:
    std::string speak() const { return "..."; }
};

class Dog : public Animal {
public:
    std::string speak() const { return "Woof"; }
};

Animal* p = new Dog();
std::cout << p->speak();  // "..." — calls Animal::speak!
                          // because p's static type is Animal*
```

This is probably not what you want. You want to call `Dog::speak` because the object is actually a `Dog`.

Virtual Functions

Mark a function `virtual` in the base class. Now calls resolve at *runtime* based on the actual object type (dynamic dispatch):

```
class Animal {
public:
    virtual std::string speak() const { return "..."; }
    virtual ~Animal() {}  // ALWAYS make destructor virtual in polymorphic bases
};

class Dog : public Animal {
public:
    std::string speak() const override { return "Woof"; }
};

class Cat : public Animal {
public:
    std::string speak() const override { return "Meow"; }
};

Animal* p = new Dog();
std::cout << p->speak();  // "Woof" — calls Dog::speak at runtime
delete p;
```

`override` is a C++11 keyword that tells the compiler "I intend to override a virtual function." If the signature doesn't match a base virtual, it's a compile error — catching typos.

How Virtual Dispatch Works (the vtable)

Each class with virtual functions has a hidden **vtable** (virtual function table) — an array of function pointers. Each object has a hidden **vptr** (vtable pointer) as its first member.

```
Dog object in memory:
[ vptr ] → Dog's vtable → [ Dog::speak, Dog::destructor, ... ]
[ Animal members: name ]
[ Dog members: ... ]
```

When you call `p->speak()`, the CPU: 1. Follows `p` to the object. 2. Follows the vptr to the vtable. 3. Calls the function at the `speak` slot.

This indirection has a tiny runtime cost (two pointer dereferences per call). For most code this is irrelevant. For extremely hot inner loops (millions of calls per second), it can matter.

Polymorphism with Containers

The real power: treat heterogeneous objects uniformly through a base pointer:

```
#include <vector>
#include <memory>

std::vector<std::unique_ptr<Animal>> zoo;
```

```
zoo.push_back(std::make_unique<Dog>("Rex"));
zoo.push_back(std::make_unique<Cat>("Whiskers"));
zoo.push_back(std::make_unique<Dog>("Buddy"));

for (const auto& animal : zoo) {
    std::cout << animal->speak() << "\n";
}
// Woof
// Meow
// Woof
```

Each `unique_ptr<Animal>` can hold any `Animal` subtype. The right `speak` is called via virtual dispatch.

`virtual` Destructor

If you `delete` a derived object through a base pointer, and the base destructor is not virtual, only the base destructor runs — the derived destructor is skipped → resource leak:

```
class Base {
public:
    ~Base() { std::cout << "Base dtor\n"; }  // NOT virtual — BUG
};
class Derived : public Base {
    int* data = new int[100];
public:
    ~Derived() { delete[] data; std::cout << "Derived dtor\n"; }
};

Base* p = new Derived();
delete p;  // only "Base dtor" — data is leaked!
```

Rule: If a class has any virtual functions, make its destructor virtual.

`final` Keyword

`final` prevents further overriding or inheritance:

```
class Shape {
public:
    virtual double area() const = 0;
};

class Circle final : public Shape {  // can't derive from Circle
    double r;
public:
    Circle(double r) : r(r) {}
    double area() const override final { return 3.14159 * r * r; }
    //                          ^^^^^
```

```
    //                          can't override area in further derived classes
};
```

Key Takeaways

- `virtual` enables runtime dispatch (calling the derived class's version through a base pointer/reference).
- `override` keyword documents intent and catches signature mismatches at compile time.
- Always make the base class destructor `virtual` if the class has any virtual functions.
- The vtable is the mechanism: a hidden per-class array of function pointers.
- Virtual dispatch has a tiny overhead (two pointer hops). It matters only in extremely hot code.

Chapter 21: Abstract Classes and Interfaces

Pure Virtual Functions

A **pure virtual function** has no body in the base class — it's marked `= 0`. A class with at least one pure virtual function is an **abstract class** — it cannot be instantiated directly.

```
class Shape {
public:
    virtual double area()      const = 0;  // pure virtual
    virtual double perimeter() const = 0;  // pure virtual
    virtual void   draw()      const = 0;  // pure virtual
    virtual ~Shape() {}
};

Shape s;  // ERROR — cannot instantiate abstract class
```

Any derived class that doesn't override all pure virtual functions is also abstract. You must override them all to create concrete objects.

```
class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius(r) {}
    double area()      const override { return 3.14159 * radius * radius; }
    double perimeter() const override { return 2 * 3.14159 * radius; }
    void   draw()      const override { std::cout << "Drawing circle\n"; }
};

Circle c(5.0);         // OK — all pure virtuals implemented
Shape* s = &c;         // OK — pointer to abstract base
std::cout << s->area(); // calls Circle::area via virtual dispatch
```

Interfaces

C++ has no `interface` keyword. An *interface* in C++ is a convention: an abstract class with **only** pure virtual functions and a virtual destructor — no data members, no non-virtual methods.

```
class Serializable {
public:
    virtual std::string serialize() const = 0;
    virtual void deserialize(const std::string& data) = 0;
    virtual ~Serializable() = default;
};

class Drawable {
public:
    virtual void draw(Canvas& canvas) const = 0;
    virtual ~Drawable() = default;
};

// A class can implement multiple interfaces
class Sprite : public Drawable, public Serializable {
    // ...
};
```

This is how C++ achieves interface-based polymorphism. It's the same idea as Java/Python abstract base classes, just with different syntax.

Abstract Base Classes vs Interfaces

- **Abstract base class**: may have data, non-virtual methods, partial implementations. Some methods are pure virtual — subclasses fill in the rest.
- **Interface**: all pure virtual, no data, no implementation. Pure contract.

```
// Abstract base class (partial implementation)
class Animal {
    std::string name;  // data member
public:
    Animal(std::string n) : name(std::move(n)) {}
    std::string get_name() const { return name; }  // concrete method
    virtual std::string speak() const = 0;          // derived must implement
    virtual ~Animal() = default;
};

// Interface (pure contract)
class Printable {
public:
    virtual std::string to_string() const = 0;
    virtual ~Printable() = default;
};
```

`dynamic_cast` — Safe Downcasting

Sometimes you have a base pointer but need to access a derived-specific method. `dynamic_cast` checks at runtime whether the cast is valid:

```
Animal* a = get_some_animal();

Dog* d = dynamic_cast<Dog*>(a);  // returns nullptr if a is not a Dog
if (d != nullptr) {
    d->fetch();  // Dog-specific method
}

// With references (throws std::bad_cast on failure instead of returning null):
try {
    Dog& d = dynamic_cast<Dog&>(*a);
    d.fetch();
} catch (const std::bad_cast& e) {
    // not a Dog
}
```

`dynamic_cast` requires at least one virtual function in the hierarchy (the vtable provides the runtime type information).

Use it sparingly — frequent downcasting is a sign of a design problem. Prefer virtual functions that do the right thing per type without explicit type checking.

Key Takeaways

- Pure virtual functions (`= 0`) make a class abstract — it cannot be instantiated.
- A class with only pure virtual functions and a virtual destructor is an *interface*.
- C++ doesn't have an `interface` keyword — it's a convention using abstract classes.
- Multiple inheritance lets a class implement multiple interfaces.
- `dynamic_cast` safely downcasts at runtime but requires virtual functions and should be used sparingly.

Chapter 22: Operator Overloading

What Is Operator Overloading?

In Python you can define `__add__`, `__eq__`, `__lt__`, etc. to make your classes work with operators. C++ has the same feature, with different syntax.

```
# Python
class Vector2D:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other):
        return Vector2D(self.x + other.x, self.y + other.y)
    def __str__(self):
        return f"({self.x}, {self.y})"
```

```cpp
// C++
struct Vector2D {
    double x, y;
    Vector2D(double x, double y) : x(x), y(y) {}

    Vector2D operator+(const Vector2D& other) const {
        return {x + other.x, y + other.y};
    }

    bool operator==(const Vector2D& other) const {
        return x == other.x && y == other.y;
    }
};

// Can also be a free function (outside the class):
std::ostream& operator<<(std::ostream& os, const Vector2D& v) {
    return os << "(" << v.x << ", " << v.y << ")";
}
```

Which Operators Can Be Overloaded?

Almost all: `+`, `-`, `*`, `/`, `%`, `==`, `!=`, `<`, `>`, `<=`, `>=`, `[]`, `()`, `++`, `--`, `<<`, `>>`, `=`, `->`, `&`, `|`, `^`, and more.

Cannot overload: `::`, `.`, `.*`, `sizeof`, `?:`.

Member vs Free Function

Operators can be member functions or free functions. Guidelines:

- **Make it a member** when the left operand is your type: `+=`, `-=`, `[]`, `()`, `->`, unary operators.
- **Make it a free function** when symmetry is needed or the left operand isn't your type: `+`, `-`, `==`, `<<`.

```cpp
struct Vec {
    double x, y;

    // Member — left operand is Vec
    Vec& operator+=(const Vec& rhs) {
        x += rhs.x; y += rhs.y;
        return *this;
    }
};

// Free function — allows: vec + vec, but also could allow double + vec
Vec operator+(Vec lhs, const Vec& rhs) {
    lhs += rhs;   // reuse +=
    return lhs;
}
```

Note the `Vec lhs` (by value, not reference) in `operator+`. This copies `lhs` so we can modify and return it.

The Spaceship Operator (C++20)

C++20 adds `<=>` (three-way comparison), which auto-generates `<`, `>`, `<=`, `>=` from one definition:

```
#include <compare>

struct Point {
    int x, y;
    auto operator<=>(const Point&) const = default;  // compiler generates all comparisons
    bool operator==(const Point&)  const = default;
};

Point a{1, 2}, b{1, 3};
bool less = a < b;   // works
bool eq   = a == b;  // works
```

Subscript Operator `[]`

```
class Matrix {
    double data[4][4];
public:
    double* operator[](int row) { return data[row]; }
    const double* operator[](int row) const { return data[row]; }
};

Matrix m;
m[0][1] = 3.14;   // m.operator[](0) returns data[0], then [1]
```

Function Call Operator `()`

A class with `operator()` is a *functor* (function object). It can be called like a function:

```
class Multiplier {
    double factor;
public:
    Multiplier(double f) : factor(f) {}
    double operator()(double x) const { return x * factor; }
};

Multiplier double_it(2.0);
std::cout << double_it(5.0);  // 10.0
std::cout << double_it(3.0);  // 6.0
```

Functors are the precursor to lambdas (Chapter 30). Lambdas generate anonymous functor classes behind the scenes.

Key Takeaways

- Operator overloading uses `operator+`, `operator==`, etc. — the same concept as Python's `__add__`, `__eq__`.
- Member operators for operations where the left operand is your type; free functions for symmetric operations.
- Implement `+=` first, then implement `+` in terms of `+=`.
- C++20's `<=>` generates all comparison operators from one definition.
- `operator()` makes a class callable — this is a functor.

Part V — Generic Programming

Chapter 23: Function and Class Templates

What Is Generic Programming?

In Python, functions are already generic — they work on any type that supports the required operations. Duck typing handles it at runtime.

```python
def add(a, b):
    return a + b  # works for int, float, str, anything with +
```

C++ is statically typed. Without templates, you'd need separate functions for every type:

```cpp
int    add(int a,    int b)    { return a + b; }
double add(double a, double b) { return a + b; }  // tedious duplication
```

Templates let you write the function once and have the compiler generate type-specific versions as needed.

Function Templates

```cpp
template<typename T>
T add(T a, T b) {
    return a + b;
}

int    r1 = add(1, 2);         // T = int
double r2 = add(1.5, 2.5);    // T = double
// compiler generates two separate functions — fully type-checked, zero overhead
```

`typename T` (or `class T` — identical meaning) declares `T` as a type parameter. You can have multiple:

```cpp
template<typename From, typename To>
To convert(From value) {
    return static_cast<To>(value);
```

```cpp
}

int n = convert<double, int>(3.7);  // explicit template arguments
auto d = convert<int, double>(5);   // explicit
```

The compiler usually deduces template arguments from function arguments, so you don't need to specify them explicitly unless disambiguation is needed.

Class Templates

```cpp
template<typename T>
class Stack {
    std::vector<T> data;
public:
    void push(const T& value) { data.push_back(value); }
    void pop()                { data.pop_back(); }
    T&   top()                { return data.back(); }
    bool empty() const        { return data.empty(); }
    int  size()  const        { return data.size(); }
};

Stack<int>         int_stack;
Stack<std::string> str_stack;

int_stack.push(42);
str_stack.push("hello");
```

Template Instantiation

When you use `Stack<int>`, the compiler generates a concrete `Stack` class with `T` replaced by `int`. This is called *instantiation*. Each instantiation is compiled independently — the code is generated only for types you actually use.

This is why **templates must be defined in headers** (not split across `.h`/`.cpp` like regular functions). The compiler needs the full template definition at every instantiation point.

Non-Type Template Parameters

Template parameters can be values, not just types:

```cpp
template<typename T, int N>
class FixedArray {
    T data[N];
    int count = 0;
public:
    void push(const T& v) {
        if (count < N) data[count++] = v;
    }
    T& operator[](int i) { return data[i]; }
```

```
    int size() const { return count; }
};

FixedArray<int, 10> arr;   // T=int, N=10 — no heap allocation
```

`std::array<T, N>` from the standard library is exactly this pattern.

Template Type Deduction

The compiler deduces template arguments from function call arguments. Understanding the rules prevents surprises:

```
template<typename T>
void foo(T x);

int   a = 5;
foo(a);      // T = int
foo(5);      // T = int
foo(5.0);    // T = double

template<typename T>
void bar(T& x);  // reference parameter

bar(a);      // T = int  (not int& — reference is part of the parameter, not T)
```

Full deduction rules are nuanced. Chapter 33 (auto) and Chapter 15 (forwarding references) cover the relevant edge cases.

Key Takeaways

- Templates let you write type-independent code that the compiler instantiates for each type used.
- `template<typename T>` precedes both function and class templates.
- Templates must be defined in headers — the compiler needs the definition at instantiation.
- Non-type template parameters (like `int N`) let you parameterize on values.
- Template instantiation is zero-cost at runtime — each use generates a type-specific compiled function.

Chapter 24: Template Specialization and Variadic Templates

Full Specialization

You can provide a custom implementation for a specific type:

```
template<typename T>
T max_val(T a, T b) {
    return a > b ? a : b;
}
```

```cpp
// Full specialization for const char* (C strings):
template<>
const char* max_val<const char*>(const char* a, const char* b) {
    return std::strcmp(a, b) > 0 ? a : b;
}

max_val(3, 5);               // general template
max_val("apple", "banana");  // uses specialization
```

Partial Specialization

Class templates (not function templates) support partial specialization — specializing on a subset of template parameters:

```cpp
// General template
template<typename T, typename U>
class Pair {
    T first;
    U second;
    // ...
};

// Partial specialization: when both types are the same
template<typename T>
class Pair<T, T> {
    T first, second;
    // can have different implementation
};

// Partial specialization: when U is a pointer
template<typename T, typename U>
class Pair<T, U*> {
    T first;
    U* second;
    // pointer-specific handling
};
```

Variadic Templates (C++11)

Variadic templates accept any number of type parameters. They power `std::tuple`, `std::make_unique`, and much of the modern standard library.

```cpp
// Variadic function: sum any number of arguments
template<typename T>
T sum(T first) {
    return first;
}

template<typename T, typename... Rest>
```

```cpp
T sum(T first, Rest... rest) {
    return first + sum(rest...);  // recursive: peel off first, recurse on rest
}

sum(1, 2, 3, 4, 5);  // 15
sum(1.1, 2.2, 3.3);  // 6.6
```

`typename... Rest` is a *parameter pack* — zero or more types. `rest...` *expands* the pack.

Fold Expressions (C++17)

Fold expressions provide a cleaner way to apply operators over parameter packs:

```cpp
template<typename... Args>
auto sum(Args... args) {
    return (args + ...);   // left fold: ((a + b) + c) + d...
}

template<typename... Args>
void print_all(Args&&... args) {
    ((std::cout << args << " "), ...);  // comma fold — calls for each
}

print_all(1, "hello", 3.14, true);  // 1 hello 3.14 1
```

Four fold forms: `(pack op ...)` (right fold), `(... op pack)` (left fold), and versions with an initial value.

`if constexpr` in Templates

C++17's `if constexpr` lets you branch on compile-time conditions inside templates:

```cpp
template<typename T>
void process(T value) {
    if constexpr (std::is_integral_v<T>) {
        std::cout << "Integer: " << value << "\n";
    } else if constexpr (std::is_floating_point_v<T>) {
        std::cout << "Float: " << std::fixed << value << "\n";
    } else {
        std::cout << "Other: " << value << "\n";
    }
}
```

Unlike a regular `if`, branches not taken are not compiled — so code that's invalid for a particular type doesn't cause errors.

Key Takeaways

- Full specialization provides a custom template implementation for one specific type.
- Partial specialization (classes only) specializes on patterns of template arguments.

- Variadic templates (`typename... T`) accept any number of types.
- Fold expressions cleanly apply operators over parameter packs without recursion.
- `if constexpr` enables compile-time branching inside templates.

Chapter 25: Concepts (C++20) — Compile-Time Duck Typing

The Problem with Unconstrained Templates

When a template constraint is violated, the error messages are notoriously bad:

```
template<typename T>
T add(T a, T b) { return a + b; }

struct Foo {};
add(Foo{}, Foo{});  // ERROR: but the error message is cryptic —
                    // "no match for operator+" buried in template instantiation
```

The error happens inside the template, far from the call site. With complex templates, error messages can be hundreds of lines.

Concepts

Concepts are named constraints on template parameters. They're checked at the call site, giving clear error messages.

```
#include <concepts>

// Define a concept: T must support + and return T
template<typename T>
concept Addable = requires(T a, T b) {
    { a + b } -> std::convertible_to<T>;
};

// Use the concept to constrain the template
template<Addable T>
T add(T a, T b) { return a + b; }

struct Foo {};
add(Foo{}, Foo{});  // CLEAR ERROR: "Foo does not satisfy Addable"
                    // at the call site
```

Standard Library Concepts

C++20 provides built-in concepts in `<concepts>`:

```
#include <concepts>

std::integral<T>           // T is an integer type
std::floating_point<T>     // T is float/double/long double
std::same_as<T, U>         // T and U are the same type
```

```
std::derived_from<T, Base> // T derives from Base
std::convertible_to<T, U>  // T converts to U
std::equality_comparable<T>// T supports ==
std::totally_ordered<T>    // T supports all comparison operators
std::copyable<T>           // T can be copied
std::movable<T>            // T can be moved
```

Using them:

```
template<std::integral T>
T greatest_common_divisor(T a, T b) {
    while (b != 0) { a %= b; std::swap(a, b); }
    return a;
}

gcd(12, 8);    // OK
gcd(1.5, 2.5); // ERROR: double doesn't satisfy std::integral
```

Abbreviated Function Templates

C++20 allows `auto` parameters with concept constraints as shorthand:

```
// Full syntax
template<std::integral T>
T square(T x) { return x * x; }

// Abbreviated (C++20):
std::integral auto square(std::integral auto x) { return x * x; }
// or just:
auto square(std::integral auto x) { return x * x; }
```

Requires Clauses and Expressions

```
template<typename T>
requires std::is_arithmetic_v<T>  // requires clause
T abs_val(T x) { return x < 0 ? -x : x; }

// Inline requires expression:
template<typename T>
T safe_divide(T a, T b)
requires requires(T x, T y) { x / y; } {   // checks if / is valid
    if (b == T{0}) throw std::invalid_argument("divide by zero");
    return a / b;
}
```

Why Concepts Matter

1. **Clear errors** — constraint violations reported at call site with meaningful messages.
2. **Documentation** — the template's requirements are explicit in the signature.

3. **Overload resolution** — more constrained templates are preferred over less constrained ones.
4. **Replaces SFINAE** — the old (C++11/14) approach of constraining templates was cryptic; concepts are readable.

Key Takeaways

- Concepts are named compile-time constraints on template parameters.
- They give clear error messages at the call site instead of deep inside template instantiation.
- Standard concepts in `<concepts>`: `std::integral`, `std::floating_point`, `std::copyable`, etc.
- C++20's abbreviated syntax: `void f(std::integral auto x)` is shorthand for a constrained template.
- Concepts replace SFINAE — they're the modern, readable way to constrain templates.

Chapter 26: An Introduction to Template Metaprogramming

What Is Template Metaprogramming?

Template metaprogramming (TMP) is using the C++ template system to perform computations at *compile time*. The "program" runs during compilation; the result is embedded in the compiled code.

It sounds esoteric but you already use its outputs: `std::tuple`, type traits, `constexpr` algorithms.

Type Traits

Type traits are compile-time predicates about types. They live in `<type_traits>`:

```
#include <type_traits>

std::is_integral_v<int>        // true
std::is_integral_v<double>     // false
std::is_pointer_v<int*>        // true
std::is_same_v<int, int>       // true
std::is_same_v<int, long>      // false
std::is_base_of_v<Base, Derived> // true if Derived inherits from Base
std::is_copy_constructible_v<T>  // true if T can be copied
```

These are used to branch at compile time:

```
template<typename T>
void serialize(const T& value) {
    if constexpr (std::is_integral_v<T>) {
        write_int(value);
    } else if constexpr (std::is_floating_point_v<T>) {
        write_float(value);
    } else {
        value.serialize_to_stream();  // user-defined method
    }
}
```

Type Transformations

Type traits can transform types at compile time:

```
std::remove_const_t<const int>    // int
std::add_pointer_t<int>           // int*
std::remove_reference_t<int&>     // int
std::decay_t<const int&>          // int  (removes const, ref, array decay)
std::conditional_t<true, int, double>  // int  (compile-time ternary)
```

Computing at Compile Time

Before `constexpr` (Chapter 34), TMP was the only way to compute at compile time. Here's the classic Fibonacci example — educational for understanding TMP recursion:

```
template<int N>
struct Fib {
    static constexpr int value = Fib<N-1>::value + Fib<N-2>::value;
};

template<> struct Fib<0> { static constexpr int value = 0; };
template<> struct Fib<1> { static constexpr int value = 1; };

constexpr int f10 = Fib<10>::value;  // 55 — computed at compile time
```

Today, `constexpr` functions are cleaner for this. TMP shines for type-level computation.

`std::tuple` and Index Sequences

`std::tuple` stores a heterogeneous collection of types — it's TMP in action:

```
#include <tuple>

auto t = std::make_tuple(42, 3.14, std::string("hello"));
auto n = std::get<0>(t);  // 42
auto d = std::get<1>(t);  // 3.14
auto s = std::get<2>(t);  // "hello"

// Size at compile time:
constexpr int sz = std::tuple_size_v<decltype(t)>;  // 3
```

To iterate over a tuple's elements at compile time, you use `std::index_sequence`:

```
template<typename Tuple, std::size_t... Is>
void print_tuple_impl(const Tuple& t, std::index_sequence<Is...>) {
    ((std::cout << std::get<Is>(t) << " "), ...);
}

template<typename Tuple>
void print_tuple(const Tuple& t) {
    print_tuple_impl(t, std::make_index_sequence<std::tuple_size_v<Tuple>>{});
```

```
}

print_tuple(std::make_tuple(1, "hi", 3.14));  // 1 hi 3.14
```

SFINAE (Substitution Failure Is Not An Error)

Before concepts, templates were constrained using SFINAE: a substitution failure in a template argument is not an error — it just removes that overload from consideration.

```
// Old style (C++11/14): enable this overload only for integral T
template<typename T, typename = std::enable_if_t<std::is_integral_v<T>>>
T square(T x) { return x * x; }
```

With concepts (Chapter 25), this becomes:

```
template<std::integral T>
T square(T x) { return x * x; }
```

You'll encounter SFINAE in legacy code. For new code, use concepts.

Key Takeaways

- TMP runs computations during compilation, embedding results in the executable.
- Type traits (`<type_traits>`) are compile-time predicates and transformations on types.
- `if constexpr` uses type traits to branch at compile time inside templates.
- `std::tuple` is the canonical TMP data structure — a heterogeneous compile-time list.
- SFINAE is the old way to constrain templates; concepts (C++20) replaced it with readable syntax.

Part VI — The Standard Library (STL)

Chapter 27: Containers: `vector`, `map`, `set`, `array`, and friends

Container Taxonomy

The STL containers are divided into three categories:

Sequence containers — ordered by position: - `std::vector<T>` — dynamic array (use by default) - `std::array<T, N>` — fixed-size array - `std::deque<T>` — double-ended queue - `std::list<T>` — doubly linked list - `std::forward_list<T>` — singly linked list

Associative containers — ordered by key (sorted): - `std::set<T>` — unique sorted keys - `std::multiset<T>` — sorted keys, duplicates allowed - `std::map<K, V>` — sorted key-value pairs - `std::multimap<K, V>` — sorted, duplicate keys allowed

Unordered containers — hash-based (average O(1)): - `std::unordered_set<T>` - `std::unordered_map<K, V>`

`std::vector` — the Default Choice

Covered in Chapter 10. Prefer `vector` unless you have a specific reason to choose something else. Its cache-friendly contiguous memory beats linked lists in almost all real workloads.

```
std::vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};
std::sort(v.begin(), v.end());   // sorts in place
```

`std::map` — Python's `dict` (sorted)

`std::map<K, V>` is an ordered dictionary. Keys are sorted. Operations are O(log n) (red-black tree internally).

```
# Python
scores = {"Alice": 95, "Bob": 87, "Carol": 92}
scores["Dave"] = 78
print(scores["Alice"])  # 95
```

```
#include <map>
std::map<std::string, int> scores;
scores["Alice"] = 95;
scores["Bob"]   = 87;
scores["Carol"] = 92;
scores["Dave"]  = 78;

std::cout << scores["Alice"] << "\n";  // 95

// Iterate in sorted key order:
for (const auto& [name, score] : scores) {
    std::cout << name << ": " << score << "\n";
}
```

`operator[]` inserts a default-value entry if the key doesn't exist. Use `.find()` when you don't want to accidentally insert:

```
auto it = scores.find("Eve");
if (it != scores.end()) {
    std::cout << it->second;
} else {
    std::cout << "not found\n";
}
```

`std::unordered_map` — Python's `dict` (hash)

For most Python-dict workloads, use `std::unordered_map` — average O(1) lookups vs O(log n) for `std::map`:

```
#include <unordered_map>
std::unordered_map<std::string, int> scores;
scores["Alice"] = 95;
auto it = scores.find("Alice");  // O(1) average
```

Use `std::map` (sorted) when you need ordered iteration or range queries. Use `std::unordered_map` (hash) for pure lookup performance.

`std::set` — Unique Sorted Elements

```python
# Python
unique = {3, 1, 4, 1, 5, 9}  # {1, 3, 4, 5, 9}
```

```cpp
#include <set>
std::set<int> unique = {3, 1, 4, 1, 5, 9};  // {1, 3, 4, 5, 9} — sorted, unique
unique.insert(7);
unique.erase(3);
bool has_5 = unique.count(5) > 0;  // or: unique.contains(5)  (C++20)
```

`std::deque` — Double-Ended Queue

Like `vector` but efficient at both ends. Use when you need frequent `push_front`:

```cpp
#include <deque>
std::deque<int> dq = {2, 3, 4};
dq.push_front(1);   // efficient
dq.push_back(5);    // efficient
dq.pop_front();
```

Choosing the Right Container

```
Need a dynamic array?              → vector
Need fast push/pop at both ends?   → deque
Need sorted unique keys?           → set / map
Need fast hash-based lookup?       → unordered_set / unordered_map
Need fixed size, compile-time?     → array<T, N>
Need O(1) insert anywhere?         → list (but cache-unfriendly)
```

Key Takeaways

- `vector` is the default. It's contiguous memory, cache-friendly, and fast for most operations.
- `map` is ordered (sorted key iteration). `unordered_map` is hash-based (faster lookups). Both are like Python's dict.
- `set` / `unordered_set` for unique elements.
- `operator[]` on map inserts on miss — use `.find()` when you only want to check.
- C++20 adds `.contains()` to all associative containers.

Chapter 28: Iterators

What Is an Iterator?

An iterator is a generalization of a pointer. It provides a uniform interface for traversing any container, regardless of how the container stores its data.

In Python, the iterator protocol is `__iter__` / `__next__`. In C++, an iterator supports `*` (dereference), `++` (advance), and `==`/`!=` (comparison).

```cpp
std::vector<int> v = {10, 20, 30, 40};

// Iterator-based loop (equivalent to range-based for):
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << "\n";  // *it dereferences the iterator
}
```

`v.begin()` returns an iterator to the first element. `v.end()` returns an iterator *one past the last element* (a sentinel — never dereference it).

Iterator Categories

Iterators come in categories based on what operations they support:

Category	Supports	Example
Input iterator	Read once, forward only	`istream_iterator`
Output iterator	Write once, forward only	`ostream_iterator`
Forward iterator	Read/write, forward	`forward_list`
Bidirectional iterator	Forward + backward (`--`)	`list`, `map`
Random access iterator	+ arithmetic, `[]`, `<`	`vector`, `deque`
Contiguous iterator	Random + contiguous memory	`vector`, `array`

Algorithms require a minimum iterator category. `std::sort` needs random access iterators — it works on `vector` but not `list`.

Common Iterator Operations

```cpp
std::vector<int> v = {10, 20, 30, 40, 50};

auto first = v.begin();       // iterator to first element
auto last  = v.end();         // one past last
auto mid   = v.begin() + 2;   // random access: iterator to element 2 (30)

*mid = 99;                    // modify through iterator

std::advance(first, 3);       // advance any iterator by n (works for all categories)
auto dist = std::distance(v.begin(), mid);  // distance between iterators
```

Reverse Iterators

Traverse in reverse:

```cpp
for (auto it = v.rbegin(); it != v.rend(); ++it) {
    std::cout << *it << "\n";  // prints 50 40 30 20 10
}
```

Insert Iterators

Special iterators that insert elements into containers when assigned:

```cpp
#include <iterator>

std::vector<int> src = {1, 2, 3};
std::vector<int> dst;

std::copy(src.begin(), src.end(), std::back_inserter(dst));
// back_inserter calls dst.push_back() for each element
```

Iterators with Algorithms

Iterators are the glue between containers and algorithms. Algorithms take iterator pairs:

```cpp
std::vector<int> v = {5, 3, 1, 4, 2};

std::sort(v.begin(), v.end());              // sort all
std::sort(v.begin(), v.begin() + 3);        // sort first 3 only

auto it = std::find(v.begin(), v.end(), 4); // find value 4
if (it != v.end()) std::cout << "found at index " << (it - v.begin());

// Work on a subrange:
std::fill(v.begin() + 1, v.begin() + 4, 0);  // set elements 1-3 to 0
```

Key Takeaways

- Iterators generalize pointers: `*it` dereferences, `++it` advances, `it != end` checks for completion.
- `begin()` points to the first element; `end()` is one past the last (a sentinel).
- Iterator categories determine what algorithms can use a container.
- `std::advance`, `std::distance` work across all iterator categories.
- Iterators are the foundation of the STL algorithm library — understanding them unlocks everything.

Chapter 29: Algorithms: `sort`, `find`, `transform`, and the rest

The STL Algorithm Philosophy

Instead of building algorithms into containers (like Python's `list.sort()`), the STL separates algorithms from data structures via iterators. Any algorithm works with any container that provides compatible iterators.

All algorithms live in `<algorithm>` (plus some in `<numeric>`).

Sorting

```cpp
#include <algorithm>
std::vector<int> v = {5, 3, 1, 4, 2};

std::sort(v.begin(), v.end());                          // ascending
std::sort(v.begin(), v.end(), std::greater<int>());     // descending
std::sort(v.begin(), v.end(), [](int a, int b) {        // custom comparator
    return a > b;
});

// Partial sort: put smallest 3 in sorted order
std::partial_sort(v.begin(), v.begin() + 3, v.end());

// Stable sort: preserves relative order of equal elements
std::stable_sort(v.begin(), v.end());
```

Searching

```cpp
std::vector<int> v = {1, 2, 3, 4, 5};

// Linear search: O(n)
auto it = std::find(v.begin(), v.end(), 3);
bool found = (it != v.end());

// Find first element matching predicate:
auto it2 = std::find_if(v.begin(), v.end(), [](int x) { return x > 3; });

// Binary search on sorted range: O(log n)
bool has_3 = std::binary_search(v.begin(), v.end(), 3);
auto pos   = std::lower_bound(v.begin(), v.end(), 3);  // first element >= 3
auto pos2  = std::upper_bound(v.begin(), v.end(), 3);  // first element > 3
```

Transforming

```python
# Python
squares = list(map(lambda x: x*x, [1, 2, 3, 4, 5]))
```

```cpp
// C++
std::vector<int> src = {1, 2, 3, 4, 5};
std::vector<int> dst(5);

std::transform(src.begin(), src.end(), dst.begin(), [](int x) {
    return x * x;
});
// dst = {1, 4, 9, 16, 25}

// Transform two ranges into one:
std::transform(a.begin(), a.end(), b.begin(), dst.begin(), std::plus<int>());
```

```
// dst[i] = a[i] + b[i]
```

Filtering: `copy_if` and `remove_if`

```
# Python
evens = list(filter(lambda x: x % 2 == 0, numbers))
```

```
// C++: copy elements matching predicate into another container
std::vector<int> src = {1, 2, 3, 4, 5, 6};
std::vector<int> evens;
std::copy_if(src.begin(), src.end(), std::back_inserter(evens),
             [](int x) { return x % 2 == 0; });
// evens = {2, 4, 6}
```

Removing from a vector uses the erase-remove idiom:

```
// Remove all even numbers from v:
v.erase(std::remove_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; }),
        v.end());
```

`std::remove_if` moves elements to be kept to the front and returns an iterator to where "deleted" elements start. `erase` then removes the tail.

Counting and Checking

```
std::vector<int> v = {1, 2, 3, 4, 5};

int cnt   = std::count(v.begin(), v.end(), 3);              // count occurrences of 3
int cnt2  = std::count_if(v.begin(), v.end(), [](int x) { return x > 3; }); // 2

bool all  = std::all_of(v.begin(), v.end(), [](int x) { return x > 0; });  // true
bool any  = std::any_of(v.begin(), v.end(), [](int x) { return x > 4; });  // true
bool none = std::none_of(v.begin(), v.end(), [](int x) { return x > 10; }); // true
```

Numeric Algorithms (`<numeric>`)

```
#include <numeric>

std::vector<int> v = {1, 2, 3, 4, 5};

int sum     = std::accumulate(v.begin(), v.end(), 0);       // 15
int product = std::accumulate(v.begin(), v.end(), 1, std::multiplies<int>()); // 120

// Prefix sums:
std::vector<int> prefix(v.size());
std::partial_sum(v.begin(), v.end(), prefix.begin());  // {1, 3, 6, 10, 15}
```

Key Takeaways

- STL algorithms work on iterator ranges — they're container-agnostic.
- `find`, `find_if` for search; `sort`, `stable_sort` for ordering; `transform` for mapping; `copy_if` for filtering.
- The erase-remove idiom: `v.erase(remove_if(...), v.end())` — the canonical way to delete elements matching a condition.
- `all_of`, `any_of`, `none_of` for predicate checks over a range.
- `<numeric>` has `accumulate` (reduce/fold), `partial_sum`, `iota`, and more.

Chapter 30: Lambdas and Function Objects

What Is a Lambda?

A lambda is an anonymous function defined inline. Python has `lambda` too, but C++ lambdas are far more powerful.

```python
# Python
square = lambda x: x * x
result = square(5)  # 25

nums = [1, 2, 3, 4, 5]
evens = list(filter(lambda x: x % 2 == 0, nums))
```

```cpp
// C++
auto square = [](int x) { return x * x; };
int result = square(5);  // 25

std::vector<int> nums = {1, 2, 3, 4, 5};
std::vector<int> evens;
std::copy_if(nums.begin(), nums.end(), std::back_inserter(evens),
             [](int x) { return x % 2 == 0; });
```

Lambda Syntax

```
[captures](parameters) -> return_type { body }
```

- **Captures**: variables from the enclosing scope the lambda can use.
- **Parameters**: like a normal function.
- **Return type**: usually deduced; specify with `->` if needed.
- **Body**: the function body.

```cpp
int threshold = 5;

auto greater_than_threshold = [threshold](int x) {   // captures threshold by value
    return x > threshold;
};

int count = std::count_if(v.begin(), v.end(), greater_than_threshold);
```

Capture Modes

```
int x = 10, y = 20;

auto f1 = [x]()    { return x; };         // capture x by value (copy)
auto f2 = [&x]()   { return x; };         // capture x by reference
auto f3 = [=]()    { return x + y; };     // capture ALL used vars by value
auto f4 = [&]()    { x = 99; return y; }; // capture ALL used vars by reference
auto f5 = [=, &y]() { return x + y; };    // all by value except y by reference
```

Capture by value: lambda has its own copy; changing it doesn't affect the original. Capture by reference: lambda sees changes to the original, and can modify it.

Danger: capturing by reference can create dangling references if the lambda outlives the captured variables. When in doubt, capture by value.

```
std::function<int()> make_adder(int base) {
    return [base]() { return base + 1; };   // OK: captures by value
    // return [&base]() { return base + 1; }; // DANGER: base is a local
}
```

Mutable Lambdas

By default, captured-by-value variables are `const` inside the lambda. Use `mutable` to allow modification of the copy:

```
int count = 0;
auto counter = [count]() mutable { return ++count; };  // modifies the copy
counter();  // 1
counter();  // 2
std::cout << count;  // still 0 — the original is unchanged
```

Generic Lambdas (C++14)

Parameters can be `auto`, making the lambda a template:

```
auto square = [](auto x) { return x * x; };

square(3);     // int
square(3.14);  // double
square(2.0f);  // float
```

Immediately Invoked Lambdas

Lambdas can be defined and called in one expression — useful for complex initialization:

```
const std::string name = []() -> std::string {
    if (std::getenv("USER")) return std::getenv("USER");
    return "unknown";
}();
```

std::function

`std::function<R(Args...)>` is a type-erased function wrapper — it holds any callable with the right signature:

```
#include <functional>

std::function<int(int, int)> op;

op = [](int a, int b) { return a + b; };
std::cout << op(3, 4);  // 7

op = [](int a, int b) { return a * b; };
std::cout << op(3, 4);  // 12
```

`std::function` has overhead (heap allocation, virtual dispatch). For performance-critical code, prefer template parameters or `auto` to store lambdas directly.

Key Takeaways

- Lambdas are anonymous functions with `[capture](params) { body }` syntax.
- Capture by value `[=]` for safety (no dangling refs), by reference `[&]` when you need to modify outer state.
- `mutable` lets the lambda modify its captured-by-value copies.
- C++14 generic lambdas use `auto` parameters — they're effectively function templates.
- `std::function` is a type-erased wrapper for any callable — convenient but has overhead.

Chapter 31: Ranges and Views (C++20)

The Problem with Iterator Pairs

The traditional STL algorithm interface takes two iterators:

```
std::sort(v.begin(), v.end());
auto it = std::find(v.begin(), v.end(), 5);
```

This is verbose and doesn't compose well. What if you want to filter, then transform, then sort? You need intermediary vectors.

Ranges

C++20 ranges treat containers as single entities (not pairs of iterators):

```
#include <algorithm>
#include <ranges>

std::vector<int> v = {5, 3, 1, 4, 2};

std::ranges::sort(v);                  // cleaner than sort(v.begin(), v.end())
auto it = std::ranges::find(v, 3);     // cleaner than find(v.begin(), v.end(), 3)
```

Views — Lazy Pipelines

Views are lightweight, lazy range adapters that transform ranges without allocating. They compose with `|`:

```python
# Python (equivalent)
result = [x*x for x in range(10) if x % 2 == 0]
```

```cpp
// C++20 ranges
#include <ranges>

auto result = std::views::iota(0, 10)        // 0, 1, 2, ... 9
            | std::views::filter([](int x) { return x % 2 == 0; })  // 0, 2, 4, 6, 8
            | std::views::transform([](int x) { return x * x; });   // 0, 4, 16, 36, 64

for (int v : result) std::cout << v << " ";
// No intermediate vectors — values computed lazily on demand
```

The `|` operator creates a pipeline. Each step is lazy: values flow through only when you actually iterate.

Common Views

```cpp
namespace sv = std::views;

sv::filter(pred)          // keep elements matching pred
sv::transform(fn)         // apply fn to each element
sv::take(n)               // first n elements
sv::drop(n)               // skip first n elements
sv::take_while(pred)      // take while pred is true
sv::drop_while(pred)      // drop while pred is true
sv::reverse               // iterate in reverse
sv::iota(start, end)      // generates start, start+1, ..., end-1
sv::enumerate             // pairs of (index, value)  (C++23)
sv::zip(r1, r2)           // pairs of elements from two ranges  (C++23)
sv::keys                  // keys of a map-like range
sv::values                // values of a map-like range
```

Materializing a View into a Vector

Views are lazy — to get an actual `vector`, use `std::ranges::to` (C++23) or the copy idiom:

```cpp
// C++23:
auto vec = sv::iota(0, 10) | sv::filter([](int x) { return x % 2 == 0; })
         | std::ranges::to<std::vector>();

// C++20 (before std::ranges::to):
std::vector<int> vec;
auto view = sv::iota(0, 10) | sv::filter([](int x) { return x % 2 == 0; });
std::ranges::copy(view, std::back_inserter(vec));
```

Key Takeaways

- `std::ranges::sort(v)` is cleaner than `std::sort(v.begin(), v.end())`.
- Views are lazy range adapters composed with `|`.
- No intermediate containers — data flows through the pipeline only when consumed.
- C++20 views: `filter`, `transform`, `take`, `drop`, `reverse`, `iota`.
- C++23 adds `enumerate`, `zip`, `std::ranges::to`.

Chapter 32: Utility Types: `optional`, `variant`, `any`, `tuple`

`std::optional` — A Value That May Not Exist

```python
# Python: return None to signal absence
def find_user(id):
    if id in database:
        return database[id]
    return None
```

```cpp
// C++: return std::optional<User>
#include <optional>

std::optional<User> find_user(int id) {
    if (database.count(id)) return database[id];
    return std::nullopt;   // nothing
}

auto user = find_user(42);
if (user.has_value()) {
    std::cout << user->name;   // access with ->
    std::cout << (*user).name; // or dereference
}

// With default:
std::string name = user.value_or(User{}).name;

// Or use if directly:
if (auto u = find_user(42)) {
    std::cout << u->name;
}
```

`optional<T>` wraps a `T` and a boolean. It's a 1-element container. Use it instead of returning sentinel values (`-1`, `nullptr`, `""`) or throwing exceptions for "not found."

`std::variant` — A Type-Safe Union

`variant<A, B, C>` holds exactly one value, which can be of type `A`, `B`, or `C`. Like a union, but safe — it knows which type it currently holds.

```python
# Python: duck typing / isinstance checks
def process(value):
    if isinstance(value, int):
        return value * 2
    elif isinstance(value, str):
        return value.upper()
```

```cpp
// C++
#include <variant>

std::variant<int, std::string, double> v = 42;

// Check type:
if (std::holds_alternative<int>(v)) {
    std::cout << std::get<int>(v) * 2;
}

// Pattern match with std::visit:
std::visit([](auto&& val) {
    using T = std::decay_t<decltype(val)>;
    if constexpr (std::is_same_v<T, int>)
        std::cout << val * 2;
    else if constexpr (std::is_same_v<T, std::string>)
        std::cout << val.size();
    else
        std::cout << val;
}, v);
```

`std::visit` is the clean way to handle all cases. If you miss a type, the compiler warns (or errors with overloaded lambdas).

`std::any` — Any Type at All

`std::any` stores a value of any type, with type erasure. Less safe than `variant` (no compile-time type checking):

```cpp
#include <any>

std::any a = 42;
a = std::string("hello");
a = 3.14;

// Extract with std::any_cast:
try {
    double d = std::any_cast<double>(a);
    std::cout << d;
} catch (const std::bad_any_cast& e) {
    std::cout << "wrong type\n";
}
```

Use `any` when the type truly can't be known at compile time. Prefer `variant` when the set of types is finite and known.

`std::tuple` — Heterogeneous Fixed Collection

```python
# Python: return multiple values
def get_info():
    return "Alice", 30, True
name, age, active = get_info()
```

```cpp
// C++
#include <tuple>

std::tuple<std::string, int, bool> get_info() {
    return {"Alice", 30, true};
}

auto [name, age, active] = get_info();  // structured bindings (C++17)
std::cout << name << " " << age << "\n";

// Access by index:
auto info = get_info();
std::cout << std::get<0>(info);  // "Alice"
std::cout << std::get<1>(info);  // 30
```

`std::pair<A, B>` is essentially `tuple<A, B>` — it's the return type of `std::map` iterators (`it->first`, `it->second`).

Summary Table

Type	Use When
`optional<T>`	Value may or may not exist (like Python's `None`)
`variant<A,B>`	One of a known set of types
`any`	Any type at all (runtime flexibility)
`tuple<A,B,C>`	Fixed heterogeneous collection (multiple return values)

Key Takeaways

- `optional<T>` replaces null/sentinel patterns. Check with `has_value()` or `if (opt)`.
- `variant<A,B,C>` is a type-safe union. Use `std::visit` to pattern-match over types.
- `any` is type erasure for truly dynamic values. Prefer `variant` when types are known.
- `tuple` returns multiple values. C++17 structured bindings (`auto [a, b] = ...`) make it ergonomic.

Part VII — Modern C++ (C++11 → C++23)

Chapter 33: `auto`, type deduction, and structured bindings

`auto` Type Deduction

`auto` asks the compiler to deduce the type from the initializer. The type is still fixed at compile time — `auto` is not dynamic typing.

```
auto x = 42;                    // int
auto y = 3.14;                  // double
auto s = std::string{"hello"};  // std::string
auto v = std::vector<int>{1,2,3}; // std::vector<int>

auto it = v.begin();            // std::vector<int>::iterator (ugly to write manually)
auto [first, second] = make_pair(1, 2); // structured binding
```

The rules mirror function template deduction (Chapter 23). Key points:

```
auto x = 5;       // int (not const int, not int&)
const auto cx = 5; // const int
auto& rx = x;     // int& — reference to x
const auto& crx = x; // const int&
```

`auto` strips top-level `const` and references. If you want them, add them explicitly.

`decltype`

`decltype(expr)` gives you the type of an expression without evaluating it:

```
int x = 5;
decltype(x)  y = 10;      // int (same type as x)
decltype(x+1.0) z = 3.0;  // double (type of x+1.0)

// Useful in templates:
template<typename A, typename B>
auto add(A a, B b) -> decltype(a + b) {  // trailing return type
    return a + b;
}
```

C++14 simplifies this: return type deduction works without the trailing type:

```
template<typename A, typename B>
auto add(A a, B b) {  // compiler deduces return type
    return a + b;
}
```

Structured Bindings (C++17)

Structured bindings destructure aggregates, pairs, tuples, and arrays:

```python
# Python
a, b, c = (1, 2, 3)
first, *rest = [1, 2, 3, 4]
```

```cpp
// C++17
auto [a, b, c] = std::tuple{1, 2.0, "three"};

// With pairs (map iteration is idiomatic with this):
std::map<std::string, int> scores = {{"Alice", 95}, {"Bob", 87}};
for (auto& [name, score] : scores) {
    std::cout << name << ": " << score << "\n";
}

// With structs:
struct Point { int x, y, z; };
Point p{1, 2, 3};
auto [x, y, z] = p;

// With arrays:
int arr[3] = {10, 20, 30};
auto [a, b, c] = arr;
```

Structured bindings bind by value by default. Use `auto&` to bind by reference (and modify the original):

```cpp
for (auto& [name, score] : scores) {
    score += 5;  // bumps every score in the map
}
```

`auto` in Function Parameters (C++20)

```cpp
// C++20: auto parameters make functions implicitly templated
void print(auto x) {
    std::cout << x << "\n";
}

print(42);      // instantiated for int
print("hello"); // instantiated for const char*
print(3.14);    // instantiated for double
```

This is equivalent to:

```cpp
template<typename T>
void print(T x) { std::cout << x << "\n"; }
```

Key Takeaways

- `auto` deduces the type from the initializer — still statically typed, determined at compile time.

- `auto` strips const and references. Add them explicitly: `const auto&`.
- `decltype(expr)` queries the type of an expression without evaluating it.
- Structured bindings (`auto [a, b] = ...`) destructure pairs, tuples, arrays, and structs.
- C++20 `auto` parameters create implicitly templated functions.

Chapter 34: `constexpr` and compile-time computation

The Motivation

Computations done at compile time are "free" at runtime — the result is baked directly into the executable. C++ uses `constexpr` to mark values and functions that must (or can) be evaluated at compile time.

```
constexpr int factorial(int n) {
    return n <= 1 ? 1 : n * factorial(n - 1);
}

constexpr int f5 = factorial(5);  // 120 — computed at compile time
int arr[factorial(5)];            // array size must be compile-time constant
```

`constexpr` Variables

A `constexpr` variable is initialized at compile time and is implicitly `const`:

```
constexpr double PI = 3.14159265358979;
constexpr int CACHE_LINE = 64;
constexpr std::size_t MAX_PLAYERS = 8;
```

`constexpr` Functions (C++11–C++23 evolution)

C++11 constexpr functions were very limited (single return statement only). Each standard relaxed the restrictions:

```
// C++14+: loops, local variables, conditionals all allowed
constexpr int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

constexpr int g = gcd(48, 18);  // 6 — at compile time
```

C++20 allows `constexpr` functions to use `try/catch`, `dynamic_cast`, virtual functions, and even allocate memory (though allocations must be freed before the expression completes).

`consteval` (C++20) — Must Be Compile-Time

`consteval` functions *must* be evaluated at compile time. Calling them with runtime arguments is a compile error:

```
consteval int compile_time_square(int x) {
    return x * x;
}

constexpr int a = compile_time_square(5);  // OK — 25 at compile time
// int n = 5; compile_time_square(n);       // ERROR — n is runtime
```

Use `consteval` for functions that only make sense at compile time (e.g., parsing format strings, generating lookup tables).

`constinit` (C++20) — Guaranteed Initialization Order

Static variables with dynamic initialization have a dreaded "static initialization order fiasco." `constinit` ensures a variable is initialized at compile time (avoiding this problem) but doesn't make it `const`:

```
constinit int counter = 0;  // initialized at compile time, can be modified later
++counter;  // OK — not const
```

Compile-Time Data Structures

C++20 made many standard library components `constexpr`, including `std::vector` and `std::string` (within constant expressions):

```
constexpr int sum_of_first_n(int n) {
    std::vector<int> v;  // vector in constexpr context (C++20)
    for (int i = 1; i <= n; ++i) v.push_back(i);
    int s = 0;
    for (int x : v) s += x;
    return s;
}

constexpr int s = sum_of_first_n(100);  // 5050 — computed at compile time
```

Key Takeaways

- `constexpr` variables are initialized at compile time, embedded in the executable.
- `constexpr` functions can be evaluated at compile time (when given constant arguments) or at runtime.
- `consteval` (C++20) forces compile-time evaluation — a runtime call is a compile error.
- `constinit` (C++20) guarantees compile-time initialization of mutable statics.
- C++20 made `vector` and `string` usable in `constexpr` contexts.

Chapter 35: `std::format` and modern string handling

The Evolution of String Formatting

C: `printf("Hello, %s! You are %d years old.\n", name, age);` — fast but unsafe (no type checking).

Old C++: `std::cout << "Hello, " << name << "! You are " << age << " years old.\n";` — verbose.

C++20: `std::format` — Python-style, type-safe, and fast.

```
# Python f-string
name, age = "Alice", 30
msg = f"Hello, {name}! You are {age} years old."
```

```
// C++20 std::format
#include <format>
std::string name = "Alice";
int age = 30;
std::string msg = std::format("Hello, {}! You are {} years old.", name, age);
```

Format Specifiers

The syntax is nearly identical to Python's format strings:

```
// Width and alignment
std::format("{:10}", "left");    // "left      " (left-aligned in 10 chars)
std::format("{:>10}", "right");  // "     right" (right-aligned)
std::format("{:^10}", "center"); // "  center  " (centered)

// Numbers
std::format("{:05d}", 42);       // "00042" (zero-padded)
std::format("{:.2f}", 3.14159);  // "3.14" (2 decimal places)
std::format("{:e}", 12345.678);  // "1.234568e+04" (scientific)
std::format("{:x}", 255);        // "ff" (hex lowercase)
std::format("{:X}", 255);        // "FF" (hex uppercase)
std::format("{:b}", 10);         // "1010" (binary)

// Positional arguments
std::format("{0} + {1} = {0}", 1, 2);  // "1 + 2 = 1"
```

`std::print` (C++23)

C++23 adds `std::print` — formats and prints in one call without constructing an intermediate string:

```
#include <print>
std::print("Hello, {}! You are {} years old.\n", name, age);
std::println("This appends a newline automatically.");
```

Formatting Custom Types

Specialize `std::formatter<T>` to make your types formattable:

```cpp
struct Point { double x, y; };

template<>
struct std::formatter<Point> {
    constexpr auto parse(std::format_parse_context& ctx) { return ctx.begin(); }

    auto format(const Point& p, std::format_context& ctx) const {
        return std::format_to(ctx.out(), "({}, {})", p.x, p.y);
    }
};

Point p{3.0, 4.0};
std::string s = std::format("Point: {}", p);  // "Point: (3, 4)"
```

String Manipulation

```cpp
#include <string>
#include <algorithm>

std::string s = "  Hello, World!  ";

// Python: s.strip() / lstrip() / rstrip()
// C++: manual or use std::string algorithms
auto start = s.find_first_not_of(" \t\n");
auto end   = s.find_last_not_of(" \t\n");
std::string trimmed = s.substr(start, end - start + 1);

// Python: s.split(',')
// C++: manual or use std::views::split (C++20)
#include <ranges>
for (auto word : s | std::views::split(',')) {
    std::cout << std::string_view(word) << "\n";
}

// Python: s.replace('l', 'L')
// C++:
std::string replaced = s;
std::replace(replaced.begin(), replaced.end(), 'l', 'L');

// Convert to upper/lower:
std::transform(s.begin(), s.end(), s.begin(), ::toupper);
```

Key Takeaways

- `std::format` (C++20) is Python-style type-safe string formatting.
- `std::print` (C++23) formats and writes without an intermediate string.
- Format specifiers: `{:05d}`, `{:.2f}`, `{:x}`, `{:>10}` — nearly identical to Python.

- Specialize `std::formatter<T>` to make custom types formattable.
- `std::string` manipulation uses `substr`, `find`, `replace`, and `<algorithm>` functions.

Chapter 36: Coroutines and Generators

What Is a Coroutine?

A coroutine is a function that can suspend its execution and later resume. Python has had them since 3.5 (`async def`, `yield`). C++20 added native coroutine support.

```python
# Python generator
def count_up(n):
    for i in range(n):
        yield i

for x in count_up(5):
    print(x)  # 0, 1, 2, 3, 4
```

```cpp
// C++20 generator (using a library — standard generator comes in C++23)
#include <generator>  // C++23

std::generator<int> count_up(int n) {
    for (int i = 0; i < n; ++i)
        co_yield i;
}

for (int x : count_up(5)) {
    std::cout << x << "\n";
}
```

Coroutine Keywords

C++20 coroutines use three keywords:

- `co_yield expr` — suspend the coroutine and yield a value (like Python's `yield`)
- `co_return expr` — complete the coroutine (like `return` in a regular function)
- `co_await expr` — suspend waiting for another async operation (like Python's `await`)

A function containing any of these becomes a coroutine.

The Coroutine Machinery

Unlike Python (where the runtime handles coroutines), C++ coroutines require you to provide a *promise type* — a class that controls suspension, resumption, and value transfer. This is complex but gives total control over memory allocation and scheduling.

The standard library's `std::generator<T>` (C++23) wraps all this machinery for the generator use case:

```cpp
#include <generator>
#include <ranges>

std::generator<int> fibonacci() {
    int a = 0, b = 1;
    while (true) {
        co_yield a;
        auto tmp = a + b;
        a = b;
        b = tmp;
    }
}

// Take the first 10 Fibonacci numbers:
for (int x : fibonacci() | std::views::take(10)) {
    std::cout << x << " ";  // 0 1 1 2 3 5 8 13 21 34
}
```

Async Coroutines

For async I/O (network requests, file operations), coroutines avoid blocking threads. This is the same use case as Python's `asyncio`:

```cpp
// Pseudocode — actual async requires a framework (ASIO, cppcoro, etc.)
Task<std::string> fetch_url(std::string url) {
    auto connection = co_await open_connection(url);
    auto data       = co_await read_all(connection);
    co_return data;
}
```

The C++ standard doesn't ship an async runtime (unlike Python's asyncio). You need a library. Popular choices: Asio (Boost.Asio or standalone Asio), cppcoro, libunifex.

Key Takeaways

- C++20 coroutines use `co_yield`, `co_return`, `co_await`.
- `std::generator<T>` (C++23) is the simple generator type — equivalent to Python's `yield`.
- Coroutines suspend without blocking a thread — ideal for lazy sequences and async I/O.
- The coroutine machinery (promise types) is complex but customizable — giving C++ more power than Python's fixed runtime model.
- For async I/O, pair with a library like Asio.

Chapter 37: Modules (C++20)

The Problem with Headers

The `#include` preprocessor is 50 years old. Its problems: - **Slow**: headers are pasted textually into every file that includes them. The standard library headers are huge. - **Order-dependent**:

symbols depend on include order. - **Macro leakage**: `#define` in one header pollutes every file that includes it. - **Redundant work**: every translation unit re-parses every included header.

Modules: The Solution

C++20 modules precompile to binary module interface units (BMI). They're included once, not re-parsed. Macros don't leak.

```
// math.cppm — a module (note the .cppm extension)
export module math;  // declares this file as module 'math'

export int add(int a, int b) {    // 'export' makes it visible to importers
    return a + b;
}

int helper(int x) { return x * 2; }  // NOT exported — internal to the module
```

```
// main.cpp — importing a module
import math;      // import the module (not #include)
import <iostream>; // import standard library (much faster than #include)

int main() {
    std::cout << add(3, 4) << "\n";  // 7
    // helper(5);  // ERROR — not exported
}
```

Standard Library Modules (C++23)

C++23 standardizes importing the entire standard library:

```
import std;         // everything in the standard library
import std.compat;  // + C compatibility headers (printf, etc.)
```

This replaces dozens of `#include <...>` headers and compiles dramatically faster.

Module Partitions

Large modules can be split into partitions:

```
// geometry-shapes.cppm
export module geometry:shapes;  // module 'geometry', partition 'shapes'

export struct Circle { double radius; };
export struct Rectangle { double width, height; };

// geometry-algorithms.cppm
export module geometry:algorithms;

import geometry:shapes;  // import another partition
export double area(const Circle& c);
export double area(const Rectangle& r);
```

```
// geometry.cppm — the primary module interface
export module geometry;
export import :shapes;      // re-export the shapes partition
export import :algorithms;  // re-export the algorithms partition
```

Build System Support

Modules require build system support because the dependency order of compilation matters (a module must be compiled before its importers). Support as of 2024:

- **CMake 3.28+**: native modules support
- **MSVC**: first-class support
- **GCC 14+**: mostly supported
- **Clang 16+**: mostly supported

Adoption is growing. New projects should use modules. Legacy code continues with headers.

Key Takeaways

- Modules replace `#include` with a precompiled binary interface.
- `export module name;` declares a module. `export` on a declaration makes it visible to importers.
- `import name;` imports a module — no macro leakage, no re-parsing.
- C++23's `import std;` replaces all standard library includes.
- Modules dramatically improve compile times for large projects.

Part VIII — The Cost Model & Performance

Chapter 38: Value vs Reference Semantics

Python Is All Reference Semantics

In Python, variables hold references (pointers) to objects. Assignment copies the reference:

```
a = [1, 2, 3]
b = a         # b and a point to the same list
b.append(4)
print(a)      # [1, 2, 3, 4] — a was modified through b
```

Everything is a reference. Mutation is the default. This is easy to reason about for simple cases but leads to surprising aliasing in complex programs.

C++ Defaults to Value Semantics

In C++, assignment copies the value. Objects are independent by default:

```cpp
std::vector<int> a = {1, 2, 3};
std::vector<int> b = a;    // b is a COPY of a
b.push_back(4);
std::cout << a.size();     // 3 — a unchanged
```

This is *value semantics*: objects are values, copying creates independent objects. It's the safe default.

Opting Into Reference Semantics

When you want sharing or aliasing, you're explicit about it:

```cpp
std::vector<int> a = {1, 2, 3};
std::vector<int>& b = a;   // b is a reference (alias) to a
b.push_back(4);
std::cout << a.size();     // 4 — a was modified through b
```

This is *reference semantics* — you say explicitly "b is an alias."

Why Value Semantics Is Better (Usually)

With value semantics: - Function arguments can't surprise you by modifying your data (unless you pass by reference). - You can reason about code locally — no need to track who else holds a reference. - No aliasing bugs. - Move semantics make "copying" cheap for large objects when the source won't be used again.

```cpp
void process(std::vector<int> data) {  // takes by VALUE — safe
    data.sort();  // sorts local copy
    // caller's data is unaffected
}
```

When to Use Reference Semantics

- **`const T&`**: pass large objects cheaply for read-only access.
- **`T&`**: pass for write access (rare — prefer returning values or using return values).
- **`T&&`** / **`std::move`**: transfer ownership without copying.
- **`shared_ptr<T>`**: when multiple owners genuinely need to share the same object.

The guideline: start with value semantics. Opt into reference semantics only when performance requires it or sharing is the correct model.

The Performance Angle

Value semantics isn't slower — it's often faster:

```cpp
// Reference semantics (Python-style) — pointer chasing:
struct NodeRef {
    std::shared_ptr<int> data;  // heap allocation + pointer hop
};

// Value semantics — data is inline:
```

```
struct NodeVal {
    int data;                    // directly in the struct — one contiguous access
};
```

A `vector<int>` is contiguous integers in memory. Iterating it is fast because the CPU's cache prefetcher works perfectly. A `vector<shared_ptr<int>>` is a vector of pointers — each * hop is a potential cache miss.

Key Takeaways

- C++ defaults to value semantics: assignment copies. Reference semantics require explicit `&` or smart pointers.
- Value semantics prevent aliasing bugs and enable local reasoning.
- Use `const T&` for cheap read access, `T&&` for transfer of ownership.
- Containers of values (`vector<T>`) are cache-friendly and fast. Containers of pointers (`vector<T*>`) cause pointer chasing and cache misses.

Chapter 39: How Memory Layout Affects Speed (cache, locality)

The Memory Hierarchy

Modern CPUs are orders of magnitude faster than RAM. To bridge the gap, the CPU has a hierarchy of caches:

```
CPU registers    ~0.3 ns    ~256 bytes
L1 cache         ~1 ns      ~32 KB
L2 cache         ~4 ns      ~256 KB
L3 cache         ~10-50 ns  ~8-32 MB
RAM              ~60-100 ns ~GB
```

A cache miss — needing data that's not in the cache — causes the CPU to stall for 60-100 ns while it fetches from RAM. At 3 GHz, 100 ns is ~300 cycles — 300 operations that could have happened.

The key insight: **spatial locality** and **temporal locality** determine cache behavior.

- **Spatial locality**: accessing nearby memory is cheap (hardware prefetches cache lines of 64 bytes).
- **Temporal locality**: recently accessed data is likely cached.

Array of Structs vs Struct of Arrays

This is the most impactful layout decision in systems programming.

Array of Structs (AoS) — natural OOP layout:

```
struct Particle {
    float x, y, z;     // position
    float vx, vy, vz;  // velocity
    float mass;
    int   type;
    // 32 bytes total
```

```
};
std::vector<Particle> particles(1'000'000);

// Updating positions:
for (auto& p : particles) {
    p.x += p.vx;  // reads x, vx, writes x — but also loads y, z, vy, vz, mass, type
    p.y += p.vy;  // already in cache
    p.z += p.vz;  // already in cache
}
```

Struct of Arrays (SoA) — data-oriented layout:

```
struct Particles {
    std::vector<float> x, y, z;
    std::vector<float> vx, vy, vz;
    std::vector<float> mass;
    std::vector<int>   type;
};
Particles particles;
// resize each to 1'000'000

// Updating positions:
for (int i = 0; i < n; ++i) {
    particles.x[i] += particles.vx[i];
}
// When we access x[0], the cache loads x[0..15] (64 bytes / 4 bytes per float).
// The next 15 iterations are cache hits — no stalls.
```

For operations that only access *some* fields, SoA is faster because you only load the data you actually need.

Linked Lists Are Slow

Textbooks use linked lists for teaching algorithms. In practice, they're often the wrong choice:

```
std::list<int> vs. std::vector<int>
```

A `std::list` node has a value, and two pointers (`next`, `prev`). These nodes are scattered throughout the heap. Traversing the list means following a pointer to a random memory address for each node — every step is likely a cache miss.

A `std::vector` is contiguous. Iterating it streams through sequential memory — the prefetcher loads ahead and you almost never miss.

Real-world benchmark: iterating and summing 1 million ints, vector vs list. Vector is typically 10-50x faster.

False Sharing

On multi-core CPUs, each core has its own L1/L2 cache. Cache lines (64 bytes) are the unit of transfer. If two threads modify different variables that happen to be on the same cache line, they

thrash each other's caches:

```
struct Counters {
    int counter_a;  // thread 1 writes this
    int counter_b;  // thread 2 writes this
    // both on the same cache line — false sharing!
};

// Fix: align to cache line boundaries
struct Counters {
    alignas(64) int counter_a;
    alignas(64) int counter_b;
};
```

Key Takeaways

- Cache misses cost ~100 CPU cycles each. Minimizing them is the #1 performance tool.
- Contiguous containers (`vector`, `array`) are fast because they exploit spatial locality.
- Array of Structs (AoS) vs Struct of Arrays (SoA): choose SoA when processing subsets of fields in tight loops.
- Linked lists are cache-unfriendly. Prefer `vector` unless you truly need O(1) mid-insertion.
- False sharing: keep data written by different threads on separate cache lines.

Chapter 40: Data-Oriented Design

The Problem with OOP for Performance

Object-oriented design groups data and behavior by object type:

```
class Enemy {
    Vector3 position;
    Vector3 velocity;
    float   health;
    float   damage;
    AI*     ai_controller;
    Mesh*   mesh;
    Sound*  sound_emitter;
    // 10 more fields...
};
std::vector<Enemy*> enemies;  // vector of pointers — heap fragmentation + cache misses
```

For 10,000 enemies, the game loop updates position (reads position and velocity), then renders (reads position and mesh). Each enemy's data is scattered in a large struct across random heap addresses.

Data-Oriented Design (DOD)

DOD organizes data around *transformation patterns* — how the data is actually accessed and processed:

```
// Separate the data accessed together from data accessed separately
struct EnemySystem {
    // Accessed together in the movement update:
    std::vector<Vector3> positions;
    std::vector<Vector3> velocities;

    // Accessed separately in the AI update:
    std::vector<float>   health;
    std::vector<float>   damage;

    // Accessed separately in the render:
    std::vector<MeshID>  mesh_ids;
    std::vector<SoundID> sound_ids;

    void update_movement() {
        for (int i = 0; i < positions.size(); ++i) {
            positions[i] += velocities[i];  // touches only positions and velocities
        }
        // Only 2 * n * sizeof(Vector3) loaded from memory — nothing else
    }
};
```

Entity Component System (ECS)

ECS is a common DOD architecture in game engines (Unity, Entt, Bevy):

```
// Each entity is just an ID
using EntityID = uint32_t;

// Components are plain data
struct Position  { float x, y, z; };
struct Velocity  { float x, y, z; };
struct Health    { float hp, max_hp; };

// Systems operate on component arrays
class MovementSystem {
    std::vector<Position>& positions;
    std::vector<Velocity>& velocities;
public:
    void update(float dt) {
        for (int i = 0; i < positions.size(); ++i) {
            positions[i].x += velocities[i].x * dt;
            positions[i].y += velocities[i].y * dt;
        }
    }
};
```

We cover ECS in detail in Chapter 50.

Hot/Cold Data Splitting

Separate frequently accessed "hot" data from rarely accessed "cold" data:

```
// BAD: every access to 'name' loads everything into cache
struct Player {
    // Hot (accessed every frame):
    Vector3 position;  // 12 bytes
    float   health;    // 4 bytes

    // Cold (accessed rarely):
    std::string name;               // 32+ bytes
    std::vector<Item> inventory;    // 24+ bytes
    AchievementBitset achievements; // 128 bytes
};

// GOOD: split hot and cold
struct PlayerHot {
    Vector3 position;
    float   health;
};
struct PlayerCold {
    std::string name;
    std::vector<Item> inventory;
    AchievementBitset achievements;
};
std::vector<PlayerHot>  hot;   // tight loop touches only this
std::vector<PlayerCold> cold;  // only accessed when needed
```

Key Takeaways

- Traditional OOP groups data by entity. DOD groups data by access pattern.
- Organize data so that a tight loop accesses contiguous, minimal data.
- Struct of Arrays is DOD's signature pattern — separate fields processed independently.
- ECS is DOD applied to game engines: entities are IDs, components are pure data, systems process component arrays.
- Hot/cold splitting: keep frequently accessed fields tight, put rarely accessed data elsewhere.

Chapter 41: Profiling and Flamegraphs

Don't Guess — Measure

The cardinal rule of optimization: *never optimize without profiling first.* Intuition about where time is spent is almost always wrong. Modern hardware is complex; the bottleneck is rarely where you think.

```
Profile → Find the bottleneck → Optimize → Measure → Repeat
```

Timing Code

The simplest profiling: time a section of code:

```cpp
#include <chrono>

auto start = std::chrono::high_resolution_clock::now();

// ... code to measure ...

auto end = std::chrono::high_resolution_clock::now();
auto ms = std::chrono::duration_cast<std::chrono::microseconds>(end - start);
std::cout << "Elapsed: " << ms.count() << " µs\n";
```

Profilers

A profiler samples the call stack at regular intervals, building a statistical picture of where time is spent.

Linux/Mac: - `perf` — Linux kernel profiler. Samples at hardware performance counters. - `gprof` — older, compile with `-pg`. - `Instruments` — macOS GUI profiler (Time Profiler). - `Valgrind/Callgrind` — instruction-level simulation, very accurate but very slow.

Windows: - Visual Studio profiler - Intel VTune

Cross-platform: - `Tracy` — real-time profiler popular in game development. - `Google Benchmark` — microbenchmarking library.

Flamegraphs

A flamegraph visualizes the call stack over time. The x-axis is time (widths proportional to CPU time spent), the y-axis is stack depth.

```
# With perf on Linux:
g++ -O2 -g -o my_program main.cpp   # -g preserves debug symbols
perf record ./my_program
perf script | ./FlameGraph/stackcollapse-perf.pl | ./FlameGraph/flamegraph.pl > flame.svg
```

Reading a flamegraph: find wide towers — they represent functions spending a lot of CPU time. The wider the bar, the more time in that function. The child bars are what that function calls.

Compile with Optimizations When Profiling

Always profile with optimizations enabled (`-O2` or `-O3`). Debug builds (`-O0`) have no inlining, redundant stores, and loads that will never appear in release — profiling them gives wrong information.

But keep debug symbols (`-g`) so the profiler can show function names.

`perf stat` — Hardware Counters

```
perf stat ./my_program
```

```
# Outputs:
# cycles:             2,345,678,901
# instructions:       5,678,901,234   # instructions per cycle (IPC) = 2.42
# cache-misses:         234,567       # cache misses
# branch-mispredictions:  12,345
```

Instructions per cycle (IPC): modern CPUs can execute 3-4 instructions per cycle. Low IPC means the CPU is stalling (cache misses, branch mispredictions, dependencies).

Cache miss rate: if a significant fraction of memory accesses miss the cache, data layout is the problem (Chapter 39-40).

Google Benchmark

For microbenchmarks — measuring a specific function in isolation:

```
#include <benchmark/benchmark.h>

static void BM_VectorSum(benchmark::State& state) {
    std::vector<int> v(1000);
    std::iota(v.begin(), v.end(), 0);

    for (auto _ : state) {
        int sum = 0;
        for (int x : v) sum += x;
        benchmark::DoNotOptimize(sum);  // prevent optimization away
    }
}
BENCHMARK(BM_VectorSum);

BENCHMARK_MAIN();
```

```
g++ -O2 -o bench bench.cpp -lbenchmark
./bench
# BM_VectorSum    234 ns
```

Key Takeaways

- Always profile before optimizing. You will be wrong about where the bottleneck is.
- Profile with optimizations enabled (`-O2`) and debug symbols (`-g`).
- Flamegraphs show the statistical call-stack profile — find the widest bars.
- `perf stat` shows hardware counters: IPC, cache miss rate, branch mispredictions.
- Google Benchmark is the standard for reproducible microbenchmarks.

Part IX — Concurrency

Chapter 42: Threads and `std::jthread`

Python's GIL vs C++ Threads

Python threads are real OS threads, but the Global Interpreter Lock (GIL) prevents more than one thread from executing Python bytecode simultaneously. Python threads excel at I/O concurrency (waiting for network, disk) but can't parallelize CPU-bound Python code.

C++ has no GIL. Multiple threads can execute simultaneously on multiple CPU cores. This enables real parallelism but also real data races.

Creating Threads

```cpp
#include <thread>
#include <iostream>

void worker(int id) {
    std::cout << "Thread " << id << " running\n";
}

int main() {
    std::thread t1(worker, 1);  // create thread, run worker(1)
    std::thread t2(worker, 2);  // create thread, run worker(2)

    t1.join();  // wait for t1 to finish
    t2.join();  // wait for t2 to finish
}
```

`std::thread` launches a new OS thread immediately. `join()` blocks until the thread finishes. If a `thread` is destroyed without joining or detaching, the program terminates (std::terminate is called).

`std::jthread` (C++20) — RAII Thread

`std::jthread` ("joining thread") auto-joins in its destructor — no need to manually call `join()`:

```cpp
#include <thread>

int main() {
    std::jthread t1(worker, 1);
    std::jthread t2(worker, 2);
    // t1 and t2 join automatically when they go out of scope
}
```

It also supports cooperative cancellation via `std::stop_token`:

```cpp
void long_task(std::stop_token stop) {
    while (!stop.stop_requested()) {
        do_work();
    }
}
```

```cpp
std::jthread t(long_task);
// later:
t.request_stop();   // signals the thread to stop
// t.join() called automatically on destruction
```

Passing Data to Threads

Use lambdas to capture data:

```cpp
std::vector<int> data(1000);
int result = 0;

std::jthread t([&data, &result]() {
    for (int x : data) result += x;
});
// DANGER: data and result must outlive the thread!
```

Parallel Algorithms (C++17)

The easiest way to parallelize standard algorithms:

```cpp
#include <algorithm>
#include <execution>

std::vector<int> v(10'000'000);

// Sequential:
std::sort(v.begin(), v.end());

// Parallel:
std::sort(std::execution::par, v.begin(), v.end());

// Parallel + unsequenced (SIMD):
std::sort(std::execution::par_unseq, v.begin(), v.end());
```

Parallel execution policies work with most STL algorithms. The implementation automatically uses a thread pool. This is the cleanest entry point for data parallelism.

Key Takeaways

- C++ threads are real threads with true parallelism — no GIL.
- `std::jthread` (C++20) joins automatically on destruction — prefer it over `std::thread`.
- Cooperative cancellation via `std::stop_token`.
- C++17 parallel execution policies (`std::execution::par`) parallelize STL algorithms with minimal code changes.

Chapter 43: Mutexes, Locks, and Race Conditions

What Is a Race Condition?

A race condition occurs when two threads access shared data concurrently and at least one access is a write, with no synchronization.

```
int counter = 0;

void increment() {
    for (int i = 0; i < 1'000'000; ++i)
        ++counter;   // RACE CONDITION — not atomic!
}

std::jthread t1(increment);
std::jthread t2(increment);
// Expected: 2,000,000. Actual: somewhere between 1,000,000 and 2,000,000.
```

`++counter` compiles to three instructions: load, add, store. Two threads can interleave these, losing increments.

`std::mutex`

A mutex ("mutual exclusion") ensures only one thread executes a critical section at a time:

```
#include <mutex>

int counter = 0;
std::mutex mtx;

void increment() {
    for (int i = 0; i < 1'000'000; ++i) {
        mtx.lock();
        ++counter;  // protected — only one thread here at a time
        mtx.unlock();
    }
}
```

But never call `lock()` / `unlock()` directly — exceptions between them leave the mutex locked forever.

`std::lock_guard` and `std::unique_lock`

RAII wrappers that unlock on destruction:

```
void increment() {
    for (int i = 0; i < 1'000'000; ++i) {
        std::lock_guard<std::mutex> lock(mtx);  // locks on construction
        ++counter;
    }   // lock_guard destructs here — mutex unlocked even if exception thrown
}

// unique_lock is more flexible (can unlock early, works with condition variables):
```

```cpp
void safe_update() {
    std::unique_lock<std::mutex> lock(mtx);
    // ... do protected work ...
    lock.unlock();  // manually unlock early if needed
    // ... do unprotected work ...
}
```

C++17: `std::scoped_lock` can lock multiple mutexes at once (deadlock-safe):

```cpp
std::scoped_lock lock(mtx1, mtx2);  // locks both, avoids deadlock
```

Deadlock

Deadlock occurs when two threads each hold a lock the other needs:

```cpp
// Thread 1:          // Thread 2:
mtx1.lock();          mtx2.lock();
mtx2.lock();  // wait mtx1.lock();  // wait → DEADLOCK
```

Rules to avoid deadlock: 1. Always lock multiple mutexes in the same order, OR 2. Use `std::scoped_lock` which handles this automatically. 3. Minimize the time locks are held.

Condition Variables

Condition variables let threads wait for a condition without polling:

```cpp
#include <condition_variable>

std::queue<int> queue;
std::mutex mtx;
std::condition_variable cv;

// Producer:
void producer() {
    for (int i = 0; i < 10; ++i) {
        {
            std::lock_guard lock(mtx);
            queue.push(i);
        }
        cv.notify_one();  // wake up one waiting consumer
    }
}

// Consumer:
void consumer() {
    while (true) {
        std::unique_lock lock(mtx);
        cv.wait(lock, []{ return !queue.empty(); });  // wait until queue non-empty
        int val = queue.front(); queue.pop();
        lock.unlock();
```

```
        process(val);
    }
}
```

Key Takeaways

- Race conditions occur when threads share data without synchronization — leads to undefined behavior.
- `std::mutex` protects critical sections. Always use RAII wrappers (`lock_guard`, `scoped_lock`).
- `std::scoped_lock` locks multiple mutexes safely — prevents deadlock from inconsistent ordering.
- `std::condition_variable` enables efficient wait-notify between threads.
- Keep critical sections as short as possible — lock only what's necessary for as little time as possible.

Chapter 44: Atomics and the C++ Memory Model

Lock-Free Programming

Mutexes are simple but have overhead: kernel calls, context switches, cache invalidation. For simple operations on single variables, *atomics* are faster — they use CPU hardware guarantees instead of OS-level locking.

```
#include <atomic>

std::atomic<int> counter = 0;

void increment() {
    for (int i = 0; i < 1'000'000; ++i)
        ++counter;   // atomic: guaranteed to be race-free, no mutex needed
}
```

`++counter` on an `std::atomic<int>` is a single atomic CPU instruction (like `LOCK XADD` on x86). No interleaving possible.

Atomic Operations

```
std::atomic<int> a = 0;

a.store(42);                // atomic write
int v = a.load();           // atomic read
int old = a.exchange(100);  // atomic swap: sets to 100, returns old value
a.fetch_add(5);             // atomic add, returns old value
a.fetch_sub(3);             // atomic subtract
++a; --a;                   // shorthand

// Compare-and-swap (CAS) — the foundation of lock-free algorithms:
int expected = 5;
```

```cpp
bool swapped = a.compare_exchange_strong(expected, 10);
// if a == expected (5), sets a = 10 and returns true
// if a != expected, sets expected = a's current value and returns false
```

Memory Ordering

This is the hardest part of C++ concurrency. The compiler and CPU are allowed to reorder instructions for performance, as long as the behavior of a single thread is unchanged. With multiple threads, reordering can break correctness.

`std::atomic` operations accept a memory order parameter:

```cpp
a.store(1, std::memory_order_relaxed);    // no ordering guarantee — just atomicity
a.store(1, std::memory_order_release);    // all writes before this are visible when a read with acquire
int v = a.load(std::memory_order_acquire); // all reads after this see writes from the paired release
a.store(1, std::memory_order_seq_cst);    // sequential consistency — default, safest, most expensive
```

The default (`seq_cst`) is safe for all cases but is the most expensive. For advanced optimization:

- `relaxed`: just atomicity, no ordering. Good for statistics counters.
- `release`/`acquire` pair: a write-then-read handoff — the release "publishes" data, the acquire "consumes" it. Used in lock-free queues.
- `seq_cst`: total global order across all atomic operations.

`std::atomic_flag` — The Simplest Atomic

`atomic_flag` is guaranteed lock-free and is the building block for spinlocks:

```cpp
std::atomic_flag flag = ATOMIC_FLAG_INIT;

// Spinlock using atomic_flag:
class Spinlock {
    std::atomic_flag flag = ATOMIC_FLAG_INIT;
public:
    void lock()   { while (flag.test_and_set(std::memory_order_acquire)); }
    void unlock() { flag.clear(std::memory_order_release); }
};
```

Use spinlocks only when lock hold time is very short (microseconds) and contention is rare. Otherwise, `std::mutex` (which yields the CPU when waiting) is better.

Key Takeaways

- Atomics provide lock-free, race-free operations on single values via hardware instructions.
- `atomic<T>` works for integral types and pointers. Operations: `store`, `load`, `exchange`, `fetch_add`, `compare_exchange_strong`.
- Memory ordering controls instruction reordering visibility across threads. Default (`seq_cst`) is safe; `relaxed` is fastest but only correct for specific patterns.
- `compare_exchange_strong` (CAS) is the foundation of lock-free algorithms.
- Use atomics for simple counters and flags; use mutexes for protecting compound operations on multiple variables.

Chapter 45: Async, Futures, and Tasks

The Problem with Raw Threads

Threads are low-level. For "run this function asynchronously and get the result," you'd need to set up shared state, mutexes, condition variables — boilerplate.

```python
# Python: simple async pattern
import concurrent.futures
with concurrent.futures.ThreadPoolExecutor() as ex:
    future = ex.submit(compute, args)
    result = future.result()
```

C++ has `std::async` and `std::future` for the same pattern.

`std::async` and `std::future`

```cpp
#include <future>

int compute(int x) {
    return x * x;
}

// Launch async task:
std::future<int> fut = std::async(std::launch::async, compute, 5);

// Do other work while compute() runs in another thread...

// Get the result (blocks if not yet done):
int result = fut.get();  // 25
```

`std::future<T>` represents a value that will be computed asynchronously. `.get()` blocks until the value is ready. It can only be called once.

Launch Policies

```cpp
// Always run in a new thread:
auto f1 = std::async(std::launch::async, fn, args...);

// Run lazily in the calling thread when .get() is called:
auto f2 = std::async(std::launch::deferred, fn, args...);

// Implementation decides (may or may not create a thread):
auto f3 = std::async(fn, args...);  // default — avoid in production code
```

`std::launch::async` is the reliable option — it always runs in a separate thread.

`std::promise` — Manual Future Control

`promise<T>` / `future<T>` is a producer/consumer pair:

```
std::promise<int> promise;
std::future<int>  fut = promise.get_future();

std::jthread producer([&promise]() {
    int result = expensive_computation();
    promise.set_value(result);    // signals the future
    // or: promise.set_exception(std::current_exception());
});

int result = fut.get();  // waits for producer to set the value
```

`std::shared_future`

`future` can only be retrieved once. `shared_future` can be copied and `.get()` called multiple times:

```
auto sf = fut.share();   // convert future to shared_future

std::jthread t1([sf]() { auto r = sf.get(); });  // both can wait
std::jthread t2([sf]() { auto r = sf.get(); });
```

Thread Pools

The standard library doesn't ship a thread pool (C++26 plans to add one). For now, use a library or write your own:

```
// Common pattern with std::async — tasks may reuse threads from an internal pool:
std::vector<std::future<int>> futures;
for (int i = 0; i < 100; ++i) {
    futures.push_back(std::async(std::launch::async, compute, i));
}
for (auto& f : futures) {
    std::cout << f.get() << "\n";
}
```

Libraries like Intel TBB, Taskflow, and Asio provide production-grade thread pools with work stealing.

Key Takeaways

- `std::async(std::launch::async, fn, args)` runs `fn` in a new thread, returns a `std::future<T>`.
- `future.get()` blocks until the result is ready.
- `std::promise<T>` + `std::future<T>` is a manual producer/consumer channel.
- `std::shared_future` allows multiple threads to wait on the same result.
- The standard has no thread pool yet (C++26); use `std::async` for simple cases, a library for production.

Part X — Specialization A: Graphics & Game Development

Chapter 46: The Math: vectors, matrices, quaternions

Why Math?

Graphics and game development require 3D spatial math. Before writing a line of OpenGL or Vulkan code, you need to understand the data structures that describe positions, orientations, and transformations in 3D space.

Vectors

A vector in 3D is a direction and magnitude — or a position (a point displaced from the origin).

```
struct Vec3 {
    float x, y, z;

    Vec3 operator+(const Vec3& o) const { return {x+o.x, y+o.y, z+o.z}; }
    Vec3 operator-(const Vec3& o) const { return {x-o.x, y-o.y, z-o.z}; }
    Vec3 operator*(float s)       const { return {x*s, y*s, z*s}; }

    float dot(const Vec3& o) const { return x*o.x + y*o.y + z*o.z; }

    Vec3 cross(const Vec3& o) const {
        return { y*o.z - z*o.y,
                 z*o.x - x*o.z,
                 x*o.y - y*o.x };
    }

    float length() const { return std::sqrt(dot(*this)); }
    Vec3  normalize() const { float l = length(); return {x/l, y/l, z/l}; }
};
```

Key operations: - **Dot product** `a · b = |a||b|cos(θ)`: measures alignment. `dot > 0` → same direction, `dot = 0` → perpendicular, `dot < 0` → opposite. - **Cross product** `a × b`: returns a vector perpendicular to both. Used for normals and "which way is left." - **Normalization**: make length = 1. Normalized direction vectors are needed for lighting.

Matrices

A 4×4 matrix represents a transformation in 3D (translation, rotation, scale, projection). Vertices are transformed by multiplying by matrices.

```
struct Mat4 {
    float m[4][4] = {};  // row-major
```

```
    static Mat4 identity() {
        Mat4 r;
        r.m[0][0] = r.m[1][1] = r.m[2][2] = r.m[3][3] = 1.0f;
        return r;
    }

    Mat4 operator*(const Mat4& o) const {
        Mat4 result;
        for (int r = 0; r < 4; ++r)
            for (int c = 0; c < 4; ++c)
                for (int k = 0; k < 4; ++k)
                    result.m[r][c] += m[r][k] * o.m[k][c];
        return result;
    }

    Vec3 transform_point(Vec3 p) const {
        float w = m[3][0]*p.x + m[3][1]*p.y + m[3][2]*p.z + m[3][3];
        return {
            (m[0][0]*p.x + m[0][1]*p.y + m[0][2]*p.z + m[0][3]) / w,
            (m[1][0]*p.x + m[1][1]*p.y + m[1][2]*p.z + m[1][3]) / w,
            (m[2][0]*p.x + m[2][1]*p.y + m[2][2]*p.z + m[2][3]) / w
        };
    }
};
```

The 4th component (w) enables translation in matrix form — this is *homogeneous coordinates*. Points have `w=1`, directions have `w=0`.

The transformation pipeline:

`Object Space → Model Matrix → World Space → View Matrix → Camera Space → Projection Matrix → Clip Space`

Quaternions

Quaternions represent rotations without gimbal lock (the problem where two rotation axes align, losing a degree of freedom). They're more compact than rotation matrices (4 floats vs 16) and easier to interpolate.

```
struct Quat {
    float w, x, y, z;  // w is the scalar part

    static Quat from_axis_angle(Vec3 axis, float angle_rad) {
        float half = angle_rad * 0.5f;
        float s = std::sin(half);
        return { std::cos(half), axis.x*s, axis.y*s, axis.z*s };
    }

    Quat operator*(const Quat& o) const {
        return {
```

```
            w*o.w - x*o.x - y*o.y - z*o.z,
            w*o.x + x*o.w + y*o.z - z*o.y,
            w*o.y - x*o.z + y*o.w + z*o.x,
            w*o.z + x*o.y - y*o.x + z*o.w
        };
    }

    // Spherical linear interpolation (smooth rotation between two orientations)
    static Quat slerp(Quat a, Quat b, float t);
};
```

Use a Library

In practice, use **GLM** (OpenGL Mathematics) — a header-only C++ math library that mirrors GLSL (the GPU shader language) syntax:

```
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/quaternion.hpp>

glm::vec3 position = {1.0f, 2.0f, 3.0f};
glm::mat4 model = glm::translate(glm::mat4(1.0f), position);
model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0, 1, 0));
model = glm::scale(model, glm::vec3(2.0f));

glm::quat rotation = glm::angleAxis(glm::radians(45.0f), glm::vec3(0, 1, 0));
glm::quat interp   = glm::slerp(q1, q2, 0.5f);
```

Key Takeaways

- Vectors: position/direction. Key ops: dot (alignment), cross (perpendicular), normalize.
- 4×4 matrices represent transformations. The pipeline is Model → View → Projection.
- Quaternions represent rotations without gimbal lock; use `slerp` for smooth interpolation.
- In practice, use GLM — it matches GLSL syntax and is highly optimized.

Chapter 47: How the GPU Works

CPU vs GPU Architecture

A CPU has a few (4-64) powerful cores, each with large caches, branch prediction, and out-of-order execution — optimized for sequential tasks and complex logic.

A GPU has thousands of small, simple cores — optimized for doing the *same operation on many data elements simultaneously* (SIMD at massive scale). A modern GPU has 10,000-80,000 shader cores.

```
CPU: 16 powerful cores
     Complex out-of-order execution
```

```
     4 MB L3 cache per core
     Excels at: sequential code, branching, complex logic

GPU: 80,000+ simple shader cores
     Simple in-order execution (many cores)
     Shared L2 cache
     Excels at: parallel data transformation, matrix multiply
```

The Graphics Pipeline

When you call a draw call (e.g., "draw this mesh"), the GPU runs it through a fixed pipeline:

```
1. Input Assembly: read vertices from a buffer
2. Vertex Shader: run once per vertex (positions, normals → clip space)
3. Rasterization: convert triangles to fragments (pixels)
4. Fragment Shader: run once per fragment (compute color, apply textures)
5. Output Merge: depth test, blend, write to framebuffer
```

Stages 2 and 4 are *programmable* — you write shader code (GLSL for OpenGL, SPIR-V for Vulkan) that the GPU executes in parallel on every vertex/fragment.

GPU Memory

```
VRAM (Video RAM): on the GPU card — fast but limited (8-24 GB typical)
  - Vertex buffers (VBO): mesh geometry
  - Index buffers (IBO): triangle index arrays
  - Textures: images
  - Uniform buffers (UBO): small per-draw-call data (matrices, colors)
  - Framebuffers: render targets (what you draw to)

System RAM: on the CPU — large but slow to transfer to GPU
```

The bottleneck is often the CPU→GPU transfer (PCIe bandwidth). Minimize uploads; keep data in VRAM.

Shaders

Shaders are programs that run on the GPU. Written in GLSL (OpenGL) or HLSL (DirectX):

```
// Vertex shader (GLSL)
#version 460

layout(location = 0) in vec3 position;
layout(location = 1) in vec2 texcoord;

uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

out vec2 frag_texcoord;
```

```
void main() {
    gl_Position = projection * view * model * vec4(position, 1.0);
    frag_texcoord = texcoord;
}
```

```
// Fragment shader (GLSL)
#version 460

in vec2 frag_texcoord;
out vec4 color;

uniform sampler2D texture_map;

void main() {
    color = texture(texture_map, frag_texcoord);
}
```

The vertex shader runs once per vertex (millions of times per frame). The fragment shader runs once per pixel covered (tens of millions of times per frame). Both run in parallel across all GPU cores.

Compute Shaders

Beyond graphics, the GPU is used for general computation (GPGPU). Compute shaders run arbitrary parallel workloads: physics simulation, ML inference, image processing.

```
// Compute shader: add two arrays
#version 460
layout(local_size_x = 256) in;

layout(std430, binding = 0) buffer A { float a[]; };
layout(std430, binding = 1) buffer B { float b[]; };
layout(std430, binding = 2) buffer C { float c[]; };

void main() {
    uint idx = gl_GlobalInvocationID.x;
    c[idx] = a[idx] + b[idx];
}
// dispatch with 10,000,000 / 256 workgroups → processes 10M floats in parallel
```

Key Takeaways

- GPUs have thousands of simple cores for massively parallel workloads.
- The graphics pipeline: vertex shader → rasterization → fragment shader.
- You control vertex and fragment shaders in GLSL/HLSL.
- Data lives in VRAM (fast) or RAM (needs upload). Minimize CPU→GPU transfers.
- Compute shaders run arbitrary GPU programs beyond rendering.

Chapter 48: OpenGL Fundamentals

What Is OpenGL?

OpenGL is a cross-platform graphics API for drawing 2D/3D graphics by communicating with the GPU. It's the oldest and most widely documented real-time graphics API. Vulkan (Chapter 49) is its modern successor.

Setup: GLFW + GLAD

- **GLFW**: creates a window and an OpenGL context.
- **GLAD**: loads OpenGL function pointers (OpenGL functions aren't in a static library — they're loaded from the driver at runtime).

```
#include <glad/glad.h>
#include <GLFW/glfw3.h>

int main() {
    glfwInit();
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 4);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 6);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

    GLFWwindow* window = glfwCreateWindow(800, 600, "OpenGL", nullptr, nullptr);
    glfwMakeContextCurrent(window);
    gladLoadGLLoader((GLADloadproc)glfwGetProcAddress);

    while (!glfwWindowShouldClose(window)) {
        glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

        // Draw calls here

        glfwSwapBuffers(window);
        glfwPollEvents();
    }
    glfwTerminate();
}
```

The Core Workflow

1. **Create vertex data** on the CPU.
2. **Upload** to a *Vertex Buffer Object* (VBO) on the GPU.
3. **Describe the layout** with a *Vertex Array Object* (VAO).
4. **Compile shaders** and link into a *shader program*.
5. **Draw** with `glDrawArrays` or `glDrawElements`.

```
// Triangle vertices: position (x,y,z) + color (r,g,b)
float vertices[] = {
    -0.5f, -0.5f, 0.0f,  1.0f, 0.0f, 0.0f,  // bottom-left (red)
```

```
     0.5f, -0.5f, 0.0f,  0.0f, 1.0f, 0.0f,  // bottom-right (green)
     0.0f,  0.5f, 0.0f,  0.0f, 0.0f, 1.0f,  // top (blue)
};

// Create and bind VAO + VBO
unsigned int VAO, VBO;
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);

glBindVertexArray(VAO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

// Describe attribute 0: position (3 floats, starts at offset 0)
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6*sizeof(float), (void*)0);
glEnableVertexAttribArray(0);

// Describe attribute 1: color (3 floats, starts at offset 12 bytes)
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6*sizeof(float), (void*)(3*sizeof(float)));
glEnableVertexAttribArray(1);

// Draw:
glUseProgram(shader_program);
glBindVertexArray(VAO);
glDrawArrays(GL_TRIANGLES, 0, 3);
```

Compiling Shaders at Runtime

```
const char* vert_src = R"glsl(
    #version 460 core
    layout(location=0) in vec3 pos;
    layout(location=1) in vec3 color;
    out vec3 fragColor;
    void main() {
        gl_Position = vec4(pos, 1.0);
        fragColor = color;
    }
)glsl";

unsigned int vert = glCreateShader(GL_VERTEX_SHADER);
glShaderSource(vert, 1, &vert_src, nullptr);
glCompileShader(vert);

// Check for errors:
int success;
glGetShaderiv(vert, GL_COMPILE_STATUS, &success);
if (!success) {
```

```
    char log[512];
    glGetShaderInfoLog(vert, 512, nullptr, log);
    std::cerr << "Shader error: " << log << "\n";
}

unsigned int program = glCreateProgram();
glAttachShader(program, vert);
glAttachShader(program, frag);
glLinkProgram(program);
glDeleteShader(vert);
glDeleteShader(frag);
```

Textures

```
// Load image with stb_image (single-header library):
#define STB_IMAGE_IMPLEMENTATION
#include "stb_image.h"

int width, height, channels;
unsigned char* data = stbi_load("texture.png", &width, &height, &channels, 0);

unsigned int texture;
glGenTextures(1, &texture);
glBindTexture(GL_TEXTURE_2D, texture);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB, GL_UNSIGNED_BYTE, data);
glGenerateMipmap(GL_TEXTURE_2D);
stbi_image_free(data);
```

Key Takeaways

- OpenGL is a state machine. Bind objects (VAO, VBO, textures, shader programs) then draw.
- VAO describes vertex layout. VBO holds vertex data. Bind both before drawing.
- Shaders are compiled at runtime from GLSL source strings.
- Modern OpenGL (4.x Core Profile) uses vertex arrays — no legacy `glBegin`/`glEnd`.
- Use GLFW for windowing and GLAD for loading function pointers.

Chapter 49: Moving to Vulkan

Why Vulkan?

OpenGL hides complexity behind a global state machine managed by the driver. The driver guesses your intentions, validates inputs, and compiles shaders behind the scenes — this causes unpredictable hitches and overhead.

Vulkan is explicit: you manage GPU memory, command buffers, synchronization, and pipeline states yourself. In exchange, you get:

- **Predictable performance** — no driver magic.
- **Lower CPU overhead** — multithreaded command recording.
- **Explicit control** — know exactly what the GPU is doing.
- **Portability** — runs on Windows, Linux, macOS (via MoltenVK), Android.

The Verbosity Trade-Off

Vulkan's "hello triangle" is ~1000 lines. OpenGL's is ~100. Every object you implicitly got from OpenGL must be explicitly created in Vulkan. This is the trade-off: control vs. boilerplate.

For this reason, most Vulkan programs use: - **vk-bootstrap**: simplifies instance, device, and swapchain creation. - **VMA (Vulkan Memory Allocator)**: manages GPU memory allocation. - Custom wrappers / engines that abstract the boilerplate.

Core Vulkan Concepts

```
VkInstance          – the Vulkan runtime
VkPhysicalDevice    – the GPU hardware
VkDevice            – logical device (interface to the GPU)
VkQueue             – command submission queue (graphics, compute, transfer)
VkSwapchain         – sequence of images presented to the screen
VkRenderPass        – describes the structure of rendering (attachments)
VkFramebuffer       – links images to a render pass
VkPipeline          – compiled graphics state (shaders + fixed function)
VkCommandBuffer     – recorded list of GPU commands
VkDescriptorSet     – table of bindings (textures, UBOs) used by shaders
VkSemaphore         – GPU-GPU synchronization
VkFence             – GPU-CPU synchronization
```

The Render Loop

```c
// Acquire image from swapchain:
uint32_t imageIndex;
vkAcquireNextImageKHR(device, swapchain, UINT64_MAX, imageAvailableSemaphore,
                      VK_NULL_HANDLE, &imageIndex);

// Record commands:
vkBeginCommandBuffer(commandBuffer, &beginInfo);
    vkCmdBeginRenderPass(commandBuffer, &renderPassInfo, VK_SUBPASS_CONTENTS_INLINE);
    vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, graphicsPipeline);
    vkCmdBindVertexBuffers(commandBuffer, 0, 1, &vertexBuffer, offsets);
    vkCmdDraw(commandBuffer, vertexCount, 1, 0, 0);
    vkCmdEndRenderPass(commandBuffer);
vkEndCommandBuffer(commandBuffer);

// Submit to GPU:
vkQueueSubmit(graphicsQueue, 1, &submitInfo, fence);
```

```
// Present to screen:
vkQueuePresentKHR(presentQueue, &presentInfo);
```

Shaders in Vulkan: SPIR-V

Vulkan doesn't accept GLSL directly — shaders must be compiled to SPIR-V (a binary IR):

```
glslc shader.vert -o shader.vert.spv
glslc shader.frag -o shader.frag.spv
```

The GLSL syntax is mostly the same as OpenGL. SPIR-V is loaded at runtime:

```
std::vector<char> code = read_file("shader.vert.spv");
VkShaderModuleCreateInfo createInfo{};
createInfo.codeSize = code.size();
createInfo.pCode = reinterpret_cast<const uint32_t*>(code.data());
VkShaderModule shaderModule;
vkCreateShaderModule(device, &createInfo, nullptr, &shaderModule);
```

Recommended Learning Path

1. Read **"Vulkan Tutorial"** (vulkan-tutorial.com) — the canonical free resource.
2. Use **vk-bootstrap** and **VMA** to reduce boilerplate.
3. Study **Sascha Willems' Vulkan samples** for specific techniques.
4. For a game engine, look at **vkguide.dev** (a game engine focused Vulkan guide).

Key Takeaways

- Vulkan is explicit where OpenGL is implicit — you manage memory, command buffers, synchronization.
- More boilerplate, but more predictable performance and lower CPU overhead.
- Use vk-bootstrap + VMA to reduce setup boilerplate in new projects.
- Shaders compile to SPIR-V (via glslc). The GLSL language is almost identical to OpenGL.
- Learn via vulkan-tutorial.com — it's the community standard.

Chapter 50: Game Loop, ECS Architecture, and Engine Design

The Game Loop

Every game has a main loop that processes input, updates state, and renders — as fast as possible (or at a fixed rate):

```
float last_time = glfwGetTime();

while (!glfwWindowShouldClose(window)) {
    float now   = glfwGetTime();
    float delta = now - last_time;    // time since last frame (seconds)
    last_time   = now;
```

```
    glfwPollEvents();                   // process OS events (input)

    update(delta);                      // update game state
    render();                           // draw the frame

    glfwSwapBuffers(window);            // present to screen
}
```

`delta` (delta time) is critical: game logic should multiply velocities by delta so the game runs at the same *speed* regardless of framerate:

```
// Bad: ties movement speed to framerate
position.x += 5.0f;  // faster on 120fps than 60fps

// Good: frame-rate independent
position.x += 5.0f * delta;  // always moves 5 units/second
```

Fixed Timestep

Physics simulation is unstable with variable delta. Use a fixed timestep with a variable render rate:

```
const float FIXED_DT = 1.0f / 60.0f;  // 60 physics steps per second
float accumulator = 0.0f;

while (running) {
    float frame_time = measure_frame_time();
    accumulator += frame_time;

    while (accumulator >= FIXED_DT) {
        physics_update(FIXED_DT);     // always runs with constant dt
        accumulator -= FIXED_DT;
    }

    float alpha = accumulator / FIXED_DT;  // fractional step for interpolation
    render(alpha);                          // render between steps
}
```

Entity Component System (ECS)

OOP game engines model game objects as class hierarchies:

```
class GameObject;
class Enemy : public GameObject;
class FlyingEnemy : public Enemy;  // deep hierarchies become painful
```

ECS separates concerns: - **Entity**: just an ID (uint32_t). - **Component**: plain data struct, no behavior. - **System**: stateless functions that operate on components.

```
// Components — plain structs
struct Transform { glm::vec3 pos, scale; glm::quat rot; };
```

```cpp
struct Velocity   { glm::vec3 linear, angular; };
struct Health     { float hp, max_hp; };
struct Renderable { MeshID mesh; MaterialID material; };

// Entity is just an ID
using Entity = uint32_t;

// ECS Registry (entt library example)
#include <entt/entt.hpp>

entt::registry registry;

// Create entities:
Entity player = registry.create();
registry.emplace<Transform>(player, glm::vec3{0,0,0}, glm::vec3{1,1,1}, glm::quat{});
registry.emplace<Health>(player, 100.0f, 100.0f);

// Systems operate on all entities with certain components:
void movement_system(entt::registry& reg, float dt) {
    auto view = reg.view<Transform, Velocity>();
    for (auto [entity, xform, vel] : view.each()) {
        xform.pos += vel.linear * dt;
    }
}

void health_system(entt::registry& reg) {
    auto view = reg.view<Health>();
    for (auto [entity, hp] : view.each()) {
        if (hp.hp <= 0) reg.destroy(entity);
    }
}
```

Why ECS?

1. **Performance**: components in contiguous arrays — cache-friendly iteration.
2. **Flexibility**: add/remove components at runtime — no class hierarchy changes.
3. **Composition**: mix and match behaviors by combining components.
4. **Testability**: systems are pure functions — easy to unit test.

entt is the most popular C++ ECS library — header-only, extremely fast.

Engine Architecture Overview

```
Engine
├── Core
│   ├── Window + Input (GLFW)
│   ├── Event system
│   └── Main loop
```

```
├── ECS (entt)
│   ├── Entity management
│   ├── Component storage
│   └── System scheduler
├── Renderer (OpenGL / Vulkan)
│   ├── Mesh loading (assimp)
│   ├── Shader management
│   ├── Material system
│   └── Render passes
├── Physics
│   └── (PhysX, Bullet, Jolt)
├── Audio
│   └── (OpenAL, FMOD)
└── Asset System
    ├── Texture loading (stb_image)
    └── Scene serialization
```

Key Takeaways

- The game loop: poll input → update → render, as fast as possible.
- Use delta time for frame-rate-independent movement.
- Use a fixed timestep for physics to ensure stability.
- ECS: Entity (ID) + Component (data) + System (behavior). Cache-friendly and flexible.
- **entt** is the go-to C++ ECS library.

Part XI — Specialization B: Systems Programming

Chapter 51: The Machine: registers, memory, syscalls

What Your C++ Code Becomes

C++ is translated to machine code — instructions for the CPU. Understanding the machine helps you write better C++ and debug at the assembly level.

```cpp
int add(int a, int b) {
    return a + b;
}
```

Compiles to (x86-64 with -O2):

```
add(int, int):
    lea eax, [rdi + rsi]   ; eax = first_arg + second_arg
    ret                     ; return eax
```

Just two instructions. The arguments arrive in registers `rdi` and `rsi` (System V AMD64 ABI). The return value goes in `rax`.

Registers

x86-64 has 16 general-purpose 64-bit registers:

```
rax — return value, caller-saved
rbx — callee-saved
rcx — 4th argument, caller-saved
rdx — 3rd argument, caller-saved
rsi — 2nd argument, caller-saved
rdi — 1st argument, caller-saved
rbp — base pointer (stack frame)
rsp — stack pointer
r8  — 5th argument
r9  — 6th argument
r10-r11 — caller-saved scratch
r12-r15 — callee-saved
```

"Caller-saved" means the calling function must save them before calling if it needs them. "Callee-saved" means the called function must restore them before returning.

Reading assembly in disassemblers (like `objdump -d`) helps diagnose generated code quality and confirm that hot paths are optimized.

Looking at Assembly

```
g++ -O2 -S -o output.s myfile.cpp   # generates assembly source
objdump -d -C ./my_binary | head -50  # disassemble binary
```

Or use **Compiler Explorer** (godbolt.org) — paste C++ and see the assembly live.

System Calls

A system call is a request from your program to the OS kernel. File I/O, network sockets, memory mapping — all go through syscalls.

```
// C++ standard library call:
std::ofstream file("data.txt");
file << "hello\n";

// Underneath, this calls:
// open("data.txt", O_CREAT|O_WRONLY|O_TRUNC, 0666)  → file descriptor
// write(fd, "hello\n", 6)
// close(fd)

// Which are wrappers around Linux syscalls:
// syscall(SYS_openat, AT_FDCWD, "data.txt", ...)
// syscall(SYS_write, fd, "hello\n", 6)
// syscall(SYS_close, fd)
```

The standard library wraps syscalls. For systems programming you sometimes call them directly via `<unistd.h>`.

Virtual Memory

Every process sees a private, contiguous virtual address space (typically 48 bits on x86-64 — 256 TB). The OS maps this to physical RAM pages (4 KB each). This provides isolation (processes can't see each other's memory) and abstraction (you don't know which physical pages you're using).

```
Virtual address space of a process:
0x0000000000000000 - 0x00007FFFFFFFFFFF  (user space, 128 TB)
  ↓ Code (.text): the compiled instructions
  ↓ Data (.data, .bss): global variables
  ↓ Heap: grows upward (new/malloc)
  ↑ Stack: grows downward (local variables)
0xFFFF800000000000 - 0xFFFFFFFFFFFFFFFF  (kernel space — mapped but inaccessible)
```

Key Takeaways

- C++ function arguments are passed in registers (`rdi`, `rsi`, …) on Linux x86-64.
- Look at generated assembly with `g++ -S` or godbolt.org to understand what your code costs.
- System calls are the boundary between user space and the OS kernel.
- Virtual memory gives each process a private address space, mapped to physical pages by the OS.

Chapter 52: Working with the OS and Linux APIs

POSIX — The Portable Interface

Linux follows POSIX (Portable Operating System Interface), a standard API for OS services. POSIX functions live in `<unistd.h>`, `<fcntl.h>`, `<sys/types.h>`, etc.

File I/O at the System Level

```cpp
#include <fcntl.h>
#include <unistd.h>
#include <cstring>

// Open file:
int fd = open("data.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
if (fd == -1) {
    perror("open");  // prints error message + errno description
    return -1;
}

// Write:
const char* msg = "Hello, world!\n";
```

```cpp
ssize_t written = write(fd, msg, strlen(msg));

// Read:
char buf[256];
ssize_t bytes_read = read(fd, buf, sizeof(buf));

// Seek:
lseek(fd, 0, SEEK_SET);  // back to beginning

// Close:
close(fd);
```

For most purposes, use C++ streams or C `FILE*`. Use raw file descriptors when you need: - `sendfile` (zero-copy file send) - `mmap` (memory-mapped files) - Non-blocking I/O - `io_uring` (modern Linux async I/O)

Memory-Mapped Files

`mmap` maps a file directly into virtual memory. Reading/writing the memory IS reading/writing the file — no explicit read/write calls:

```cpp
#include <sys/mman.h>
#include <sys/stat.h>

int fd = open("large_file.bin", O_RDONLY);
struct stat st;
fstat(fd, &st);

void* data = mmap(nullptr, st.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
close(fd);  // can close fd after mmap

// Access the file as if it were an array in memory:
uint32_t* ints = static_cast<uint32_t*>(data);
for (size_t i = 0; i < st.st_size / 4; ++i) {
    process(ints[i]);  // OS pages in file data on demand
}

munmap(data, st.st_size);  // unmap when done
```

`mmap` is extremely fast for read-heavy workloads — no kernel/user copy. The OS handles caching and reads pages on demand (demand paging).

Signals

Signals are asynchronous notifications to a process (like Python's `signal` module):

```cpp
#include <csignal>

volatile std::atomic<bool> running = true;
```

```cpp
void handle_sigint(int signal) {
    running = false;  // set flag, exit main loop cleanly
}

int main() {
    std::signal(SIGINT, handle_sigint);  // handle Ctrl+C

    while (running) {
        do_work();
    }
    cleanup();
}
```

Key signals: `SIGINT` (Ctrl+C), `SIGTERM` (kill command), `SIGSEGV` (segfault), `SIGKILL` (uncatchable kill).

Processes: `fork` and `exec`

```cpp
#include <unistd.h>
#include <sys/wait.h>

pid_t pid = fork();   // creates a copy of the current process
if (pid == 0) {
    // Child process:
    execl("/bin/ls", "ls", "-la", nullptr);  // replace child with 'ls'
    perror("execl");  // only reached if execl fails
    _exit(1);
} else if (pid > 0) {
    // Parent process:
    int status;
    waitpid(pid, &status, 0);  // wait for child to finish
    std::cout << "Child exited with " << WEXITSTATUS(status) << "\n";
} else {
    perror("fork");
}
```

Python's `subprocess` module wraps fork/exec. In C++, you call it directly.

Key Takeaways

- POSIX provides direct OS APIs: `open`/`read`/`write`/`close` for files, `fork`/`exec` for processes, `mmap` for memory-mapped I/O.
- `mmap` is the fastest way to read large files — zero copy, OS-managed paging.
- Signals are async notifications. Use `volatile atomic<bool>` flags as signal-safe state.
- `fork` creates a process copy. `exec` replaces a process image. Together they launch child programs.

Chapter 53: Networking from the Ground Up

Sockets

Network programming on Unix uses *sockets* — file descriptors connected to network endpoints. The socket API is the foundation of all networking, including what Python's `socket` module wraps.

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>

// TCP Client connecting to a server:
int sock = socket(AF_INET, SOCK_STREAM, 0);  // IPv4, TCP, default protocol

sockaddr_in server_addr{};
server_addr.sin_family = AF_INET;
server_addr.sin_port   = htons(8080);         // host-to-network byte order
inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);

connect(sock, (sockaddr*)&server_addr, sizeof(server_addr));

const char* msg = "GET / HTTP/1.0\r\n\r\n";
send(sock, msg, strlen(msg), 0);

char buf[4096];
ssize_t bytes = recv(sock, buf, sizeof(buf)-1, 0);
buf[bytes] = '\0';
std::cout << buf;

close(sock);
```

TCP Server

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);

// Allow address reuse (avoid "address already in use" on restart):
int opt = 1;
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

sockaddr_in addr{};
addr.sin_family      = AF_INET;
addr.sin_addr.s_addr = INADDR_ANY;  // accept on all interfaces
addr.sin_port        = htons(8080);

bind(server_fd, (sockaddr*)&addr, sizeof(addr));
listen(server_fd, 10);  // backlog of 10 pending connections
```

```cpp
while (true) {
    sockaddr_in client_addr{};
    socklen_t len = sizeof(client_addr);
    int client_fd = accept(server_fd, (sockaddr*)&client_addr, &len);

    // Handle client (in a thread for concurrency):
    std::jthread t([client_fd]() {
        char buf[1024];
        ssize_t n = recv(client_fd, buf, sizeof(buf), 0);
        send(client_fd, buf, n, 0);  // echo back
        close(client_fd);
    });
}
```

Non-Blocking I/O and `select`/`epoll`

One-thread-per-connection doesn't scale. `select`/`poll`/`epoll` let one thread monitor many connections:

```cpp
// epoll — the Linux-specific, high-performance variant:
#include <sys/epoll.h>

int epfd = epoll_create1(0);

epoll_event ev{};
ev.events  = EPOLLIN;  // notify when data available to read
ev.data.fd = server_fd;
epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);

epoll_event events[64];
while (true) {
    int n = epoll_wait(epfd, events, 64, -1);  // blocks until events ready
    for (int i = 0; i < n; ++i) {
        if (events[i].data.fd == server_fd) {
            // New connection
        } else {
            // Data available on events[i].data.fd
        }
    }
}
```

`epoll` scales to millions of connections — it's what nginx, Redis, and most modern servers use under the hood.

Modern Approach: Asio

The Asio library (standalone or via Boost) provides a high-level async networking API:

```cpp
#include <asio.hpp>

asio::io_context io;
asio::ip::tcp::acceptor acceptor(io, {asio::ip::tcp::v4(), 8080});

auto handle_client = [](asio::ip::tcp::socket socket) -> asio::awaitable<void> {
    char buf[1024];
    auto n = co_await socket.async_read_some(asio::buffer(buf), asio::use_awaitable);
    co_await async_write(socket, asio::buffer(buf, n), asio::use_awaitable);
};

auto listener = [&]() -> asio::awaitable<void> {
    while (true) {
        auto socket = co_await acceptor.async_accept(asio::use_awaitable);
        asio::co_spawn(io, handle_client(std::move(socket)), asio::detached);
    }
};

asio::co_spawn(io, listener(), asio::detached);
io.run();
```

Asio + C++20 coroutines gives async networking with Python asyncio-style clarity.

Key Takeaways

- Sockets are file descriptors for network connections. `socket` → `bind`/`connect` → `send`/`recv` → `close`.
- TCP: use `SOCK_STREAM`. UDP: use `SOCK_DGRAM`.
- `epoll` enables one thread to handle thousands of connections efficiently.
- Use Asio (with C++20 coroutines) for production async networking — cleaner than raw epoll.

Chapter 54: Where C++ Meets C, eBPF, and Go

Interoperability with C

C++ is backward-compatible with C. Every C function is callable from C++. But C++ has name mangling (function names get decorated with type information for overload resolution), so C libraries can't call C++ functions directly without a wrapper.

```cpp
// Calling a C library from C++ — just include the header:
#include <stdlib.h>   // C headers — just works
#include <string.h>

void* buf = malloc(1024);  // C function — callable from C++
free(buf);
```

When writing a C++ library that C code must call, export with `extern "C"`:

```c
// mylib.h
#ifdef __cplusplus
extern "C" {  // prevent C++ name mangling
#endif

void init_library(void);
int  compute(int x, int y);
void shutdown_library(void);

#ifdef __cplusplus
}
#endif
```

```c
// mylib.cpp
#include "mylib.h"

extern "C" {
    void init_library()        { /* ... */ }
    int  compute(int x, int y) { return x + y; }
    void shutdown_library()    { /* ... */ }
}
```

Now C code can `#include "mylib.h"` and call these functions. Python's `ctypes` and `cffi` can too.

C++ → Python: pybind11

pybind11 wraps C++ classes and functions for Python with minimal boilerplate:

```cpp
#include <pybind11/pybind11.h>
namespace py = pybind11;

int add(int a, int b) { return a + b; }

struct Dog {
    std::string name;
    Dog(std::string n) : name(std::move(n)) {}
    std::string bark() const { return name + " says: Woof!"; }
};

PYBIND11_MODULE(mymodule, m) {
    m.def("add", &add, "Add two integers");

    py::class_<Dog>(m, "Dog")
        .def(py::init<std::string>())
        .def("bark", &Dog::bark)
        .def_readwrite("name", &Dog::name);
}
```

```
import mymodule
print(mymodule.add(3, 4))   # 7
d = mymodule.Dog("Rex")
print(d.bark())              # "Rex says: Woof!"
```

This is how NumPy, PyTorch, OpenCV, and most high-performance Python libraries work — C++ core, Python wrapper.

eBPF — Programmable Kernel

eBPF (extended Berkeley Packet Filter) lets you run sandboxed programs inside the Linux kernel without modifying kernel source or loading kernel modules. It's used for performance tracing, security monitoring, and networking.

From C++, you write eBPF programs in restricted C, compile to eBPF bytecode, and load them using `bpf()` syscall or the **libbpf** library:

```
// bpf_prog.c — compiled with clang to BPF target
#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>

SEC("tracepoint/syscalls/sys_enter_write")
int trace_write(struct trace_event_raw_sys_enter* ctx) {
    bpf_printk("write called, fd=%d\n", ctx->args[0]);
    return 0;
}

char LICENSE[] SEC("license") = "GPL";
```

```
clang -O2 -target bpf -c bpf_prog.c -o bpf_prog.o
```

The C++ userspace loader attaches and reads data:

```
#include <bpf/libbpf.h>

struct bpf_object* obj = bpf_object__open("bpf_prog.o");
bpf_object__load(obj);
// attach to tracepoint, read from ring buffer...
```

Tools like **bpftrace**, **BCC**, and **Cilium** are built on eBPF. It's the modern way to observe and control Linux kernel behavior.

C++ and Go Interoperability

Go can call C (and thus C++) via CGo:

```
// Go file
package main

/*
#include "mylib.h"
```

```
#cgo LDFLAGS: -L. -lmylib
*/
import "C"

func main() {
    result := C.compute(3, 4)
    println(int(result))
}
```

C++ calling Go requires exporting Go functions with `//export`:

```
//export go_function
func go_function(x C.int) C.int {
    return x * 2
}
```

Then compile Go to a C archive (`go build -buildmode=c-archive`) and link with the C++ program. This is how mixed Go/C++ systems like CockroachDB work.

Key Takeaways

- C++ is backward compatible with C. Use `extern "C"` to prevent name mangling when C code must call C++.
- **pybind11** wraps C++ for Python — how NumPy, PyTorch, and OpenCV expose their C++ cores.
- **eBPF** runs sandboxed C programs inside the Linux kernel for tracing, security, and networking.
- Go C++ interop via CGo + `extern "C"` — used in mixed-language systems.

Appendices

Appendix A: Setting Up Your Toolchain

Linux (Ubuntu/Debian)

```
# Install GCC and G++
sudo apt update
sudo apt install build-essential

# Install Clang (recommended for better error messages)
sudo apt install clang clang-format clang-tidy

# Install CMake (the standard C++ build system)
sudo apt install cmake
```

```
# Check versions:
g++ --version
clang++ --version
cmake --version
```

macOS

```
# Install Xcode Command Line Tools (provides clang):
xcode-select --install

# Install GCC and CMake via Homebrew:
brew install gcc cmake ninja
```

Windows

Option 1 — **MSVC** (Microsoft's compiler, best Windows integration): - Install Visual Studio Community (free) from visualstudio.microsoft.com - Select "Desktop Development with C++"

Option 2 — **WSL2 + GCC/Clang** (Linux development environment on Windows):

```
wsl --install         # installs Ubuntu
# then follow Linux instructions above
```

Option 3 — **MinGW-w64** (GCC for Windows natively):

```
winget install mingw  # or download from mingw-w64.org
```

Choosing Your Compiler

GCC: best standards compliance, good diagnostics, best on Linux. **Clang**: best error messages (most beginner-friendly), required for Xcode on Mac. **MSVC**: best Windows debugging experience, required for some Windows APIs.

For learning: **use Clang** — its error messages are the clearest.

Compiling Manually

```
# Single file:
g++ -std=c++23 -Wall -Wextra -O2 -o output main.cpp

# Multiple files:
g++ -std=c++23 -Wall -Wextra -O2 -o output main.cpp util.cpp

# Common flags:
# -std=c++23   : use C++23 standard (also: c++20, c++17)
# -Wall        : enable all common warnings
# -Wextra      : enable extra warnings
# -O0          : no optimization (debug, default)
# -O2          : optimize (use for release)
```

```
# -O3          : aggressive optimization
# -g           : include debug symbols
# -fsanitize=address : AddressSanitizer (detect memory errors)
# -fsanitize=undefined : UBSanitizer (detect undefined behavior)
```

CMake

CMake is the standard build system for C++ projects. `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.20)
project(MyProject VERSION 1.0 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 23)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(my_program
    src/main.cpp
    src/util.cpp
)

target_include_directories(my_program PRIVATE include)

# Add a library:
add_library(mylib STATIC src/mylib.cpp)
target_include_directories(mylib PUBLIC include)
target_link_libraries(my_program PRIVATE mylib)
```

```
mkdir build && cd build
cmake -DCMAKE_BUILD_TYPE=Release ..
make -j$(nproc)     # parallel build
./my_program
```

Sanitizers (Essential for Development)

Run your program with sanitizers during development to catch bugs that otherwise produce silent undefined behavior:

```
g++ -std=c++23 -g -fsanitize=address,undefined -o program main.cpp
./program
# AddressSanitizer reports: use-after-free, buffer overflow, stack overflow
# UBSanitizer reports: signed integer overflow, null dereference, alignment issues
```

Always develop with sanitizers. Only disable them for performance benchmarking.

Appendix B: Compiler Flags Reference

Warning Flags

```
-Wall                   # Enable "all" common warnings
-Wextra                 # Extra warnings (more than -Wall)
-Wpedantic              # Strict ISO C++ compliance
-Wconversion            # Warn on implicit type conversions
-Wshadow                # Warn when a variable shadows an outer one
-Wnon-virtual-dtor      # Warn on non-virtual destructors in polymorphic classes
-Wold-style-cast        # Warn on C-style casts
-Woverloaded-virtual    # Warn when a derived class hides a virtual function

# Treat warnings as errors (recommended):
-Werror
```

Optimization Flags

```
-O0     # No optimization (default) — best for debugging
-O1     # Basic optimizations
-O2     # Recommended for release — good balance
-O3     # Aggressive optimizations (may increase code size)
-Os     # Optimize for code size
-Og     # Optimize for debuggability (better than -O0 for debug builds)
-Ofast  # -O3 + unsafe math optimizations (may break IEEE 754)

# Link-time optimization (cross-file inlining):
-flto   # compile and link with LTO
```

Debug and Profiling Flags

```
-g      # Generate debug info (DWARF format on Linux)
-g3     # Maximum debug info (includes macros)
-ggdb   # Debug info optimized for GDB

-pg     # gprof profiling instrumentation
-fno-omit-frame-pointer  # Keep frame pointers (needed for perf/flamegraphs)
```

Sanitizer Flags

```
-fsanitize=address          # AddressSanitizer: buffer overflows, use-after-free
-fsanitize=memory           # MemorySanitizer: uninitialized reads (Clang only)
-fsanitize=undefined        # UBSan: undefined behavior
-fsanitize=thread           # ThreadSanitizer: data races
-fsanitize=leak             # LeakSanitizer: memory leaks

# Typical dev build:
-g -fsanitize=address,undefined -fno-omit-frame-pointer
```

Architecture-Specific

```
-march=native       # Optimize for current machine (not portable binary)
-march=x86-64-v3    # AVX2 baseline (good for modern machines)
-msse4.2            # Enable SSE4.2
-mavx2              # Enable AVX2 (256-bit SIMD)
-mavx512f           # Enable AVX-512 (512-bit SIMD)

# Position-Independent Code (needed for shared libraries):
-fPIC
```

Recommended Build Configurations

```
# Debug:
g++ -std=c++23 -g -O0 -Wall -Wextra -fsanitize=address,undefined main.cpp

# Release:
g++ -std=c++23 -O2 -DNDEBUG -Wall -Wextra main.cpp

# Release + LTO:
g++ -std=c++23 -O2 -flto -DNDEBUG main.cpp

# Profiling:
g++ -std=c++23 -O2 -g -fno-omit-frame-pointer main.cpp
```

Appendix C: Python → C++ Idiom Cheat Sheet

Python	C++
`x = 5`	`int x = 5;` or `auto x = 5;`
`x = 5.0`	`double x = 5.0;`
`x = "hello"`	`std::string x = "hello";`
`x = [1,2,3]`	`std::vector<int> x = {1,2,3};`
`x = (1, 2, 3)`	`auto x = std::tuple{1, 2, 3};`
`x = {1,2,3}` (set)	`std::set<int> x = {1,2,3};`
`x = {"a":1}` (dict)	`std::unordered_map<std::string,int> x = {{"a",1}};`
`len(x)`	`x.size()`
`x.append(v)`	`x.push_back(v)`
`x.pop()`	`x.pop_back()`
`x[i]`	`x[i]` (no bounds check) or `x.at(i)`
`x[1:4]` (slice)	`std::vector<int>(x.begin()+1, x.begin()+4)`
`for v in x:`	`for (auto& v : x)`
`for i,v in enumerate(x):`	`for (auto [i,v] : std::views::enumerate(x))` (C++23)

Python	C++
`if x is None:`	`if (x == nullptr):` or `if (!x)` (smart ptr)
`None`	`nullptr` (pointers) or `std::nullopt` (optional)
`lambda x: x*2`	`[](int x){ return x*2; }`
`map(fn, lst)`	`std::ranges::transform` or `|` `std::views::transform(fn)`
`filter(fn, lst)`	`std::ranges::copy_if` or `|` `std::views::filter(fn)`
`sorted(lst)`	`std::ranges::sort(lst)` (in-place)
`sum(lst)`	`std::accumulate(lst.begin(), lst.end(), 0)`
`print(x)`	`std::cout << x << "\n";` or `std::println("{}",x)` (C++23)
`f"{x} {y}"`	`std::format("{} {}", x, y)`
`class Foo:`	`class Foo { public: ... };`
`def __init__(self):`	`Foo() { }` (constructor)
`def __del__(self):`	`~Foo() { }` (destructor)
`self.x`	`this->x` or just `x` in member function
`isinstance(x, T)`	`dynamic_cast<T*>(x) != nullptr`
`try: ... except E:`	`try { ... } catch (const E& e) { ... }`
`raise ValueError("msg")`	`throw std::invalid_argument("msg");`
`with open(f) as h:`	`std::ifstream h(f);` (RAII — auto closes)
`import module`	`#include "module.h"`
`from mod import fn`	`using mod::fn;` or just use `mod::fn()`
`a if cond else b`	`cond ? a : b`
`a // b`	`a / b` (when both are ints)
`a ** b`	`std::pow(a, b)`
`not x`	`!x`
`x and y`	`x && y`
`x or y`	`x || y`

Appendix D: Common Mistakes and How to Debug Them

1. Uninitialized Variables

```cpp
int x;
if (x > 0) ...  // BUG: x has garbage value — undefined behavior
```

Fix: Always initialize. `int x = 0;` or `int x{};`. **Detect**: Compile with `-Wall`. Run with `-fsanitize=memory` (Clang).

2. Out-of-Bounds Array Access

```
int arr[5];
arr[5] = 10;  // BUG: one past the end — undefined behavior
```

Fix: Use `at()` on vectors and arrays. Check indices. **Detect**: `-fsanitize=address` (AddressSanitizer).

3. Use After Free

```
int* p = new int(42);
delete p;
std::cout << *p;  // BUG: use after free — undefined behavior
```

Fix: Set pointer to `nullptr` after delete. Use smart pointers. **Detect**: `-fsanitize=address`.

4. Memory Leak

```
void leak() {
    int* p = new int[1000];
    if (error) return;  // forgets to delete[] p
    delete[] p;
}
```

Fix: Use `std::vector` or `std::unique_ptr` instead of raw arrays. **Detect**: `-fsanitize=leak` or Valgrind (`valgrind --leak-check=full ./prog`).

5. Dangling Reference

```
int& bad() {
    int local = 42;
    return local;  // returns reference to local — destroyed on return!
}
int& r = bad();  // r refers to destroyed memory
```

Fix: Never return references to local variables. Return values or use smart pointers. **Detect**: `-Wall -Wextra` sometimes catches this. Sanitizers help.

6. Integer Overflow

```
int a = 2'000'000'000;
int b = a + a;  // BUG: signed overflow — undefined behavior
```

Fix: Use `int64_t` or check before overflow. **Detect**: `-fsanitize=undefined`.

7. Slicing

```
class Base { virtual void f(); };
class Derived : public Base { int extra; };
```

```
Derived d;
Base b = d;     // BUG: slices — copies only the Base part, discards extra
b.f();          // calls Base::f, not Derived::f
```

Fix: Use pointers or references for polymorphism. Never copy polymorphic objects by value. **Detect**: Code review. `-Wvirtual-move-assign` can warn in some cases.

8. Missing `virtual` Destructor

```
class Base { ~Base() {} };
class Derived : public Base { int* data = new int[100]; ~Derived() { delete[] data; } };

Base* p = new Derived();
delete p;  // BUG: calls ~Base() only — data is leaked
```

Fix: Make `~Base()` `virtual`. **Detect**: `-Wnon-virtual-dtor`.

9. Race Condition

```
int counter = 0;
std::jthread t([](){ for(int i=0;i<1e6;++i) ++counter; });  // BUG: race on counter
```

Fix: Use `std::atomic<int>` or `std::mutex`. **Detect**: `-fsanitize=thread` (ThreadSanitizer).

10. Wrong Delete

```
int* arr = new int[10];
delete arr;   // BUG: should be delete[]
```

Fix: Match `new` with `delete`, `new[]` with `delete[]`. Or use `std::vector`. **Detect**: `-fsanitize=address` often catches this.

GDB Quick Reference

```
g++ -g -O0 -o prog main.cpp   # compile with debug symbols, no optimization
gdb ./prog

(gdb) run                     # run the program
(gdb) bt                      # backtrace — show call stack on crash
(gdb) b main.cpp:42           # set breakpoint at line 42
(gdb) n                       # next line (step over)
(gdb) s                       # step into function
(gdb) p variable_name         # print variable
(gdb) info locals             # print all local variables
(gdb) watch variable_name     # break when variable changes
(gdb) q                       # quit
```

Reading a Crash (Segfault)

```
# 1. Compile with debug symbols:
g++ -g -O0 -fsanitize=address -o prog main.cpp

# 2. Run — AddressSanitizer prints exactly where the bug is:
./prog
# ERROR: AddressSanitizer: heap-use-after-free on address 0x602000000050
# READ of size 4 at 0x602000000050 thread T0
#     #0 0x4012ab in main /home/user/prog.cpp:15
```

AddressSanitizer is almost always the fastest way to find memory bugs. Enable it by default in your debug builds.

End of Modern C++ for Python Programmers.
